

DBA级SQLServer数据库教程

Hello 小伙伴们大家好！在开发这条道路上，数据库是我们应用型系统开发者必备的技术了。我们朝夕教育给大家带来了关于SqlServer数据的完整教程。本教程会包含多个阶段来学习，从零基础到高级进阶，再到DBA的全套教程，这里教会包含多个阶段，在第一个阶段中，我们给大家带来了以下内容。

DBA级SQLServer数据库教程

DBA级SQLServer数据库教程

一、环境安装

- 1、Windows安装SqlServer
- 2、Linux安装SqlServer
- 3、基于Docker安装SqlServer
- 4、基于docker安装后测试
- 5、sqlcmd 实用工具安装
- 6、使用sqlcmd 工具链接
- 7、Win下SQL Server管理工具连接

二、SqlServer用户权限

三、SqlServer基本操作

五、进阶查询

六、视图

七、同义词

八、数据类型

九、T-SQL编程

十、存储过程

十一、函数

十二、触发器

十三、自定义类型

十四、序列

十五、约束

十六、索引

十七、游标

十八、数据库备份

十九、误操作恢复

二十、SqlServer事务

二十一、SqlServer锁

二十二、数据库设计

二十三、执行计划

二十四、性能优化

DBA级SQLServer数据库教程

欢迎大家来到朝夕教育给大家专供的DBA级SQLServer数据库教程。

本教程由朝夕教育科技有限公司专供。

我是朝夕教育的Richard老师

一、环境安装

1. 版本：SQL Server 2019
2. 服务器引擎安装
3. 客户端操作工具安装

1、Windows安装SqlServer

安装步骤

1. 安装服务器引擎
2. 安装客户端工具
3. 下载地址：<https://www.microsoft.com/zh-cn/sql-server/sql-server-downloads>
4. 安装管理工具

无法下载安装包问题

关于SqlServer安装工具下载不了，下载地址无法访问问题的解决方案

1. 访问 <https://tool.chinaz.com/dns/?type=1&host=aka.ms&ip=>
2. 输入无法打开的网站域名
3. 点击检测，找到TTL值最小的那个
4. 打开本地：C:\Windows\System32\drivers\etc\hosts下的Hosts文件
5. 输入TTL值最小的 IP 和域名对应即可
6. 保存Hosts文件

2、Linux安装SqlServer

微软官方指导文档：<https://docs.microsoft.com/zh-cn/sql/linux/sql-server-linux-release-notes-2019?view=sql-server-ver15#supported-platforms>

支持的平台

平台	文件系统	安装指南	获取
Red Hat Enterprise Linux 7.7-7.9 或 8.0-8.5 服务器	XFS 或 EXT4	安装指南	获取 RHEL 8.0
SUSE Enterprise Linux Server v12 (SP3 - SP5) 或 v15	XFS 或 EXT4	安装指南	获取 SLES v15
Ubuntu 16.04 LTS、18.04 LTS、20.04 LTS	XFS 或 EXT4	安装指南	Ubuntu 20.04
适用于 Windows、Mac 或 Linux 的 Docker 引擎 1.8 及更高版本	空值	安装指南	获取 Docker

系统要求

	要求
内存	2 GB
文件系统	XFS 或 EXT4（其他文件系统均不受支持，如 BTRFS）。
磁盘空间	6 GB
处理器速度	2 GHz
处理器核心数	2 个核心
处理器类型	仅兼容 x64

注册

```
subscription-manager register --username=18672713698@163.com --password=1030499676wang --auto-attach --force
```

Linux查看操作系统信息

#查看Linux系统信息

```
cat /etc/*-release
```

#查看版本

```
cat /proc/version
```

查看文件系统信息

#查看文件系统信息

```
df -Th
```

```
lsblk
```

CPU信息

#查看cpu信息

```
lscpu
```

#查看内存

```
free -m
```

3、基于Docker安装SqlServer

系统要求要求

- 1、Linux系统-CentOS7
- 2、远程工具MobaXterm
- 3、必须docker 引擎 1.8 及更高版本

SqlServer安装先决条件

- 2、以下内存和处理器要求适用于所有版本的 SQL Server:

组件	要求
硬盘	SQL Server 要求最少 6 GB 的可用硬盘空间。磁盘空间要求将随所安装的 SQL Server 组件不同而发生变化。有关详细信息，请参阅本文后面部分的 硬盘空间要求 。有关支持的数据文件存储类型的信息，请参阅 Storage Types for Data Files 。
监视	SQL Server 要求有 Super-VGA (800x600) 或更高分辨率的显示器。
Internet	使用 Internet 功能需要连接 Internet（可能需要付费）。
内存*	最低要求： Express Edition: 512 MB 所有其他版本: 1 GB 推荐： Express Edition: 1 GB 所有其他版本: 至少 4 GB，并且应随着数据库大小的增加而增加来确保最佳性能。
处理器速度	最低要求： x64 处理器: 1.4 GHz 推荐： 2.0 GHz 或更快
处理器类型	x64 处理器: AMD Opteron、AMD Athlon 64、支持 Intel EM64T 的 Intel Xeon，以及支持 EM64T 的 Intel Pentium IV

开始安装

1、拉取SqlServer2019镜像

```
sudo docker pull mcr.microsoft.com/mssql/server:2019-latest
```

2、准备数据库文件的挂在目录--/root/docker_volume/mssql 同时给目录设置权限

```
chmod -R 777 /root/docker_volume/mssql
```

2.启动一个容器

```
sudo docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=Sa123456" -p 1433:1433 --name sql1 --hostname sql1 -d mcr.microsoft.com/mssql/server:2019-latest
```

```
sudo docker run -e "ACCEPT_EULA=Y" -e "SA_PASSWORD=Sa123456" \
  -p 1433:1433 --name sql1 --hostname sql1 \
  -d \
  -v /root/docker_volume/mssql:/var/opt/mssql \
  mcr.microsoft.com/mssql/server:2019-latest
```

docker run 的参数说明

参数	说明
<code>-e "ACCEPT_EULA=Y"</code>	将 <code>ACCEPT_EULA</code> 变量设置为任意值，以确认接受最终用户许可协议。SQL Server 映像的必需设置。
<code>-e "SA_PASSWORD=Sa123456"</code>	指定至少包含 8 个字符且符合 SQL Server 密码要求 的强密码。SQL Server 映像的必需设置。
<code>-p 1433:1433</code>	将主机环境中的 TCP 端口（第一个值）映射到容器中的 TCP 端口（第二个值）。在此示例中，SQL Server 侦听容器中的 TCP 1433，此容器端口随后会对主机上的 TCP 端口 1433 公开。
<code>--name sql1</code>	为容器指定一个自定义名称，而不是使用随机生成的名称。如果运行多个容器，则无法重复使用相同的名称。
<code>--hostname sql1</code>	用于显式设置容器主机名。如果未指定主机名，则主机名默认为容器 ID，这是随机生成的系统 GUID。
<code>-d</code>	在后台运行容器（守护程序）。
<code>mcr.microsoft.com/mssql/server:2019-latest</code>	SQL Server Linux 容器映像。

3、要查看 Docker 容器，请使用 `docker ps` 命令。

```
sudo docker ps -a
```

4、基于docker安装后测试

连接到 SQL Server

下列步骤在容器内部使用 SQL Server 命令行工具 `sqlcmd` 来连接 SQL Server。

1、使用 `docker exec -it` 命令在运行的容器内部启动交互式 Bash Shell。 在下面的示例中，`sql1` 是在创建容器时由 `--name` 参数指定的名称。

```
sudo docker exec -it sql1 "bash"
```

2、在容器内部使用完整路径通过 `sqlcmd` 进行本地连接。

```
/opt/mssql-tools/bin/sqlcmd -S localhost -U SA -P "Sa123456"
```

3、如果成功，应会显示 sqlcmd 命令提示符： `1>`。

注意防火墙和端口问题

1、启动防火墙就必须手动放行1433端口、或者关闭防火墙

#查看防火墙状态

```
firewall-cmd --state
```

#开启防火墙

```
systemctl start firewalld.service
```

#查看某个端口是否放行

```
firewall-cmd --query-port=端口号``/tcp
```

#指定放行某个端口

```
firewall-cmd --zone=public --add-port=1433/tcp --permanent
```

#重启防火墙

```
systemctl restart firewalld.service
```

#重新载入配置

```
firewall-cmd --reload
```

创建数据库和表

以下步骤创建一个名为 `TestDB` 的新数据库。

1、在 sqlcmd 命令提示符中，粘贴以下 Transact-SQL 命令以创建测试数据库：

```
CREATE DATABASE TestDB;
```

2、在下一行中，编写一个查询以返回服务器上所有数据库的名称：

```
SELECT Name from sys.databases;
```

3、前两个命令没有立即运行。 在新行中键入 `GO` 以运行前面的命令：

```
GO
```

插入数据

1、在 sqlcmd 命令提示符中，将上下文切换到新的 `TestDB` 数据库

```
USE TestDB;
```

2、创建名为 `Inventory` 的新表

```
CREATE TABLE Inventory (id INT, name NVARCHAR(50), quantity INT);
```

3、将数据插入新表

```
INSERT INTO Inventory VALUES (1, 'banana', 150); INSERT INTO Inventory VALUES (2, 'orange', 154);
```

4、要运行上述命令的类型 `GO`

```
GO
```

查询数据

1、通过 `sqlcmd` 命令提示符输入查询，以返回 `Inventory` 表中数量大于 152 的行

```
SELECT * FROM Inventory WHERE quantity > 152;
```

2、运行以下命令：

```
GO
```

退出 `sqlcmd` 命令提示符

1、要结束 `sqlcmd` 会话，请键入 `QUIT`：

```
QUIT
```

2、要在容器中退出交互式命令提示，请键入 `exit`。退出交互式 Bash Shell 后，容器将继续运行

```
exit
```

5、`sqlcmd` 实用工具安装

在Windows上安装Salcmd工具

下载地址：<https://docs.microsoft.com/zh-cn/sql/tools/sqlcmd-utility?view=sql-server-ver15>

6、使用`sqlcmd` 工具链接

从容器外连接

还可以从支持 SQL 连接的任何 Linux、Windows 或 macOS 外部工具连接到 Docker 计算机上的 SQL Server 实例。以下步骤在容器外使用 **sqlcmd** 连接在容器中运行的 SQL Server。这些步骤假定你已在容器外安装了 SQL Server 命令行工具。使用其他工具时，同样的原则依然适用，但连接过程因工具而异。

- 1、使用 `ifconfig` 或 `ip addr` 查找容器主机的 IP 地址。
- 2、对于本示例，请在客户端计算机上安装 **sqlcmd** 工具。有关详细信息，请参阅 [在 Windows 上安装 sqlcmd](#) 或 [在 Linux 上安装 sqlcmd](#)
- 3、运行 `sqlcmd`，指定 IP 地址和映射容器中的端口 1433 的端口。本例中为主机上的相同端口 1433。如果在主机上指定了不同的映射端口，则在此处使用它。还需要在防火墙上打开相应的入站端口以允许连接

```
sqlcmd -S 192.168.3.97,1433 -U SA -P Sa123456
```

7、Win下SQL Server管理工具连接

- 1、输入用户名 sa
- 2、输入密码 Sa123456
- 3、连接数据库引擎

安装客户端Microsoft SQL Server Management Studio 18

二、SqlServer用户权限

设置权限用户的意义

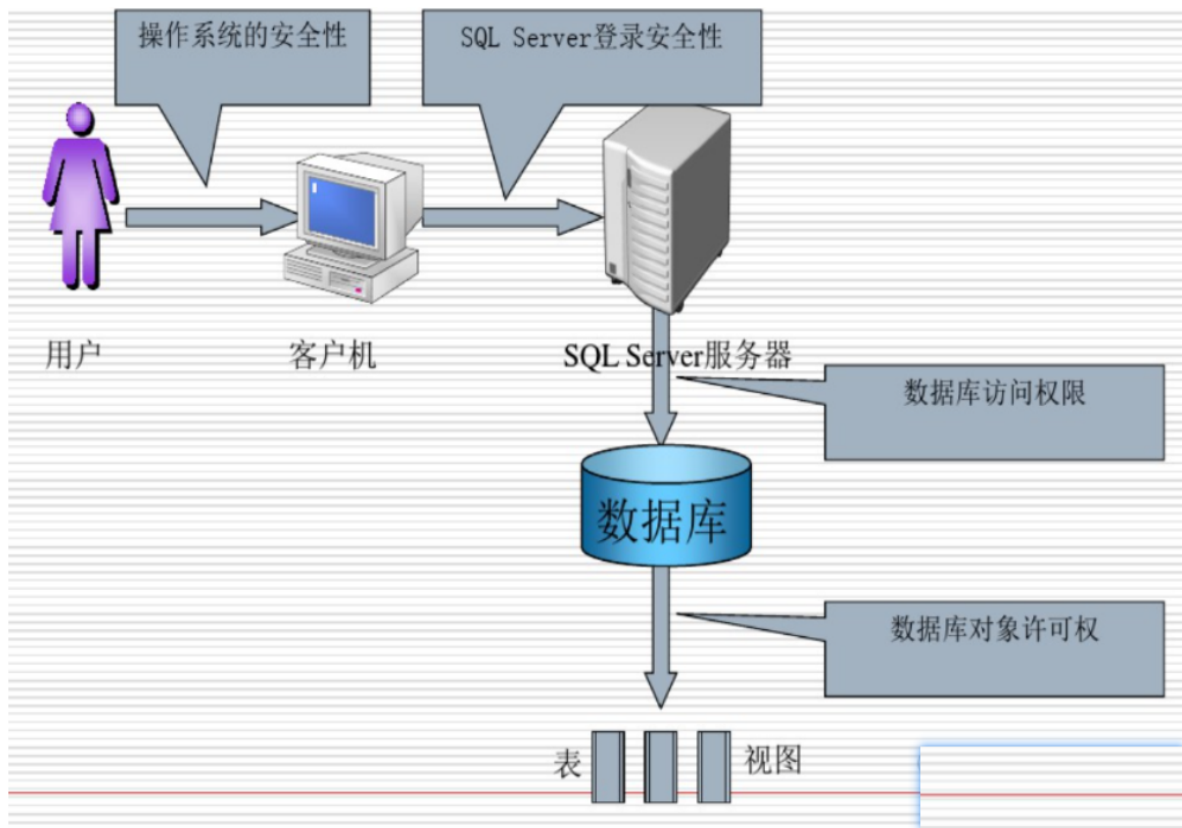
数据库是一个应用程序，运行起来对应一个进程，这个进程中，有很多很强大的功能支持，对于开发者或者是数据库管理者，都是充分的来应用这些功能，应用这些功能，需要客户端/或者是应用程序链接进来，通过客户端/应用程序来操作数据库。操作客户端来使用数据库功能的最终是人在使用。为了保证数据库的安全性，必须对于数据库操作者有不同的权限控制。

权限

权：能做什么事儿，能使用数据库的哪些功能？

限：限制不能做哪些事儿。

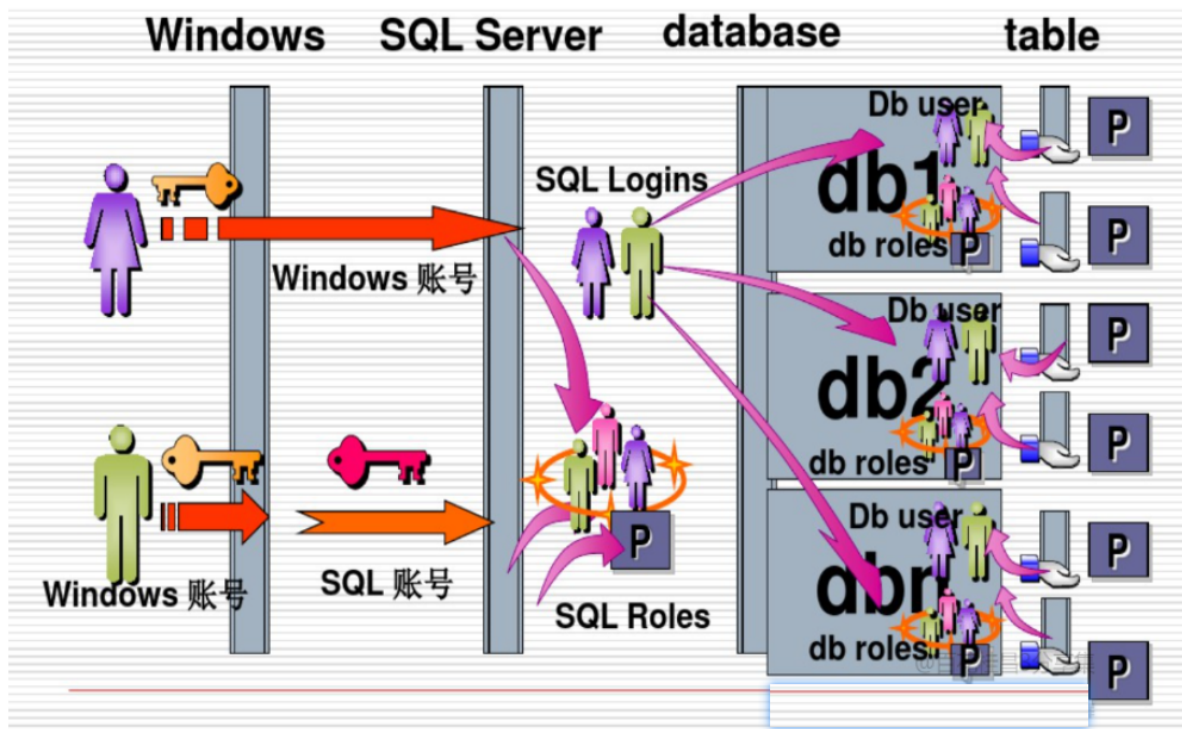
如何做到权限管理呢？通过不同的用户，不同的用户角色来分配。这样管控，就可以通过登录数据库的用户不同，赋予相应的权利，和限制相应的操作。



SqlServer登录方式

- 1、Windows 身份验证方式，
- 2、SqlServer身份验证方式：

Windows 身份验证方式，该用户具备最高权限，仅能SQLSERVER安装所在的服务器登录；
SQLSERVER 身份验证方式（sa用户），一般会授予该用户最高权限，可以在同一网络环境下的任何电脑上登录。出于这样或那样的原因，有时需要对外开放接口，给对方一个账号，又希望限制对方访问操作权限的时候，就需要设置权限用户。



用户登录，最终目的是为了能够控制登录的用户，在数据库的很多功能中，只能操作某一部分功能；
用户+用户角色+每一个用户角色可以做哪些事儿；从而得到不同的用户可以做不同的事儿；

SqlServer服务器角色

服务器级的固定角色	说明
sysadmin	sysadmin 服务器角色的成员可以在服务器上执行任何活动。
serveradmin	serveradmin 服务器角色的成员可以更改服务器范围的配置选项和关闭服务器。
securityadmin	securityadmin 服务器角色的成员可以管理登录名及其属性。他们可以 GRANT、DENY 和 REVOKE 服务器级权限。他们还可以 GRANT、DENY 和 REVOKE 数据库级权限（如果他们具有数据库的访问权限）。此外，他们还可以重置 SQL Server 登录名的密码。 重要提示： 如果能够授予对数据库引擎的访问权限和配置用户权限，安全管理员可以分配大多数服务器权限。 securityadmin 角色应视为与 sysadmin 角色等效。
processadmin	processadmin 服务器角色的成员可以终止在 SQL Server 实例中运行的进程。
setupadmin	setupadmin 服务器角色的成员可以使用 Transact-SQL 语句添加或删除链接服务器。（使用 Management Studio 时需要 sysadmin 成员资格。）
bulkadmin	bulkadmin 服务器角色的成员可以运行 BULK INSERT 语句。Linux 上的 SQL Server 不支持 bulkadmin 角色或管理 BULK OPERATIONS 权限。只有 sysadmin 才能对 Linux 上的 SQL Server 执行批量插入。
diskadmin	diskadmin 服务器角色用于管理磁盘文件。
dbcreator	dbcreator 服务器角色的成员可以创建、更改、删除和还原任何数据库。
public	每个 SQL Server 登录名都属于 public 服务器角色。如果未向服务器主体授予或拒绝对安全对象的特定权限，用户将继承授予对该对象的 公共 权限。只有在希望所有用户都能使用对象时，才在对象上分配 Public 权限。不能更改公共成员身份。注意：public 与其他角色的实现方式不同，可通过 public 固定服务器角色授予、拒绝或撤销权限。

有时需要对外开放接口，给对方一个账号，又希望限制对方访问操作权限的时候，就需要设置权限用户。

SqlServer固定数据库角色

下表显示了固定数据库角色及其能够执行的操作。所有数据库中都有这些角色。无法更改分配给固定数据库角色的权限，“公共”数据库角色除外。

固定数据库角色名	说明
db_owner	db_owner 固定数据库角色的成员可以执行数据库的所有配置和维护活动，还可以 drop SQL Server 中的数据库。（在 SQL 数据库 和 Synapse Analytics 中，某些维护活动需要服务器级别权限，并且不能由 db_owners 执行。）
db_securityadmin	db_securityadmin 固定数据库角色的成员可以仅修改自定义角色的角色成员资格和管理权限。此角色的成员可能会提升其权限，应监视其操作。
db_accessadmin	db_accessadmin 固定数据库角色的成员可以为 Windows 登录名、Windows 组和 SQL Server 登录名添加或删除数据库访问权限。
db_backupoperator	db_backupoperator 固定数据库角色的成员可以备份数据库。
db_ddladmin	db_ddladmin 固定数据库角色的成员可以在数据库中运行任何数据定义语言 (DDL) 命令。
db_datawriter	db_datawriter 固定数据库角色的成员可以在所有用户表中添加、删除或更改数据。
db_datareader	db_datareader 固定数据库角色的成员可以从所有用户表和视图中读取所有数据。用户对象可能存在于除 sys 和 INFORMATION_SCHEMA 以外的任何架构中。
db_denydatawriter	db_denydatawriter 固定数据库角色的成员不能添加、修改或删除数据库内用户表中的任何数据。
db_denydatareader	db_denydatareader 固定数据库角色的成员不能读取数据库内用户表和视图中的任何数据。

角色---归属于用户

- 1、用户登录---服务器判断用户属于是什么角色，根据觉得能做的事儿，分配给当前用户能做的事儿；
- 2、可以做到给用户配置角色

三、SqlServer基本操作

1、工具建库建表

2、脚本建库建表

```
USE [master]
GO
CREATE DATABASE [ZhaoXiEdu]
    CONTAINMENT = NONE
    ON PRIMARY
    ( NAME = N'ScoreInfo', FILENAME = N'D:\ScoreInfo.mdf' , SIZE = 8192KB , MAXSIZE = UNLIMITED, FILEGROWTH = 65536KB )
    LOG ON
    ( NAME = N'ScoreInfo_log', FILENAME = N'D:\ScoreInfo_log.ldf' , SIZE = 8192KB , MAXSIZE = 2048GB , FILEGROWTH = 65536KB )
    WITH CATALOG_COLLATION = DATABASE_DEFAULT
GO
```

```
GO
CREATE TABLE [dbo].[ScoreInfo](
    [id] [int] IDENTITY(1,1) NOT NULL,
    [name] [varchar](255) NULL,
    [course] [varchar](255) NULL,
    [score] [int] NULL
) ON [PRIMARY]
GO
```

3、工具删除数据库删除表

4、脚本删除数据库删除表

```
USE [master]
GO

if db_id('ZhaoXiEdu') is not null

Drop database    [ZhaoXiEdu]
```

```
USE [master]
GO
if EXISTS (SELECT * from sysobjects WHERE name='ScoreInfo')

Drop table    [ScoreInfo]
```

数据基本CRUD

对于表中数据的操作

1、查询

```
use ZhaoxiEdu
```

```
SELECT 字段名称  
      , 字段名称  
      , 字段名称  
      , 字段名称  
FROM 表名称
```

```
SELECT *  
FROM 表名称
```

2、新增

```
use ZhaoxiEdu
```

```
GO  
insert into ScoreInfo(name, course, score) values ('张三','高级班', 80)  
insert into ScoreInfo(name, course, score) values ('张三','架构班', 52)  
insert into ScoreInfo(name, course, score) values ('张三','上位机班', 150)  
insert into ScoreInfo(name, course, score) values ('李四','高级班', 44)  
insert into ScoreInfo(name, course, score) values ('李四','数学', 111)  
insert into ScoreInfo(name, course, score) values ('李四','英语', 110)  
insert into ScoreInfo(name, course, score) values ('王五','高级班', 140)  
insert into ScoreInfo(name, course, score) values ('王五','数学', 80)  
insert into ScoreInfo(name, course, score) values ('王五','英语', 92)  
insert into ScoreInfo(name, course, score) values ('王五','物理', 77)  
insert into ScoreInfo(name, course, score) values ('王五','化学', 65)
```

3、修改

```
use ZhaoxiEdu
```

```
GO  
UPDATE [dbo].[ScoreInfo]  
      SET [name] = <name, varchar(255),>  
      , [course] = <course, varchar(255),>  
      , [score] = <score, int,>  
WHERE <搜索条件,,>
```

4、删除

```
USE [ZhaoXiEdu]  
GO  
  
DELETE FROM [dbo].[ScoreInfo]  
      WHERE <搜索条件,,>  
GO
```

五、进阶查询

1、别名，查询结果拼接

```
SELECT TOP (1000) [Id]      主键
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo]
```

2、条件查询

```
SELECT TOP (1000) [Id]      主键
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where id=4
```

3、范围查询

```
SELECT TOP (1000) [Id]      主键
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where id>5 and id<10
```

```
SELECT TOP (1000) [Id]      主键
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where id between 4 and 10
```

4、null判断

```
SELECT TOP (1000) [Id]      主键
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where course is null
```

5、查询前多少行/按比例查询结果

```
select top 3 * from ScoreInfo
select top(20) percent * from ScoreInfo
```

6、case when 判断

#案例1
#分数有个范围：<90 不及格
 90—120--及格
 >120 良好

```
SELECT TOP (1000) [Id]
      , [name]
      , [course]
      , [score]
```



```

        ,case when score<90 then '不及格'
            when score>=90 and score<120 then '及格'
            when score>=120 and score<130 then '良好'
            when score>=130 then '优秀'
            else '鸭蛋'
            end as '等级'
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] order by [score] asc

```

#案例2

```

SELECT TOP (1000) [Id]
        , [name]
        , [course]
        , [score]
        , case course when '高级班' then '走高高级开发'
            when '架构班' then '走向架构师'
            when '全栈班' then '走向全栈开发'
            end as '开发方向'
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] order by [score] asc

```

7、in查询

```

SELECT TOP (1000) [Id]
        , [name]
        , [course]
        , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where id in (4,5,6,7,8)

```

8、like查询

模糊匹配--和通配符%一起使用才有效果 %表示可以匹配任何字符

```

SELECT TOP (1000) [Id]
        , [name]
        , [course]
        , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where [name] like '张%'

```

9、with关键字查询

相当于是sql查询中的sql片段

```

with score as (SELECT * FROM [ZhaoxiEdu].[dbo].[ScoreInfo] )
select * from score where [name] ='张三'

```

10、子查询/exists关键字查询

#子查询

```

SELECT * FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where id in (SELECT Id FROM [ZhaoxiEdu].[dbo].[ScoreInfo]
where [name]='张三')

```

#exists关键字

```

SELECT * FROM [ZhaoxiEdu].[dbo].[ScoreInfo] t1 where
exists(SELECT * FROM [ZhaoxiEdu].[dbo].[ScoreInfo] t2 where t1.Id=t2.Id and t2.[name]='张三')

```

11、复制新表/表数据复制

```
#复制新表
select * into ScoreInfo2 from ScoreInfo

#把另外一个结构相同的表数据复制到指定表中
insert [ZhaoxiEdu].[dbo].[ScoreInfo2] select [name]
      , [course]
      , [score] from [ZhaoxiEdu].[dbo].[ScoreInfo2]
```

12、distinct 同一列去掉重复

```
#普通查询某一列
SELECT score FROM [ZhaoxiEdu].[dbo].[ScoreInfo2] order by score desc

#查询某一列去掉重复
SELECT distinct(score) FROM [ZhaoxiEdu].[dbo].[ScoreInfo2] order by score desc
```

13、排序

升序asc 降序desc 可以多列排序 从左往右优先级

```
with ttt as ( SELECT max(id) as id, [name], course, score FROM [ZhaoxiEdu].[dbo].[ScoreInfo2] group
by [name], course, score)
select * from ttt order by [course], score desc
```

14、聚合查询分组

聚合---一般在汇总的时候需要用到

```
#汇总---每个人的总分
SELECT
      [name]
      , sum([score]) score
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] group by [name]
```

15、分页查询

#必须带有主键Id，且主键Id是标识列，必须是自增的

原理：需要拿出数据库的第5页，就是40-50条记录。首先拿出数据库中的前40条记录的id值，然后再拿出剩余部分的前10条元素

```
declare @pagesize int;
select @pagesize=5;

declare @pageindex int;
select @pageindex=3;

select top (@pagesize) *
from ScoreInfoNew2
where id not in
(
      --(10*(2-1)) 为页大小 * (当前第几页-1)
      select top (@pagesize*(@pageindex-1)) id from ScoreInfoNew2 order by id
)
order by id
```

#分页查询二

原理：先查询前40条记录，然后获得其最id值，如果id值为null的，那么就返回0
然后查询id值大于前40条记录的最大id值的记录。
这个查询有一个条件，就是id必须是int类型的。

```
declare @pagesize int;
select @pagesize=2;

declare @pageindex int;
select @pageindex=1;

select top (@pagesize) *
from ScoreInfo
where id >
(
    select isnull(max(id),0)
    from
    (
        --(10*(2-1)) 为页大小 * (当前第几页-1)
        select top ((@pagesize)*(@pageindex)-1)) id from ScoreInfo order by id
    ) A
)
order by id
```

#分页查询三

原理：先把表中的所有数据都按照一个rowNumber进行排序，然后查询rownumber大于40的前十条记录
这种方法和oracle中的一种分页方式类似，不过只支持2005版本以上的

```
declare @pagesize int;
select @pagesize=10;

declare @pageindex int;
select @pageindex=1;

select top (@pagesize) *
from
(
    select row_number() over(order by id) as rownumber,* from ScoreInfo
) A

--(10*(2-1)) 为页大小 * (当前第几页-1)
where rownumber > ((@pagesize)*(@pageindex)-1))
```

#分页查询四

要求必须在SqlServer2012版本之后方可支持

```
declare @pagesize int;
select @pagesize=2;

declare @pageindex int;
select @pageindex=2;

SELECT *
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] order by Id offset (@pagesize*(@pageindex-1)) --间隔多条条开始
rows fetch next (@pagesize) --获取多少条
rows only
```

16、20、union/union all操作

可以把查询到的多个数据结构完全相同个表，合并起来

union: 自动去重 union all 不会去掉重复

```
SELECT TOP (1000) [Id]
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where [name]='张三' union all
SELECT TOP (1000) [Id]
      , [name]
      , [course]
      , [score]
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] where [name]='张三'
```

17、行转列/列转行

```
#行转列-----
SELECT [name],
       isnull(sum(case course when '高级班' then score end),0) as '高级班',
       isnull(sum(case course when '架构班' then score end),0) as '架构班',
       isnull(sum(case course when '上位机班' then score end),0) as '上位机班',
       isnull(sum(case course when 'Web前端' then score end),0) as 'Web前端',
       isnull(sum(case course when '全栈班' then score end),0) as '全栈班'
FROM [ZhaoxiEdu].[dbo].[ScoreInfo] group by [name]

#初始化数据表脚本-----
SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[RowColumnConversion](
    [Id] [int] IDENTITY(1,1) NOT NULL,
    [Name] [varchar](50) NULL,
    [Advanced] [int] NULL,
    [Framework] [int] NULL,
    [UpperComputer] [int] NULL,
    [Web] [int] NULL,
    [FullStack] [nchar](10) NULL,
    CONSTRAINT [PK_RowColumnConversion] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO

#列转行-----
select row_number() over(order by id desc) as id, [name], t.course, t.score from(
    select id, [name], course='高级班', score=[Advanced] from [ZhaoxiEdu].[dbo].[RowColumnConversion]
    union all
```

```

select id,[name],course='架构班',score=Framework from [ZhaoxiEdu].[dbo].[RowColumnConversion]
union all
select id,[name],course='上位机班',score=UpperComputer from [ZhaoxiEdu].[dbo].[RowColumnConversion]
union all
select id,[name],course='Web前端',score=Web from [ZhaoxiEdu].[dbo].[RowColumnConversion]
union all
select id,[name],course='全栈班',score=FullStack from [ZhaoxiEdu].[dbo].[RowColumnConversion]
) t where score is not null order by id asc
,case t.course when '语文' then 1 when '数学' then 2 when '英语' then 3 when '物理' then 4 when '化学'
then 5 end

with tt as(
select id, [Name],course='高级班',score=[Advanced] FROM [ZhaoxiEdu].[dbo].[RowColumnConversion]
Union all
select id,[Name],course='架构班',score=[Framework] FROM [ZhaoxiEdu].[dbo].[RowColumnConversion]
Union all
select id,[Name],course='上位机',score=[UpperComputer] FROM [ZhaoxiEdu].[dbo].[RowColumnConversion]
Union all
select id,[Name],course='Web前端',score=[Web] FROM [ZhaoxiEdu].[dbo].[RowColumnConversion]
Union all
select id,[Name],course='全栈班',score=[FullStack] FROM [ZhaoxiEdu].[dbo].[RowColumnConversion])

select * from tt where score<>0 order by [Name] desc

```

18、连接查询

多个表通过字段之间的关系进行关联，在关联后，通过查询得到我们想要的数据库。

数据准备--创建Company表--初始化数据

```

Use ZhaoxiEdu

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Company](
    [Id] [int] IDENTITY(1,1) NOT FOR REPLICATION NOT NULL,
    [Name] [varchar](50) NULL,
    [CreateTime] [datetime] NULL,
    [CreatorId] [int] NOT NULL,
    [LastModifierId] [int] NULL,
    [LastModifyTime] [datetime] NULL,
    CONSTRAINT [PK_Company] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO

GO
SET IDENTITY_INSERT [dbo].[Company] ON
GO

```

```

INSERT [dbo].[Company] ([Id], [Name], [CreateTime], [CreatorId], [LastModifierId], [LastModifyTime])
VALUES (1, N'朝夕教育', CAST(N'2021-10-09T16:37:14.350' AS DateTime), 1, 1, CAST(N'2021-10-09T16:37:14.350' AS DateTime))
GO
INSERT [dbo].[Company] ([Id], [Name], [CreateTime], [CreatorId], [LastModifierId], [LastModifyTime])
VALUES (2, N'阿里巴巴', CAST(N'2021-10-09T16:37:14.717' AS DateTime), 1, 1, CAST(N'2021-10-09T16:37:14.717' AS DateTime))
GO
INSERT [dbo].[Company] ([Id], [Name], [CreateTime], [CreatorId], [LastModifierId], [LastModifyTime])
VALUES (3, N'百度', CAST(N'2021-10-09T16:37:14.717' AS DateTime), 1, 1, CAST(N'2021-10-09T16:37:14.717' AS DateTime))
GO
INSERT [dbo].[Company] ([Id], [Name], [CreateTime], [CreatorId], [LastModifierId], [LastModifyTime])
VALUES (4, N'腾讯', CAST(N'2021-10-09T16:37:14.717' AS DateTime), 1, 1, CAST(N'2021-10-09T16:37:14.717' AS DateTime))
GO

SET IDENTITY_INSERT [dbo].[Company] OFF

```

数据准备--创建SysUser表-初始化数据

```

Use ZhaoxiEdu

SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[SysUser](
    [Id] [int] IDENTITY(1,1) NOT FOR REPLICATION NOT NULL,
    [Name] [varchar](50) NULL,
    [CompanyId] [int] NULL,
    [Password] [varchar](50) NULL,
    [Status] [int] NOT NULL,
    [Phone] [varchar](12) NULL,
    [Mobile] [varchar](12) NULL,
    [Address] [varchar](500) NULL,
    [Email] [varchar](50) NULL,
    [QQ] [bigint] NULL,
    [WeChat] [varchar](50) NULL,
    [Sex] [int] NULL,
    [LastLoginTime] [datetime] NULL,
    [CreateTime] [datetime] NULL,
    [CreateId] [int] NULL,
    [LastModifyTime] [datetime] NULL,
    [LastModifyId] [int] NULL
CONSTRAINT [PK_SysUser] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO

GO
SET IDENTITY_INSERT [dbo].[SysUser] ON
GO

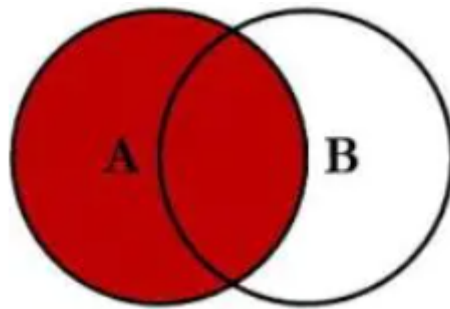
```

```

INSERT [dbo].[SysUser] ([Id], [Name], [Password], [Status], [Phone], [Mobile], [Address], [Email],
[QQ], [WeChat], [Sex], [LastLoginTime], [CreateTime], [CreateId], [LastModifyTime], [LastModifyId],
[CompanyId]) VALUES (1, N'张三', N'123456', 1, N'18672713698', N'18672713698', N'武汉市',
N'18672713698@163.com', 123456789, NULL, 0, CAST(N'2020-12-15T00:00:00.000' AS DateTime), CAST(N'2020-
12-15T00:00:00.000' AS DateTime), 1, CAST(N'2020-12-15T00:00:00.000' AS DateTime), 1, 1)
GO
INSERT [dbo].[SysUser] ([Id], [Name], [Password], [Status], [Phone], [Mobile], [Address], [Email],
[QQ], [WeChat], [Sex], [LastLoginTime], [CreateTime], [CreateId], [LastModifyTime], [LastModifyId],
[CompanyId]) VALUES (3, N'李四', N'123456', 1, N'18672713698', N'18672713698', N'武汉市',
N'18672713698@163.com', 123456789, NULL, 0, CAST(N'2020-12-15T00:00:00.000' AS DateTime), CAST(N'2020-
12-15T00:00:00.000' AS DateTime), 1, CAST(N'2020-12-15T00:00:00.000' AS DateTime), 1, 1)
GO
INSERT [dbo].[SysUser] ([Id], [Name], [Password], [Status], [Phone], [Mobile], [Address], [Email],
[QQ], [WeChat], [Sex], [LastLoginTime], [CreateTime], [CreateId], [LastModifyTime], [LastModifyId],
[CompanyId]) VALUES (5, N'王五', N'Pass', 1, N'PHone', N'18672713698', N'abcc', N'', 123456798,
N'Wechat', 1, CAST(N'2021-10-09T15:03:43.673' AS DateTime), CAST(N'2021-10-09T15:03:43.673' AS
DateTime), 1, CAST(N'2021-10-09T15:03:43.677' AS DateTime), 1, 1)
GO
INSERT [dbo].[SysUser] ([Id], [Name], [Password], [Status], [Phone], [Mobile], [Address], [Email],
[QQ], [WeChat], [Sex], [LastLoginTime], [CreateTime], [CreateId], [LastModifyTime], [LastModifyId],
[CompanyId]) VALUES (6, N'赵六', N'Pass', 1, N'PHone', N'18672713698', N'abcc', N'', 123456798,
N'Wechat', 1, CAST(N'2021-10-09T15:11:06.473' AS DateTime), CAST(N'2021-10-09T15:11:06.473' AS
DateTime), 1, CAST(N'2021-10-09T15:11:06.473' AS DateTime), 1, 1)
GO
INSERT [dbo].[SysUser] ([Id], [Name], [Password], [Status], [Phone], [Mobile], [Address], [Email],
[QQ], [WeChat], [Sex], [LastLoginTime], [CreateTime], [CreateId], [LastModifyTime], [LastModifyId],
[CompanyId]) VALUES (7, N'添加', N'Pass', 1, N'PHone', N'18672713698', N'abcc', N'', 123456798,
N'Wechat', 1, CAST(N'2021-10-09T16:01:45.010' AS DateTime), CAST(N'2021-10-09T16:01:45.010' AS
DateTime), 1, CAST(N'2021-10-09T16:01:45.010' AS DateTime), 1, 1)
GO
SET IDENTITY_INSERT [dbo].[SysUser] OFF

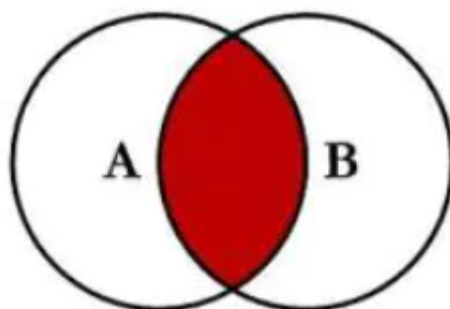
```

左连接



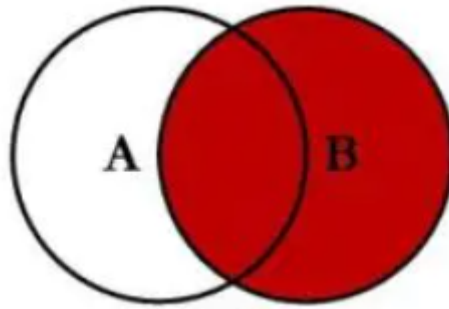
```
select * from Company c left join SysUser u on c.Id =u.CompanyId
```

19、内连接查询



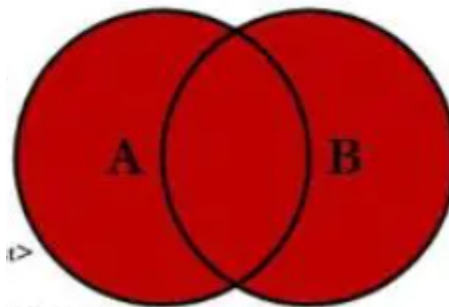
```
select * from Company c inner join SysUser u on c.Id =u.CompanyId
```

20、右连接查询



```
select * from Company c right join SysUser u on c.Id =u.CompanyId
```

21、全连接



```
select * from Company c full join SysUser u on c.Id =u.CompanyId
```

21、递归查询

可以通过一个参数，可以查询到无限层级下的数据，带有无限的层级查询。

初始化数据

```
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[MenuInfo](
    [Id] [int] IDENTITY(1,1) NOT NULL,
    [MenuName] [varchar](40) NULL,
    [ParentId] [int] NULL,
    CONSTRAINT [PK_MenuInfo] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO
/***** Object: Table [dbo].[RowColumnConversion]    Script Date: 2022/8/5 10:38:03 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
```



```

SET IDENTITY_INSERT [dbo].[MenuInfo] ON
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (1, N'一级菜单', NULL)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (2, N'一级-二级菜单-1', 1)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (3, N'一级-二级菜单-2', 1)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (4, N'一级-二级菜单-3', 1)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (5, N'一级-二级菜单-1-三级菜单-1', 2)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (6, N'一级-二级菜单-1-三级菜单-2', 2)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (7, N'一级-二级菜单-1-三级菜单-3', 2)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (8, N'一级-二级菜单-2-三级菜单-1', 3)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (9, N'一级-二级菜单-2-三级菜单-2', 3)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (10, N'一级-二级菜单-2-三级菜单-3', 3)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (11, N'一级-二级菜单-3-三级菜单-1', 4)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (12, N'一级-二级菜单-3-三级菜单-2', 4)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (13, N'一级-二级菜单-3-三级菜单-3', 4)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (14, N'一级-二级菜单-3-三级菜单-1-四级菜单-1', 11)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (15, N'一级-二级菜单-3-三级菜单-1-四级菜单-2', 11)
GO
INSERT [dbo].[MenuInfo] ([Id], [MenuName], [ParentId]) VALUES (16, N'一级-二级菜单-3-三级菜单-1-四级菜单-3', 11)
GO
SET IDENTITY_INSERT [dbo].[MenuInfo] OFF

```

希望能够根据一个父菜单，查询出当前菜单下的所有的子菜单；

递归语句

就是去查询层级结构的数据存储，查询数据树形结构数据。

```

with Con(Id, MenuName, ParentId, le) as
(
    select Id, MenuName, ParentId, le=1 from [ZhaoxiEdu].[dbo].[MenuInfo] WHERE Id=4
    union all
    select a. Id, a. MenuName, a. ParentId, le=le+1 from [ZhaoxiEdu].[dbo].[MenuInfo] a join Con on
    a. ParentId=con. Id
)
select Id, MenuName, ParentId, le from Con

```

六、视图

视图核心价值

在数据查询中，可以看到数据表设计过程中，考虑到数据的冗余度低、数据一致性问题，通常对数据表的设计要满足范式的要求，因此也会造成一个实体的所有信息保存在多个表中。当检索数据时，往往在一个表中不能够得到想要的所有信息。为了解决这种矛盾，在SQL Server中提供了视图。

视图的概念

视图是一种数据库对象，是从一个或者多个数据表或视图中导出的虚表，视图的结构和数据是对数据表进行查询的结果；

只存放视图的定义，不存放视图对应的数据；

基表中的数据发生变化，从视图中查询出的数据也随之改变。

创建视图

数据库管理工具创建

Sql脚本创建

视图查询，修改、删除，增加

推荐大家在使用视图的时候，仅用作查询。

不推荐大家基于视图来做增删改

七、同义词

1、同义词定义

同义词是用来实现下列用途的数据库对象：

- 为可以存在于本地或远程服务器上的其他数据库对象（称为基对象）提供备用名称---有点像别名。
- 提供抽象层以免对客户端应用程序基对象的名称或位置进行更改。

2、案例

多个数据库服务器连接查询。数据库存在于不同的对象中。

1、通过链接服务器对象来完成。

2、如果要查询会很不方便，尤其是在多关联查询的时候。

可以把其他SqlServer服务器上的数据库定义成一个同义词。直接链接同义词查询即可。

3、管理工具创建同义词

1. 在“**对象资源管理器**”中，展开要创建新视图的数据库。
2. 右键单击“同义词”文件夹，然后选择“新建同义词...”。
3. 在“**添加同义词**”对话框中，输入以下信息。

同义词名称

键入将用于此对象的新名称。

同义词架构

键入将用于此对象的新名称的架构。

服务器名称

键入要连接到的服务器实例。

数据库名称

键入或选择包含该对象的数据库。

架构

键入或选择该对象所属的架构。

对象类型

选择对象的类型。

对象名称

键入同义词所引用的对象的名称。

4、可以为哪些数据库对象创建同义词

1. 表
2. 视图
3. 函数
4. 存储过程

5、脚本创建同义词

表

```
USE [ZhaoxiEdu]
GO
/***** Object:  Synonym [dbo].[myCompany]      Script Date: 2022/8/9 10:53:53 *****/
CREATE SYNONYM [dbo].[myCompany] FOR [ZhaoxiEdu].[dbo].[Company]
GO
```

视图

```
USE [ZhaoxiEdu]
GO
/***** Object:  Synonym [dbo].[mycompanyUserSynonym]      Script Date: 2022/8/9 10:59:33 *****/
CREATE SYNONYM [dbo].[mycompanyUserSynonym] FOR [ZhaoxiEdu].[dbo].[CompanyUserView]
GO
```

存储过程

```
USE [ZhaoxiEdu]
```

```
GO
```

```
/***** Object: Synonym [dbo].[PageSpSynonym] Script Date: 2022/8/9 11:07:51 *****/
```

```
CREATE SYNONYM [dbo].[PageSpSynonym] FOR [ZhaoxiEdu].[dbo].[SP_CustomPager]
```

```
GO
```

八、数据类型

bigint、int、smallint、tinyint

使用整数数据的精确数字数据类型。若要节省数据库空间，请使用能够可靠包含所有可能值的最小数据类型。例如，对于一个人的年龄，tinyint 就足够了，因为没人活到 255 岁以上。但对于建筑物的年龄，tinyint 就不再适应，因为建筑物的年龄可能超过 255 年。

数据类型	范围	存储
bigint	-2 ⁶³ (-9,223,372,036,854,775,808) 到 2 ⁶³ -1 (9,223,372,036,854,775,807)	8 字节
int	-2 ³¹ (-2,147,483,648) 到 2 ³¹ -1 (2,147,483,647)	4 个字节
smallint	-2 ¹⁵ (-32,768) 到 2 ¹⁵ -1 (32,767)	2 字节
tinyint	0 到 255	1 字节

bit

SQL Server 数据库引擎优化位列的存储。如果表中的 bit 列为 8 列或更少，则这些列作为 1 个字节存储。如果 bit 列为 9 到 16 列，则这些列作为 2 个字节存储，以此类推。字符串值 TRUE 和 FALSE 可转换为 **bit** 值：TRUE 将转换为 1，FALSE 将转换为 0。

性别

bool值

numeric和decimal

decimal[(p[,s])] 和 numeric[(p[,s])]

固定精度和小数位数。使用最大精度时，有效值的范围为 - 10³⁸ + 1 到 10³⁸ - 1。decimal 的 ISO 同义词为 dec 和 dec(p,s)。numeric 在功能上完全等同于 decimal。高精度，小数点后面最多可以有38位

金额---精度很高，小数点后面可能有很多位；后面的小数点精度可以达到38位

smallmoney和money

小数点存储都为4位，如果出现第5位，根据第5位做四舍五入。

数据类型	范围	存储
money	-922,337,203,685,477.5808 到 922,337,203,685,477.5807 (	8 个字节
smallmoney	-214,748.3648 到 214,748.3647	4 个字节

float和real

这两个类型保存的是一个近似值。保存的数据，可能和真是要保存的数据有细微的偏差。

注意：数据筛选的时候，只适合> >= < <= 不适合去等于

float [(n)] 其中 n 为用于存储 float 数值尾数的位数（以科学记数法表示），因此可以确定精度和存储大小。如果指定了 n，则它必须是介于 1 和 53 之间的某个值。n 的默认值为 53。

n 值	Precision	存储大小
1-24	7 位数	4 个字节
25-53	15 位数	8 字节

date类型 说明

date类型可用于需要一个日期值而不需要时间部分时。

properties	值
语法	date
使用情况	DECLARE @MyDate date CREATE TABLE Table1 (Column1 date)
默认的字符串文字格式（用于下级客户端）	YYYY-MM-DD 有关详细信息，请参阅后面的“下级客户端的向后兼容性”部分。
范围	0001-01-01 到 9999-12-31（对于 Informatica，为 1582-10-15 到 9999-12-31） 公元 1 年 1 月 1 日（公历纪元）到公元 9999 年 12 月 31 日（对于 Informatica，公元 1582 年 10 月 15 日到公元 9999 年 12 月 31 日）
各元素的范围	YYYY 是表示年份的四位数字，范围为从 0001 到 9999。对于 Informatica，YYYY 限为 1582 年到 9999 年。MM 是表示指定年份中的月份的两位数字，范围为从 01 到 12。DD 是表示指定月份几号的两位数字，介于 01 和 31 之间（具体视月份而定）。
字符长度	10 位
精度、小数位数	10, 0
存储大小	固定 3 个字节
存储结构	1、3 字节整数存储日期。
精确度	一天
默认值	1900-01-01 此值用于从 time 隐式转换到 datetime2 或 datetimeoffset 时追加的日期部分。
日历	公历
用户定义的秒的小数部分精度	否
时区偏移量感知和保留	否
夏时制感知	否

datetime 说明

可用于需要同时包含日期和时间信息的值。

properties	值
语法	datetime
使用情况	DECLARE @MyDatetime datetime CREATE TABLE Table1 (Column1 datetime)
默认的字符串文字格式 (用于下级客户端)	不适用
日期范围	1753 年 1 月 1 日到 9999 年 12 月 31 日
时间范围	00:00:00 到 23:59:59.997
时区偏移量范围	无
各元素的范围	YYYY 是表示年份的四位数字，范围为 1753 到 9999。MM 是表示指定年份中的月份的两位数字，范围为 01 到 12。DD 是表示指定月份中的某一天的两位数字，范围为 01 到 31（最高值取决于相应月份）。hh 是表示小时的两位数字，范围为 00 到 23。mm 是表示分钟的两位数字，范围为 00 到 59。ss 是表示秒钟的两位数字，范围为 00 到 59。n* 为一个 0 到 3 位的数字，范围为 0 到 999，表示秒的小数部分。
字符长度	最低 19 位到最高 23 位
存储大小	8 个字节
精确度	舍入到 .000、.003 或 .007 秒三个增量。
默认值	1900-01-01 00:00:00
日历	公历（包括完整的年份范围。）
用户定义的秒的小数部分精度	否
时区偏移量感知和保留	否
夏时制感知	否

time 说明

属性	值
语法	time [(fractional second scale)]
使用情况	DECLARE @MyTime time(7) CREATE TABLE Table1 (Column1 time(7))
fractional seconds scale	为秒的小数部分指定数字的位数。这可以是 0 到 7 的整数。对于 Informatica，这可以是 0 到 3 的整数。默认小数位数为 7 (100ns)。
默认的字符串文字格式（用于下级客户端）	对于 Informatica，为 hh:mm:ss[.nnnnnnnn] 有关详细信息，请参阅 下级客户端的后向兼容性 部分。
范围	00:00:00.00000000 到 23:59:59.99999999（对于 Informatica，为 00:00:00.000 到 23:59:59.999）
各元素的范围	hh 是表示小时的两位数字，范围为 0 到 23。mm 是表示分钟的两位数字，范围为 0 到 59。ss 是表示秒的两位数字，范围为 0 到 59。n* 是 0 到 7 位数字，范围为 0 到 9999999，它表示秒的小数部分。对于 Informatica，n* 是零到三位数字，范围为 0 到 999。
字符长度	最小 8 位 (hh:mm:ss)，最大 16 位 (hh:mm:ss.nnnnnnnn)。对于 Informatica，最大值为 12 位 (hh:mm:ss.nnn)。
精度、小数位数（用户只能指定小数位数）	请参阅下表。
存储大小	固定 5 个字节，是使用默认的 100ns 秒的小数部分精度时的默认存储大小。在 Informatica 中，默认为 4 个字节，固定不变，同时秒的小数部分精度默认为 1 毫秒。
精确度	100 纳秒（Informatica 中为 1 毫秒）
默认值	00:00:00 此值用作从 date 隐式转换到 datetime2 或 datetimeoffset 时追加的时间部分。
用户定义的秒的小数部分精度	是
时区偏移量感知和保留	否
夏时制感知	否

属性	值
语法	datetime2 [(fractional seconds precision)]
使用情况	DECLARE @MyDatetime2 datetime2(7) CREATE TABLE Table1 (Column1 datetime2(7))
默认的字符串文字格式 (用于下级客户端)	YYYY-MM-DD hh:mm:ss[.fractional seconds] 有关详细信息, 请参阅后面的“下级客户端的向后兼容性”部分。
日期范围	0001-01-01 到 31.12.99 公元 1 年 1 月 1 日到公元 9999 年 12 月 31 日
时间范围	00:00:00 到 23:59:59.9999999
时区偏移量范围	无
各元素的范围	YYYY 是一个四位数, 范围从 0001 到 9999, 表示年份。MM 是一个两位数, 范围从 01 到 12, 它表示指定年份中的月份。DD 是一个两位数, 范围为 01 到 31 (具体取决于月份), 它表示指定月份中的某一天。hh 是一个两位数, 范围从 00 到 23, 它表示小时。mm 是一个两位数, 范围从 00 到 59, 它表示分钟。ss 是一个两位数, 范围从 00 到 59, 它表示秒钟。n* 代表 0 到 7 位数字, 范围从 0 到 9999999, 它表示秒小数部分。在 Informatica 中, 当 n > 3 时, 秒的小数部分会被截断。
字符长度	最低 19 位 (YYYY-MM-DD hh:mm:ss), 最高 27 位 (YYYY-MM-DD hh:mm:ss.0000000)
精度、小数位数	0 至 7 位, 准确度为 100ns。默认精度为 7 位数。
存储大小 1	精度小于 3 的 6 个字节。精度为 3 和 4 的 6 个字节。所有其他精度需要 8 个字节。2
精确度	100 纳秒
默认值	1900-01-01 00:00:00
日历	公历

属性	值
用户定义的秒的小数部分精度	是
时区偏移量感知和保留	否
夏时制感知	否

smalldatetime 说明

属性	值
语法	smalldatetime
使用情况	DECLARE @MySmalldatetime smalldatetime CREATE TABLE Table1 (Column1 smalldatetime)
默认的字符串文字格式 (用于下级客户端)	不适用
日期范围	1900-01-01 到 2079-06-06 1900 年 1 月 1 日到 2079 年 6 月 6 日
时间范围	00:00:00 到 23:59:59 2007-05-09 23:59:59 将舍入为 2007-05-10 00:00:00
各元素的范围	YYYY 是表示年份的四位数字，范围为 1900 到 2079。MM 是表示指定年份中的月份的两位数字，范围为 01 到 12。DD 是表示指定月份中的某一天的两位数字，范围为 01 到 31（最高值取决于相应月份）。hh 是表示小时的两位数字，范围为 00 到 23。mm 是表示分钟的两位数字，范围为 00 到 59。ss 是表示秒钟的两位数字，范围为 00 到 59。值为 29.998 秒或更小值时会向下舍入到最接近的分钟数。值为 29.999 秒或更大值时会向上舍入到最接近的分钟数。
字符长度	最高 19 位
存储大小	固定 4 个字节。
精确度	一分钟
默认值	1900-01-01 00:00:00
日历	公历（不包括完整的年份范围。）
用户定义的秒的小数部分精度	否

属性	值
时区 偏移 量感 知和 保留	否
夏时 制感 知	否

datetimeoffset 说明

属性	值
语法	datetimeoffset [(fractional seconds precision)]
使用情况	DECLARE @MyDatetimeoffset datetimeoffset(7) CREATE TABLE Table1 (Column1 datetimeoffset(7))
默认字符串文字格式 (用于下级客户端)	YYYY-MM-DD hh:mm:ss[.nnnnnnnn] [{+ -}hh:mm] 。
日期范围	0001-01-01 到 31.12.99 公元 1 年 1 月 1 日到公元 9999 年 12 月 31 日
时间范围	00:00:00 到 23:59:59.9999999
时区偏移量范围	-14:00 到 +14:00
各元素的范围	YYYY 是表示年份的四位数字，范围为 0001 到 9999。MM 是表示指定年份中的月份的两位数字，范围为 01 到 12。DD 是表示指定月份中的某一天的两位数字，范围为 01 到 31（最高值取决于相应月份）。hh 是表示小时的两位数字，范围为 00 到 23。mm 是表示分钟的两位数字，范围为 00 到 59。ss 是表示秒钟的两位数字，范围为 00 到 59。n* 是 0 到 7 位数字，范围为 0 到 9999999，它表示秒的小数部分。hh 是两位数，范围为 -14 到 +14。mm 是两位数，范围为 00 到 59。
字符长度	最低 26 位 (YYYY-MM-DD hh:mm:ss {+ -}hh:mm) 到最高 34 位 (YYYY-MM-DD hh:mm:ss.nnnnnnn {+ -}hh:mm)
精度、小数位数	请参阅下表。
存储大小	默认值为 10 个字节的固定大小，默认的秒的小数部分精度为 100ns。
精确度	100 纳秒
默认值	1900-01-01 00:00:00 00:00
日历	公历

属性	值
用户定义的秒的小数部分精度	是
时区偏移量感知和保留	是
夏时制感知	否

char和varchar

`char ( n )` 固定大小字符串数据。 `n` 用于定义字符串大小（以字节为单位），并且它必须为 1 到 8,000 之间的值。对于单字节编码字符集（如拉丁文），存储大小为 `n` 个字节，并且可存储的字符数也为 `n`。对于多字节编码字符集，存储大小仍为 `n` 个字节，但可存储的字符数可能小于 `n`。 `char` 的 ISO 同义词是 `character`。有关字符集的详细信息，请参阅[单字节和多字节字符集](#)。

`varchar ( n | max )` 可变大小字符串数据。使用 `n` 定义字符串大小（以字节为单位），可以是介于 1 和 8,000 之间的值；或使用 `max` 指明列约束大小上限为最大存储 $2^{31}-1$ 个字节 (2GB)。对于单字节编码字符集（如拉丁文），存储大小为 `n + 2` 个字节，并且可存储的字符数也为 `n`。对于多字节编码字符集，存储大小仍为 `n + 2` 个字节，但可存储的字符数可能小于 `n`。 `varchar` 的 ISO 同义词是 `charvarying` 或 `charactervarying`

nchar 和 nvarchar

nchar (n)

无论存储中文还是英文、数字等，每个字符都是占用两个字节。

固定大小字符串数据。 `n` 用于定义字符串大小（以双字节为单位），并且它必须为 1 到 4,000 之间的值。存储大小为 `n` 字节的两倍。对于 **UCS-2** 编码，存储大小为 `n` 个字节的两倍，并且可存储的字符数也为 `n`。对于 UTF-16 编码，存储大小仍为 `n` 个字节的两倍，但可存储的字符数可能小于 `n`，因为补充字符使用两个双字节（也称为[代理项对](#)）。 `nchar` 的 ISO 同义词是 `national char` 和 `national character`。

nvarchar (n | max)

可变大小字符串数据。 `n` 用于定义字符串大小（以双字节为单位），并且它可能为 1 到 4,000 之间的值。 `max` 指示最大存储大小是 $2^{30}-1$ 个字符 (2 GB)。存储大小为 `n` 字节的两倍 + 2 个字节。对于 **UCS-2** 编码，存储大小为 `n` 个字节的两倍 + 2 个字节，并且可存储的字符数也为 `n`。对于 UTF-16 编码，存储大小仍为 `n` 个字节的两倍 + 2 个字节，但可存储的字符数可能小于 `n`，因为补充字符使用两个双字节（也称为[代理项对](#)）。 `nvarchar` 的 ISO 同义词是 `national char varying` 和 `national character varying`。

n`text`、`text`和`images`

`ntext`

长度可变的 Unicode 数据，字符串最大长度为 $2^{30} - 1$ (1,073,741,823) 个字节。存储大小是所输入字符串长度的两倍（以字节为单位）。

`text`

服务器代码页中长度可变的非 Unicode 数据，字符串最大长度为 $2^{31}-1$ (2,147,483,647) 个字节。当服务器代码页使用双字节字符时，存储仍是 2,147,483,647 字节。根据字符串，存储大小可能小于 2,147,483,647 字节。

`image`

长度可变的二进制数据，从 0 到 $2^{31}-1$ (2,147,483,647) 个字节。

`Binary`和`varbinary`

参数

`binary [(n)]` 长度为 `n` 字节的固定长度二进制数据，其中 `n` 是从 1 到 8,000 的值。存储大小为 `n` 字节。

`varbinary [(n|max)]` 可变长度二进制数据。`n` 的取值范围为 1 至 8,000。`max` 指示最大存储大小是 $2^{31}-1$ 个字节。存储大小为所输入数据的实际长度 + 2 个字节。所输入数据的长度可以是 0 字节。

备注

如果没有在数据定义或变量声明语句中指定 `n`，则默认长度为 1。如果没有使用 `CAST` 函数指定 `n`，则默认长度为 30。

数据类型	何时使用...
<code>binary</code>	列数据项的大小一致。
<code>varbinary</code>	列数据项的大小差异相当大。
<code>varbinary(max)</code>	列数据条目超出 8,000 字节。

`uniqueidentifier` 说明

备注

`uniqueidentifier` 数据类型的列或局部变量可通过以下方式初始化为一个值：---全球唯一的一个值

- 通过使用 **NEWID** 或 **NEWSEQUENTIALID** 函数。
- 通过从 `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` 形式的字符串常量进行转换，其中，每个 `x` 都是 0-9 或 `a-f` 范围内的十六进制数字。例如，`6F9619FF-8B86-D011-B42D-00C04FC964FF` 为有效的 `uniqueidentifier` 值。

比较运算符可与 `uniqueidentifier` 值一起使用。不过，排序不是通过比较两个值的位模式来实现的。可针对 `uniqueidentifier` 值执行的运算只有比较运算（`<>`、`<`、`>`、`<=`、`>=`）以及检查是否为 NULL（`IS NULL` 和 `IS NOT NULL`）。不能使用其他算术运算符。除 `IDENTITY` 之外的所有列约束和属性均可对 `uniqueidentifier` 数据类型使用。

具有更新订阅的合并复制和事务复制使用 `uniqueidentifier` 列来确保在表的多个副本中唯一地标识行。

转换 `uniqueidentifier` 数据

出于从字符表达式转换的目的将 `uniqueidentifier` 类型视为字符类型，因此在转换到字符类型时要遵循截断规则。也即，如果将字符表达式转换为不同大小的字符数据类型，则对于新数据类型而言过长的值将被截断。请参阅“示例”部分。

XML 说明

参数

CONTENT

将 `xml` 实例限制为格式正确的 XML 片段。XML 数据的顶层可包含多个零或多个元素。还允许在顶层使用文本节点。

此选项为默认行为。

DOCUMENT

将 `xml` 实例限制为格式正确的 XML 片段。XML 数据必须且只能有一个根元素。不允许在顶层使用文本节点。

`xml_schema_collection`

XML 架构集合的名称。若要创建类型化的 `xml` 列或变量，可选择指定 XML 架构集合名称。有关类型化和非类型化 XML 的详细信息。

备注

`xml` 数据类型实例所占据的存储空间大小不能超过 2 GB。

`CONTENT` 和 `DOCUMENT` 方面仅应用于类型化的 XML

九、T-SQL编程

通过Sql语法来完成业务的处理，执行编写好的sql语句，就可以完成业务处理。

变量

SQL Server中变量分为：局部变量 和 全局变量

局部变量

局部变量是一个能够拥有特定数据类型的对象，它的作用范围仅限制在程序内部。在批处理和脚本中变量可以有如下用途：作为计数器计算循环执行的次数或控制循环执行的次数，保存数据值供控制流语句测试，以及保存由存储过程代码返回的数据值或者函数返回值。

注意：

数据类型不可以是TEXT、NTEXT、IMAGE类型，局部变量被引用时要在其名称前加上标志 "@", 如果不为局部变量赋值，则默认赋值为null

定义变量:

```
declare @text varchar(20); -- (变量名开始必须是@)
```

赋值:

--方法1:

```
set @text = '你好' -- (不推荐)
```

--方法2:

```
select @text = 'hello'
```

输出内容

```
print @text
```

方法1:

```
select @text --映射到结果集  
select '测试一下~'
```

方法2:

```
print @text --打印到消息框  
print '测试一下~'
```

全局变量

在全局可用，系统预定义，用户不可以定义全局变量，用户不可以修改全局变量，全局变量以@@开头

常用全局变量:

@@CONNECTIONS	返回SQL Server自上次启动以来尝试的连接数，无论连接是成功还是失败
@@CPU_BUSY	返回SQL Server自上次启动后的工作时间。 其结果以CPU时间增量或“滴答数”来表示，此值为CPU工作时间的累积值，因此，可能会超出实际占用CPU的时间。 乘以@@TIMETICKS即可转换为微秒。
@@CURSOR_ROWS	返回连接的数据库上打开的上一个游标中的当前限定行的数目，为了提高性能。 SQL Server可异步填充大型键集和静态游标。可调用@@CURSOR_ROWS以确定当其被调用时检索了游标符合条件的行数。
@@DATEFIRST	针对会话返回 SET DATEFIRST的当前值。
@@DBTS	返回当前数据库的当前 timestamp 数据类型的值。这一时间戳值在数据库中必须是唯一的。
@@ERROR	返回执行的上一个Transact-SQL语句出现错误时对应的错误编号。
@@FETCH_STATUS	返回针对连接的数据库当前打开的任何游标，发出的上一条游标FETCH语句的状态。
@@IDENTITY	返回插入到数据表的IDENTITY列的最后一个值。
@@IDLE	返回SQL Server自上次启动后的空闲时间。结果以CPU时间增量或“时钟周期”来表示，是所有的累积值 因此该值可能超过实际经过的时间。乘以@@TIMETICKS即可转换为微秒。
@@IO_BUSY	返回自 SQL Server最近一次启动以来，SQL Server已经用于执行输入和输出操作的时间。 其结果是CPU时间增量(时钟周期)，是CPU执行操作的累积值，这个值可能超过实际消逝的时间。 乘以@@TIMETICKS即可转换为微秒。
@@LANGID	返回当前使用的语言对应的本地语言标识符(ID)。
@@LANGUAGE	返回当前所用语言的名称。

@@LOCK_TIMEOUT	返回当前会话的锁定超时的设置值(单位为毫秒)。
@@MAX_CONNECTIONS	返回 SQL Server 实例允许同时进行的最大用户连接数。返回的数值不一定是当前配置的数值。
@@MAX_PRECISION	按照服务器中的当前设置，返回decimal和numeric 数据类型所用的精度级别。默认情况下，最大精度级别38。
@@NESTLEVEL	返回在本地服务器上执行的当前存储过程的嵌套级别(初始值为0)。
@@OPTIONS	返回有关当前SET选项的信息。
@@PACK_RECEIVED	返回 SQL Server 自上次启动后从网络读取的输入数据包数。
@@PACK_SENT	返回SQL Server自上次启动后写入网络的输出数据包个数。
@@PACKET_ERRORS	返回自上次启动 SQL Server后，在SQL Server 连接上发生的网络数据包错误数。
@@ROWCOUNT	返回上一次语句影响的数据行的行数。
@@PROCID	返回 Transact-SQL当前模块的对象标识符(ID)。Transact-SQL模块可以是存储过程、用户定义函数或触发器。 不能在CLR模块或进程内的数据访问接口中指定@@PROCID。
@@SERVERNAME	返回运行SQL Server的本地服务器的名称。
@@SERVICENAME	返回SQL Server正在运行的注册表项的名称。若当前实例为默认实例，则@@SERVICENAME返回MSSQLSERVER； 若当前实例是命名实例，则该函数返回该实例名。
@@SPID	返回当前用户进程的会话ID。
@@TEXTSIZE	返回SET语句的TEXTSIZE选项的当前值， 它指定 SELECT 语句返回的text或image数据类型的最大长度，其单位为字节。
@@TIMETICKS	返回每个时钟周期的微秒数。
@@TOTAL_ERRORS	返回自上次启动SQL Server之后，SQL Server所遇到的磁盘写入错误数。
@@TOTAL_READ	返回 SQL Server自上次启动后，由SQL Server 读取(非缓存读取)的磁盘的数目。
@@TOTAL_WRITE	返回自上次启动SQL Server以来，SQL Server所执行的磁盘写入数。
@@TRANCOUNT	返回当前连接的活动事务数。
@@VERSION	返回当前安装的日期、版本和处理器类型。

运算符

运算符	含义
+ （加法）	加法，注意：除了做数字运算，还可以做字符串拼接
- （减法）	减法
* （乘法）	乘法
/ (除法)	除法
% （取模/取余）	返回一个除法的整数余数，注意：两边必须是整数数据，例如12%5=2,这是因为12除以5的余数为2

```

declare @num1 int;
declare @num2 int;
set @num1=12;
set @num2=10;

--数字相加
select @num1+@num2

declare @str1 nvarchar(10);
declare @str2 nvarchar(10);
set @str1=' Richard';
set @str2=' 老师';

```

```
--字符串相加+
select @str1+@str2

--字符串数字相加需要类型转换
select @str2+ convert (varchar(10),@num1)

--数字相减
select @num1-@num2

--数字相乘
select @num1 * @num2

--数字相除
select @num1 / @num2

--数字取余
select @num1 % @num2
```

比较运算符

运算符	含义	运算符	含义
>	大于	<	小于
=	等于	>=	大于等于
<=	小于等于	<>或!=	不等于
!>	不大于	!<	不小于

```
SELECT *
FROM [ZhaoxiEdu].[dbo].[Company] where Createtime>
where Createtime<
where Createtime=
where Createtime>=
where Createtime<=
where Createtime<>或!=
where Createtime!>
where Createtime!<
```

逻辑运算符

运算符	含义
AND 且	如果两个逻辑表达式都为TRUE,那么结果为TRUE
OR 或	如果两个逻辑表达式中的一个为TRUE,那么结果为TRUE
NOT 非	对任何其他逻辑运算符的值取反
BETWEEN	如果操作数在某个范围之内, 那么结果为TRUE
IN	如果操作数等于表达式列表中的一个, 那么结果为TRUE
LIKE	如果操作数与一种模式相匹配, 那么结果为TRUE
ANY/SOME	如果一组的比较中任何一个为TRUE,那么结果为TRUE
ALL	如果一组的比较都为TRUE,那么结果为TRUE
EXISTS	如果子查询包含一些行, 那么结果为TRUE

连接运算符

加号(+)是字符串串接运算符, 可以将两个或两个以上字符串合并串接成一个字符串。其他所有字符串操作则可以调用字符串函数(如SUBSTRING())进行处理。默认情况下, 对于varchar数据类型的数据, 在INSERT或赋值语句中, 空的字符串将被解释为空字符串。在串接varchar、char或text数据类型的数据时, 空的字符串被解释为空字符串。例如: 'abc'+' '+'def'被存储为'abcdef'

表达式

根据连接表达式的运算符进行分类, 可以将表达式分为算术表达式、比较表达式、逻辑表达式、按位运算表达式和混合表达式等; 根据表达式的作用进行分类, 可以将表达式分为字段名表达式、目标表达式和条件表达式。

字段名表达式

字段名表达式可以是单个字段或几个字段的组合, 还可以是由字段、作用于字段的集合函数和常量的任意算术运算(+、-、*、/)组成的运算表达式。主要包括数值表达式、字符表达式、逻辑表达式和日期表达式4种。

目标表达式

目标表达式有4种构成方式。

- (1) :表示选择相应基表和视图的所有字段。
- (2) <表名>. : 表示选择指定的基表和视图的所有字段。
- (3) 集函数(): 表示在相应的表中按集函数操作和运算。
- (4) [<表名>].字段名表达式 [, [<表名> .] <字段名表达式>] . . . 表示按字段名表达式在多个指定的表中选择。

条件表达式

常用的条件表达式有以下6种：

- 1、比较大小 - 应用比较运算符构成表达式，主要的比较运算符有 =、>、>=、<、<=、!=、< (不大于)、> (不小于)、NOT(与比较运算符相同，对条件求非)。
- 2、指定范围 - (NOT)BETWEEN...AND...运算符查找字段值在或者不在指定范围内的记录。BETWEEN后面指定范围的最小值，AND后面指定范围的最大值。
- 3、集合(NOT)IN-查询字段值属于或者不属于指定集合内的记录。
- 4、字符匹配 - (NOT)LIKE<匹配字符串> ' [ESCAPE'<换码字符>] 查找字段值满足 < 匹配字符串 > 中指定匹配条件的记录。 < 匹配字符串 > 可以是一个完整的字符串，也可以包含通配符 "" 和 "%" ， "" 代表任意单个字符， "%" 代表任意长度的字符串。
- 5、空值IS(NOT)NULL-查找字段值为空(不为空)的记录。NULL不能用来表示无形值、默认值、不可用值、以及取最低值或取最高值。SQL规定，在含有运算符 +、-、*、/ 的算术。表达式中，若有一个值是空值，则该算术表达式的值也是空值；任何一个含有NULL比较操作结果的取值都为FALSE。
- 6、多重条件AND和OR-AND表达式用来找出字段值同时满足AND相连接的查询条件的记录。OR表达式用来找出字段值满足OR连接的查询条件中的记录。AND运算符的优先级高于OR运算符。

流程控制

批处理

一条或多条SQL语句的集合，一个批作为一个字符串交给服务器去执行。
使用GO表示批，一个批出现错误，批中的语句都将会被回滚。

```
GO
-- SQL语句
GO
```

分支结构

```
declare @a int;
declare @b int;

select @a=4;
select @b=2;

if @a>@b ---（不用写（））
begin
    print ' @a的值大于@b的值'
end
else
begin
    print ' @b的值大于@a的值'
end
```

begin end 相当于{}

循环

循环语句——类似于C#中的循环，循环四要素：初始条件，循环条件，循环体，状态改变

```

declare @aa int ;
select @aa=1;      --初始条件

while @aa<10
begin
    select @aa as 结果;      --循环体
    print '结果为:' + convert(varchar(10),@aa)
    select @aa=@aa+1; --状态改变
end

```

循环退出

```

BREAK;
CONTINUE;

```

自定义错误处理

```

BEGIN TRY
    -- SQL
END TRY
BEGIN CATCH
    --SQL
END CATCH

```

可以在CATCH中使用以下语句抛出错误:

```

THROW {error_number | @local_variable},
{ message | @local_variable },
{ state | @local_variable };

```

十、存储过程

存储过程的概念:

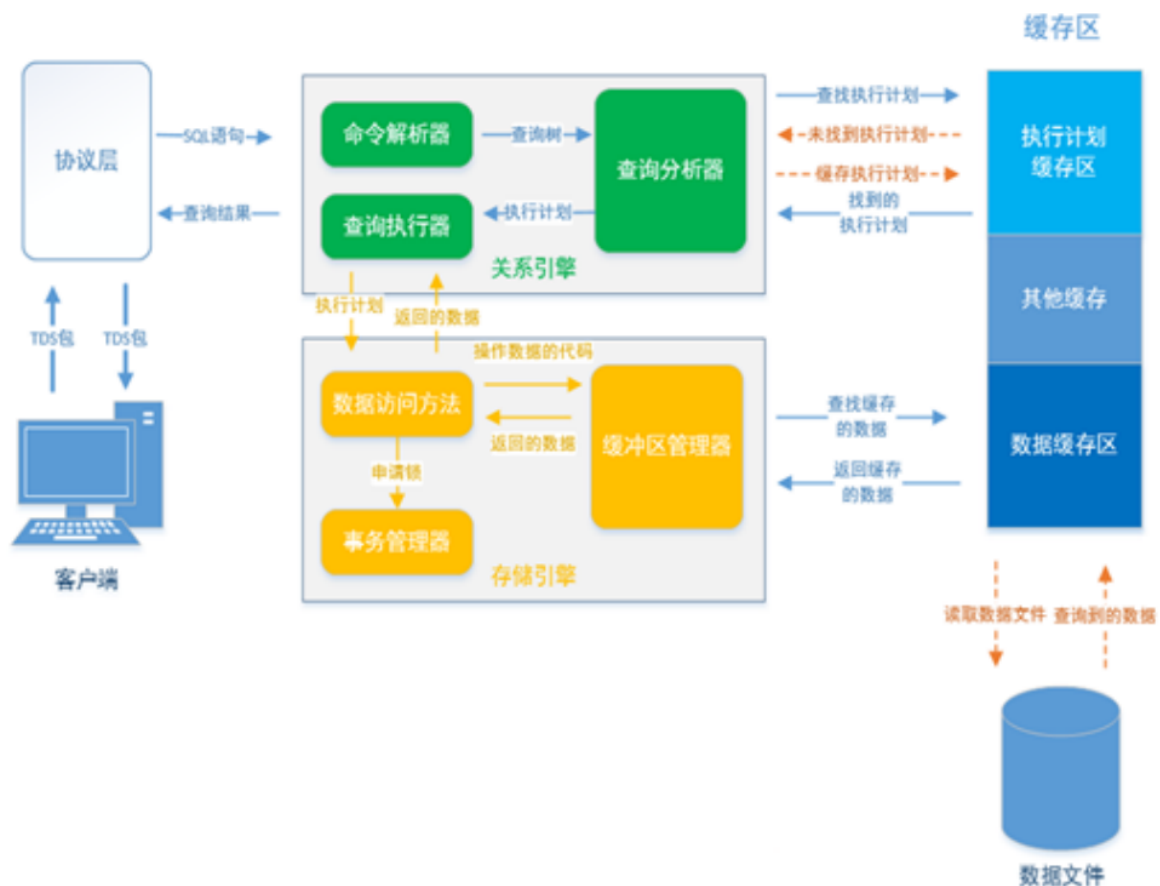
存储过程 (procedure) 是一组为了完成特定功能的SQL语句集合，经编译后存储在数据库中，用户通过指定存储过程的名称并给出参数来执行。

存储过程优点

通过把处理封装在简单易用的单元中，简化复杂的操作存储过程创建后可以在程序中被多次调用执行，而不必重新编写该存储过程的SQL语句。可以针对于某一项业务逻辑处理，把处理过程就定义好。需要处理业务，就直接调用存储过程。

提高性能。使用存储过程比使用单独的SQL语句要快如果某一操作包含大量的SQL语句代码，分别被多次执行，那么存储过程要比批处理的执行速度快得多。

安全,调用者只需要知道如何调用指定的存储过程即可，而不用关心存储过程的内容，防止SQL注入。存储过程减轻网络流量对于同一个针对数据库对象的操作，这一操作所涉及到的T-SQL语句被组织成一存储过程，那么当在客户机上调用该存储过程时，网络中传递的只是该调用语句，否则将会是多条SQL语句。从而减轻了网络流量,降低了网络负载



存储过程缺点

编写复杂

如果没有相应的权限,你将无法创建存储过程

当服务器调用过多存储过程,用户访问量大了,那么压力就丢给数据库来解决,数据库压力会过大过多的存储过程,优化过于麻烦.

常见系统存储过程

存储过程 含义

exec sp_databases; 查看所有数据库

exec sp_helpdb; 查询数据库信息

exec sp_helpdb 数据库名; 查询指定数据库信息

exec sp_renamedb '旧库名', '新库名'; 更改数据库名称

exec sp_tables; 查询当前数据库的所有表

exec sp_columns 表名; 查看列

exec sp_help 表名; 返回表的所有信息

exec sp_helpIndex 表名; 查看索引

exec sp_helpConstraint 表名; 约束

exec sp_stored_procedures; 当前环境的所有存储

exec sp_helptext '存储过程'; 查看存储过程源码

exec sp_rename '旧名', '新名'; 修改表、索引、列的名称

exec sp_defaultdb '旧库名', '新库名'; 更改登录名的默认数据库

注意: exec 用于调用存储过程

自定义存储过程创建语法

```
create proc | procedure 存储名(
    [{@参数 数据类型} [=默认值] [out|output],
    {@参数 数据类型} [=默认值] [out|output],
    ....]
)
as
begin
    SQL_statements
end
go
```

存储过程修改语法

```
alter proc | procedure 存储过程名
as
beign
    sql语句;
end
```

存储过程删除语法

```
drop proc | procedure 存储过程名;
```

调用语法

```
exec 存储过程名;                ---不带参数调用
exec 存储过程名 参数1 out|output, 参数2 out|output;    ---带参数调用
```

分页存储过程

```
DROP PROCEDURE IF EXISTS [dbo].[SP_CustomPager];
GO
CREATE PROCEDURE [dbo].[SP_CustomPager]
    @TableName VARCHAR(50),                --表名
    @ReFieldsStr VARCHAR(200) = '*',        --字段名(全部字段为*)
    @OrderString VARCHAR(200),              --排序字段(必须!支持多字段不用加order by)
    @WhereString VARCHAR(500) =N'',         --条件语句(不用加where)
    @PageSize INT,                          --每页多少条记录
    @PageIndex INT = 1,                     --指定当前为第几页
    @TotalRecord INT OUTPUT                 --返回总记录数
AS
BEGIN
    --处理开始点和结束点
    DECLARE @StartRecord INT;
    DECLARE @EndRecord INT;
    DECLARE @TotalCountSql NVARCHAR(500);
    DECLARE @SqlString NVARCHAR(2000);
    SET @StartRecord = (@PageIndex-1)*@PageSize + 1--起始记录
    SET @EndRecord = @StartRecord + @PageSize - 1 --结尾记录
    SET @TotalCountSql= N' select @TotalRecord = count(*) from ' + @TableName;--总记录数语句
```



```

SET @SqlString = N' (select row_number() over (order by ' + @OrderString + ') as
rowId, ' + @ReFieldsStr + ' from ' + @TableName; --查询语句

IF (@WhereString! = '' or @WhereString!=null)
BEGIN
    SET @TotalCountSql=@TotalCountSql + ' where ' + @WhereString;
    SET @SqlString=@SqlString+ ' where ' + @WhereString;
END

--第一次执行得到
EXEC sp_executesql @totalCountSql,N'@TotalRecord int out',@TotalRecord OUTPUT;--返回总记录数
---执行主语句
SET @SqlString ='select * from ' + @SqlString + ') as t where rowId between ' +
ltrim(str(@StartRecord)) + ' and ' + ltrim(str(@EndRecord));
Exec(@SqlString)
END

declare @total int;
EXEC SP_CustomPager 'JDCategories','*', 'id', '', 20, 1, @total output
select @total

```

十一、函数

说到函数，首先和存储过程作个比较吧，两者有一个共同点都是预编译优化后存储在磁盘中，所以效率要比T-SQL高一点点。值得注意的是，存储过程可以创建或访问临时表，而函数不可以；同时函数不可以修改表中的数据，或调用产生副作用的函数，比如rand,newid,getdate（当然这并不是绝对的）；但是函数可以作为select 或from或where子句的一部分，而存储过程不可以。

系统函数

1 字符串函数

1.1 长度与分析用

```

datalength(Char_expr) 返回字符串包含字符数,但不包含后面的空格
substring(expression, start, length) 不多说了,取子串
right(char_expr, int_expr) 返回字符串右边int_expr个字符
left(char_expr, int_expr) 返回字符串左边int_expr个字符

```

1.2 字符操作类

```

upper(char_expr) 转为大写
lower(char_expr) 转为小写
space(int_expr) 生成int_expr个空格
replicate(char_expr, int_expr) 复制字符串int_expr次
reverse(char_expr) 反转字符串
stuff(char_expr1, start, length, char_expr2) 将字符串char_expr1中的从
start开始的length个字符用char_expr2代替
ltrim(char_expr) rtrim(char_expr) 取掉空格
ascii(char) char(ascii) 两函数对应,取ascii码,根据ascii码取字符

```

1.3 字符串查找

charindex(char_expr, expression) 返回char_expr的起始位置
patindex('%pattern%', expression) 返回指定模式的起始位置, 否则为0

2、数学函数

abs(numeric_expr) 求绝对值。
ceiling(numeric_expr) 取大于等于指定值的最小整数（即向上取整）。
floor(numeric_expr) 小于等于指定值得最大整数（即向下取整）。
avg(numeric_expr) 取平均数。
exp(float_expr) 返回e的n次方。
pi() 3.1415926.....（即圆周率 π ）。
power(底数m, 指数n) 返回m的n次方。
rand([int_expr]) 随机数产生器。
round(numeric_expr, int_expr) 按照int_expr规定的精度四舍五入。
sign(int_expr) 根据正数, 零, 负数, 返回+1, 0, -1。
sqrt(float_expr) 返回平方根。

3、日期时间函数

getdate() 返回日期
datetime(datepart, date_expr) 返回名称如 June
datepart(datepart, date_expr) 取日期一部份
datediff(datepart, date_expr1, dateexpr2) 日期差
dateadd(datepart, number, date_expr) 返回日期加上 number
上述函数中datepart的
写法 取值和意义
yy 1753-9999 年份
qq 1-4 刻
mm 1-12 月
dy 1-366 日
dd 1-31 日
wk 1-54 周
dw 1-7 周几
hh 0-23 小时
mi 0-59 分钟
ss 0-59 秒
ms 0-999 毫秒
日期转换
convert()

4、系统其他函数

suser_name() 用户登录名
user_name() 用户在数据库中的名字
show_role() 对当前用户起作用的规则
db_name() 数据库名
object_name(obj_id) 数据库对象名
col_name(obj_id, col_id) 列名
col_length(objname, colname) 列长度
valid_name(char_expr) 是否是有效标识符

5、类型转换函数

convert (数据类型[(长度)], 表达式[, 样式])
将一种数据类型的表达式显式转换为另一种数据类型的表达式;
长度: 如果数据类型允许设置长度, 可以设置长度, 例如 varchar(10);
样式: 用于将日期类型数据转换为字符数据类型的日期格式的样式。
cast (表达式 AS 数据类型[(长度)])
将一种数据类型的表达式显式转换为另一种数据类型的表达式。
例如: select cast(123 as nvarchar) 返回123
select N'年龄:' + cast(23 as nvarchar) 返回 年龄: 23

6、系统函数

newid 无参数
返回一个GUID(全局唯一表示符)值
例如: select newid()
返回: 2E6861EF-F4DB-4FFE-86EB-637482FE982J2
isnumeric (任意表达式)
判断表达式是否为数值类型或者是否可以转换成数值。
是: 返回1, 不是: 返回0
例如: select isnumeric(1111) 返回 1
select isnumeric('123rr') 返回 0
select isnumeric('123') 返回 1
isnull (任意表达式1, 任意表达式2)
如果任意表达式1不为NULL, 则返回它的值; 否则, 在将任意表达式2的类型转换为任意表达式1的类型(如果这两个类型不同)后, 返回任意表达式2的值。
例如: select isnull(null, N'没有值') 返回 没有值
select isnull(N'具体的值', N'没有值') 返回 具体的值
isdate (任意表达式)
确定输入表达式是否为有效日期或可转成有效的日期;
是: 返回1, 不是: 返回0
例如: select isdate(getdate()) 返回1
select isdate('2013-01-02') 返回1
select isdate('198') 返回0

7、排名函数

row_number 无参数
为结果集内每一行进行编号, 从1开始后面行依次加1, 常用于产生序号;
例如: select row_number() over(order by userid desc) as [NO], username, password from T_USER

8、聚合函数

count()
返回组中的总条数, count(*)返回组中所有条数, 包括NULL值和重复值项, 如果抒写表达式, 则忽略空值, 表达式为任意表达式。
max()
返回组中的最大值, 空值将被忽略, 表达式为数值表达式, 字符串表达式, 日期。
min()
返回组中的最小值, 空值将被忽略, 表达式为数值表达式, 字符串表达式, 日期。
sum()
返回组中所有值的和, 空值将被忽略, 表达式为数据表达式。
avg()
返回组中所有值的平均值, 空值将被忽略, 表达式为数据表达式。

表值函数: 返回一张表, 比如returns table as ... return select

标量值函数：返回单个数据类型的值（除BLOB、游标、时间戳），比如returns int as ... return @var;

标量值函数

标量值函数：返回单个数据类型的值（除BLOB、游标、时间戳），比如returns int as ... return @var;

语法：

--声明数据库引用

use 数据库名;

go

--判断是否存在需要创建的函数，如果存在则删除

if exists(select * from sys.objects where name=函数名称)

drop function 函数名称;

go

--创建用户自定义函数

create function [schema_name.] function_name

(@parameter_name [as] [type_schema_name.] parameter_data_type [= default] [readonly],...n)

returns return_data_type

[with] [encryption][,][schemabinding][,][returns null on null input | called on null input][,][execute as Clause]

as

begin

function_body;

return scalar_expression;

end

go

语法解析：

--schema_name

--用户定义函数所属的架构的名称。

--function_name

--用户定义函数的名称。 函数名称必须符合标识符规则，并且在数据库中以及对其架构来说是唯一的,即使未指定参数，函数名称后也需要加上括号。

--@parameter_name

--用户定义函数中的参数。 可声明一个或多个参数。

--一个函数最多可以有 2,100 个参数。 执行函数时，如果未定义参数的默认值，则用户必须提供每个已声明参数的值。

--通过将 at 符号 (@) 用作第一个字符来指定参数名称。 参数名称必须符合标识符规则。 参数是对应于函数的局部参数；其他函数中可使用相同的参数名称。

--参数只能代替常量，而不能用于代替表名、列名或其他数据库对象的名称。

--[type_schema_name.] parameter_data_type

--参数的数据类型及其所属的架构，后者为可选项。 对于 Transact-SQL 函数，允许使用除 timestamp 数据类型之外的所有数据类型（包括 CLR 用户定义类型和用户定义表类型）。

--对于 CLR 函数，允许使用除 text、ntext、image、用户定义表类型和 timestamp 数据类型之外的所有数据类型（包括 CLR 用户定义类型）。 在 Transact-SQL 函数或 CLR 函数中，

--不能将非标量类型 cursor 和 table 指定为参数数据类型。

--如果未指定 type_schema_name， 数据库引擎会按以下顺序查找 scalar_parameter_data_type:

--包含 SQL Server 系统数据类型名称的架构。

--当前数据库中当前用户的默认架构。

--当前数据库中的 dbo 架构。

--[=default]

--参数的默认值。 如果定义了 default 值，则无需指定此参数的值即可执行函数。

--如果函数的参数有默认值，则调用该函数以检索默认值时，必须指定关键字 DEFAULT。 此行为与在存储过程中使用具有默认值的参数不同，在后一种情况下，不提供参数同样意味着使用默认值。

--但在通过使用 EXECUTE 语句调用标量函数时，DEFAULT 关键字不是必需的。

--readonly

--指示不能在函数定义中更新或修改参数。 如果参数类型为用户定义的表类型，则应指定 READONLY。

--return_data_type

--标量用户定义函数的返回值。 对于 Transact-SQL 函数，可以使用除 timestamp 数据类型之外的所有数据类型（包括 CLR 用户定义类型）。

--对于 CLR 函数，允许使用除 text、ntext、image 和 timestamp 数据类型之外的所有数据类型（包括 CLR 用户定义类型）。 在 Transact-SQL 函数或 CLR 函数中，不能将非标量类型 cursor 和 table 指定为返回数据类型。

--encryption

--适用范围： SQL Server 2008 到 SQL Server 2017。

--指示 数据库引擎会将 CREATE FUNCTION 语句的原始文本转换为模糊格式。 模糊代码的输出在任何目录视图中都不能直接显示。 对系统表或数据库文件没有访问权限的用户不能检索模糊文本。

--但是，可以通过 DAC 端口访问系统表的特权用户或直接访问数据库文件的特权用户可以使用此文本。 此外，能够向服务器进程附加调试器的用户可在运行时从内存中检索原始过程。

--使用此选项可防止将函数作为 SQL Server 复制的一部分发布。 不能为 CLR 函数指定此选项。

--schemabinding

--指定将函数绑定到其引用的数据库对象。 如果指定了 SCHEMABINDING，则不能按照将影响函数定义的方式修改基对象。 必须首先修改或删除函数定义本身，才能删除将要修改的对象的依赖关系。

--returns null on null input | called on null input

--指定标量值函数的 OnNULLCall 属性。 如果未指定，则默认为 CALLED ON NULL INPUT。 这意味着即使传递的参数为 NULL，也将执行函数体。

--如果在 CLR 函数中指定了 RETURNS NULL ON NULL INPUT，它指示当 SQL Server 接收到的任何一个参数为 NULL 时，它可以返回 NULL，而无需实际调用函数体。 如果 <method_specifier> 中指定的 CLR 函数的方法已具有指示 RETURNS NULL ON NULL INPUT 的自定义属性，

--但 CREATE FUNCTION 语句指示 CALLED ON NULL INPUT，则优先采用 CREATE FUNCTION 语句指示的属性。 不能为 CLR 表值函数指定 OnNULLCall 属性。

--execute as 子句

--对于本机编译的标量用户定义函数，EXECUTE AS 是必需的。

--SELF

--EXECUTE AS SELF 与 EXECUTE AS user_name 等价，其中指定用户是创建或更改模块的用户。 创建或更改模块的用户实际用户 ID 存储在 sys.sql_modules 或 sys.service_queues 目录视图的 execute_as_principal_id 列中。

--OWNER

--指定模块内的语句在模块的当前所有者上下文中执行。 如果模块没有指定的所有者，则使用模块架构的所有者。 不能为 DDL 或登录触发器指定 OWNER。

--' user_name '

--指定模块内的语句在 user_name 指定的用户的上下文中执行。 将根据 user_name 来验证对模块内任意对象的权限。 不能为具有服务器作用域的 DDL 触发器或登录触发器指定 user_name。 请改用 login_name。

--user_name 必须存在于当前数据库中，并且必须是单一实例帐户。 user_name 不能为组、角色、证书、密钥或内置帐户，如 NT AUTHORITY\LocalService、NT AUTHORITY\NetworkService 或 NT AUTHORITY\LocalSystem。

--执行上下文的用户 ID 存储在元数据中，可以在 sys.sql_modules 或 sys.assembly_modules 目录视图的 execute_as_principal_id 列查看。

--' login_name '

--指定模块内的语句在 login_name 指定的 SQL Server 登录的上下文中执行。 将根据 login_name 来验证对模块内任意对象的权限。 只能为具有服务器作用域的 DDL 触发器或登录触发器指定 login_name。

--login_name 不能为组、角色、证书、密钥或内置帐户，如 NT AUTHORITY\LocalService、NT AUTHORITY\NetworkService 或 NT AUTHORITY\LocalSystem。

```
--function_body
--指定一系列定义函数值的 Transact-SQL 语句，这些语句在一起使用不会产生负面影响（例如修改表）。
function_body 仅用于标量函数和多语句表值函数。
--在标量函数中，function_body 是一系列 Transact-SQL 语句，这些语句一起使用可计算出标量值。
--在多语句表值函数中，function_body 是一系列 Transact-SQL 语句，这些语句将填充 TABLE 返回变量。
```

```
--scalar_expression
--指定标量函数返回的标量值。
```

示例：

```
--声明数据库引用
```

```
use testss;
```

```
go
```

```
--判断是否存在需要创建的函数，如果存在则删除
```

```
if exists(select * from sys.objects where name='scalarfun')
```

```
drop function scalarfun;
```

```
go
```

```
--创建用户自定义函数
```

```
create function dbo.scalarfun
```

```
(@num1 int,@num2 int,@num3 int =null)
```

```
returns int
```

```
--with encryption,schemabinding,returns null on null input,execute as owner
```

```
as
```

```
begin
```

```
declare @sum1 int=null;
```

```
if (@num1 is not null) and (@num2 is not null)
```

```
set @sum1=@num1+@num2;
```

```
else if (@num1 is not null)
```

```
set @sum1=@num1;
```

```
else if (@num2 is not null)
```

```
set @sum1=@num2;
```

```
else
```

```
set @sum1=0;
```

```
return @sum1;
```

```
end
```

```
go
```

表值函数

表值函数：返回一张表，比如returns table as ... return select

```
use ZhaoxiEdu
```

```
go
```

```
if exists(select * from sys.objects where name='useridList')
```

```
drop function mySub
```

```
go
```

```
create function dbo.useridList
```

```
(
```

```
    @parameter1 int
```

```
) returns @UserId table(userid int)
```

```
as
begin
    insert @UserId select Id from SysUser where companyId=@parameter1;
    return;
end
```

调用

```
select * from useridList(1)
```

十二、触发器

触发器 (trigger) 是SQL server 提供给程序员和数据分析师来保证数据完整性的一种方法，它是与表事件相关的特殊的存储过程，它的执行不是由程序调用，也不是手工启动，而是由事件来触发，比如当对一个表进行操作（insert，delete，update）时就会激活它执行。触发器经常用于加强数据的完整性约束和业务规则等。

触发器分类

1. DML触发器、
2. DDL触发器
3. 登录触发器。

DML触发器是一些附加在特定表或视图上的操作代码，当数据库服务器中发生数据操作语言事件时执行这些操作。SqlServer中的DML触发器有三种：

1. Insert触发器： 向表中插入数据时被触发。

Insert 触发器：

在向目标表中插入数据后，会触发该表的Insert 触发器，系统自动在内存中创建inserted表； 如果不满足判断数据会进行回滚，插入的数据操作会失败。

```
USE [ZhaoxiEdu]
GO

SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
ALTER TRIGGER [dbo].[Trigger_CompanyInsert]
    ON      [dbo].[Company]
    AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    Declare @name varchar(50);
    Select @name=[Name] From inserted

    IF(@name='朝夕教育')
        Begin
            Insert into CompanyNew([Name]
                                   , [CreateTime]
                                   , [CreatorId]
```

```

        , [LastModifierId]
        , [LastModifyTime])
Select [Name]
        , [CreateTime]
        , [CreatorId]
        , [LastModifierId]
        , [LastModifyTime]
From inserted

End
--ELSE
-- Begin
--     print('不添加')
--     rollback transaction      --数据回滚
-- END

END

```

2. delete触发器：从表中删除数据时被触发。

Delete 触发器：

在向目标表中删除数据后，会触发该表的Delete 触发器，系统自动在内存中创建deleted表，deleted表存放的是删除的数据。

```

USE [ZhaoxiEdu]
GO

SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO

--Delete 触发器
ALTER TRIGGER [dbo].[Trigger_CompanyDelete]
ON [dbo].[Company]
AFTER DELETE
AS
BEGIN
    SET NOCOUNT ON;

    Declare @name varchar(50);
    select @name=[Name] from Deleted;
    Delete CompanyNew where [Name]=@name

END

```

3. update触发器：修改表中数据时被触发。

Update 触发器：

在向目标表中更新数据后，会触发该表的Update 触发器，系统自动在内存中创建deleted表和inserted表，deleted表存放的是更新前的数据，inserted表存放的是更新的数据。

```

USE [ZhaoxiEdu]
GO

SET ANSI_NULLS ON
GO

```



```

SET QUOTED_IDENTIFIER ON
GO

--Update 触发器
ALTER TRIGGER [dbo].[Trigger_CompanyUpdate]
    ON [dbo].[Company]
    AFTER UPDATE
AS

BEGIN
    SET NOCOUNT ON;
    Declare @name varchar(50);
    select @name=[Name] from Deleted;
    Delete CompanyNew where [Name]=@name;
    Insert into CompanyNew([Name]
                                , [CreateTime]
                                , [CreatorId]
                                , [LastModifierId]
                                , [LastModifyTime])
    Select [Name]
            , [CreateTime]
            , [CreatorId]
            , [LastModifierId]
            , [LastModifyTime]
    From inserted

END

```

当遇到下列情形时，应考虑使用DML触发器：

1. 通过数据库中的相关表实现级联更改
2. 防止恶意或者错误的insert、update和delete操作，并强制执行check约束定义的限制更为复杂的其他限制。
3. 评估数据修改前后表的状态，并根据该差异才去措施。

DDL触发器(数据定义语言,Data Definition Language)

DDL触发器是当服务器或者数据库中发生数据定义语言(主要是以create,drop,alter开头的语句)事件时被激活使用，使用DDL触发器可以防止对数据架构进行的某些更改或记录数据中的更改或事件操作。

登录触发器

登录触发器将为响应 LOGIN 事件而激发存储过程。与 SQL Server 实例建立用户会话时将引发此事件。登录触发器将在登录的身份验证阶段完成之后且用户会话实际建立之前激发。因此，来自触发器内部且通常将到达用户的所有消息（例如错误消息和来自 PRINT 语句的消息）会传送到 SQL Server 错误日志。如果身份验证失败，将不激发登录触发器。

触发器优缺点：

- 1、强化约束：强制符合业务的规则和要求，能实现比check语句更为复杂的约束。
- 2、跟踪变化：触发器可以侦测数据库内的操作，从而禁止数据库中未经许可的更新和变化。
- 3、级联运行：侦测数据库内的操作时，可自动地级联影响整个数据库的各项内容。
- 4、嵌套调用：触发器可以调用一个或多个存储过程。触发器最多可以嵌套32层。

- 5、可移植性差。
- 6、占用服务器资源，给服务器造成压力。
- 7、执行速度主要取决于数据库服务器的性能与触发器代码的复杂程度。
- 8、嵌套调用一旦出现问题，排错困难，而且数据容易造成不一致，后期维护不方便。

使用建议

- 1、尽量避免在触发器中执行耗时操作，因为触发器会与SQL语句认为在同一事务中，事务不结束，就无法释放锁。
- 2、避免在触发器中做复杂操作，影响触发器性能的因素比较多（Eg:产品版本，所使用的架构等），要想编写高效的触发器考虑因素比较多，编写高性能触发器还是很难的。
- 3、触发器编写时注意多行触发时的处理。（一般不建议使用游标）

十三、自定义类型

用户定义数据类型

通俗定义：用户自己设计并实现的数据类型就称为用户自定义数据类型，即使这些数据类型基于系统数据类型。也可以理解为基础类型的一个延伸。

用户定义数据类型三要素：

1. 数据类型的名称
2. 所基于的系统数据类型
3. 数据类型的可空性（是否可以为空）

开始实操

```
USE [Advanced14]
GO

/***** Object:  UserDefinedDataType [dbo].[Age]      Script Date: 2022/8/22 11:35:49 *****/
CREATE TYPE [dbo].[Age] FROM [int] NOT NULL
GO
```

用户定义表类型

定义：类型类似于一个表结构，使用基础类型包含了字段，可以用来作为存储过程的参数传递。

开始实操

```
USE [ZhaoxiEdu]
GO

/***** Object:  UserDefinedTableType [dbo].[PageParameter]      Script Date: 2022/8/23 11:18:20 *****/
CREATE TYPE [dbo].[PageParameter] AS TABLE(
    [TableName] [varchar](50) NOT NULL,
    [ReFieldsStr] [varchar](200) NOT NULL,
    [OrderString] [varchar](200) NOT NULL,
    [WhereString] [varchar](500) NOT NULL,
    [PageSize] [int] NOT NULL,
    [PageIndex] [int] NOT NULL,
    [TotalRecord] [int] NOT NULL
)
```

```
GO
```

```
--调用存储过程
```

```
DECLARE @PageParam PageParameter
```

```
INSERT INTO @PageParam VALUES('Company','*','Id','','',10,2,0)
```

```
SELECT * FROM @PageParam
```

使用用户自定义表类型存储过程

```
USE [ZhaoxiEdu]
```

```
GO
```

```
/***** Object:  StoredProcedure [dbo].[SP_CustomPager]      Script Date: 2022/8/23 10:21:59 *****/
```

```
SET ANSI_NULLS ON
```

```
GO
```

```
SET QUOTED_IDENTIFIER ON
```

```
GO
```

```
Alter PROCEDURE [dbo].[SP_CustomPagerForTableType]
```

```
    @PageParameter PageParameter READONLY,
```

```
    @TotalRecord INT OUTPUT                --返回总记录数
```

```
AS
```

```
BEGIN
```

```
    --select * from @PageParameter
```

```
    --处理开始点和结束点
```

```
    DECLARE @TableName VARCHAR(50);        --表名
```

```
    DECLARE @ReFieldsStr VARCHAR(200) = '*'; --字段名(全部字段为*)
```

```
    DECLARE @OrderString VARCHAR(200);      --排序字段(必须!支持多字段不用加order by)
```

```
    DECLARE @WhereString VARCHAR(500) =N''; --条件语句(不用加where)
```

```
    DECLARE @PageSize INT;                  --每页多少条记录
```

```
    DECLARE @PageIndex INT = 1;             --指定当前为第几页
```

```
    --DECLARE @TotalRecord INT OUTPUT;      --返回总记录数
```

```
    select top 1
```

```
    @TableName=[TableName],
```

```
    @ReFieldsStr=[ReFieldsStr],
```

```
    @OrderString=[OrderString],
```

```
    @WhereString=[WhereString],
```

```
    @PageSize=[PageSize],
```

```
    @PageIndex=[PageIndex]
```

```
    from @PageParameter
```

```
    DECLARE @StartRecord INT;
```

```
    DECLARE @EndRecord INT;
```

```
    DECLARE @TotalCountSql NVARCHAR(500);
```

```
    DECLARE @SqlString NVARCHAR(2000);
```

```
    SET @StartRecord = (@PageIndex-1)*@PageSize + 1--起始记录
```

```
    SET @EndRecord = @StartRecord + @PageSize - 1 --结尾记录
```

```
    SET @TotalCountSql= N' select @TotalRecord = count(*) from ' + @TableName;--总记录数语句
```

```
    SET @SqlString = N'(select row_number() over (order by ' + @OrderString + ') as  
rowId,' + @ReFieldsStr+ ' from ' + @TableName;--查询语句
```

```
    IF (@WhereString! = '' or @WhereString!=null)
```

```
    BEGIN
```

```
        SET @TotalCountSql=@TotalCountSql + ' where ' + @WhereString;
```

```
        SET @SqlString =@SqlString+ ' where ' + @WhereString;
```

```
    END
```

```

--第一次执行得到
EXEC sp_executesql @totalCountSql,N'@TotalRecord int out',@TotalRecord OUTPUT;--返回总记录数
---执行主语句
SET @SqlString ='select * from ' + @SqlString + ') as t where rowId between ' +
ltrim(str(@StartRecord)) + ' and ' + ltrim(str(@EndRecord));
Exec(@SqlString)
END
GO

-----调用存储过程
--DECLARE @PageParam PageParameter
--DECLARE @TotalRecord INT
--INSERT INTO @PageParam VALUES('Company','*','Id','','',10,2,0)
----SELECT * FROM @PageParam
--EXEC SP_CustomPagerForTableType @PageParam,@TotalRecord

```

十四、序列

在之前的SQL SERVER版本中，一般采用GUID或者IDENTITY来作为标示符，但是IDENTITY是一个表对象，只能保证在**一张表**里面的序列，当我们遇到以下情况时：

表1		
ID	订单类型	价格
1	火车票	200
4	飞机票	2000
5	船票	600

表2		
ID	订单类型	价格
2	酒店	400
3	公园门票	170
6	租车费用	300

如上表，我们需要在多表之间，实现ID的一致性，在SQL SERVER里面就会有一定的麻烦，通常我们会使用额外使用一张TEMP表来映射这些ID的关系然后再从中取序列来完成。

SQL SERVER 2012的SEQUENCE功能，是一个基于SCHEMA的对象，所以可以被多表调用。

序列是用户定义的绑定到架构的对象，该对象可根据创建序列所依据的规范来生成数值序列。这组数值以定义的间隔按升序或降序生成，并且可配置为用尽时重新启动（循环）。序列不与特定表相关，这一点与标识列不同。应用程序将引用某一序列对象以便检索其下一个值。序列与表之间的关系由应用程序控制。用户应用程序可以引用一个序列对象，并跨多个行和表协调值。与在插入行时生成的标识列值不同，应用程序可以获得下一个序列号，而不必通过调用 NEXT VALUE FOR 函数来插入行。使用 sp_sequence_get_range 同时获取多个序列号。

创建脚本

```

USE [ZhaoxiEdu]
GO

/***** Object:  Sequence [dbo].[Sequence-20220824-113918]      Script Date: 2022/8/24 11:43:58 *****/
CREATE SEQUENCE [dbo].[Sequence-20220824-113918]
AS [bigint]
START WITH 5
INCREMENT BY 5
MINVALUE 5
MAXVALUE 50
CYCLE
CACHE
GO

```

获取序列

```
select NEXT VALUE FOR [dbo].[Sequence-20220824-113918]
```

存储过程获取序列

```
DECLARE
    @variableNumberOfIdsNeeded INT = 7, -- This will change for each call
    @FirstSeqNum SQL_VARIANT ,
    @LastSeqNum sql_variant,
    @SeqIncr sql_variant;

EXEC sys.sp_sequence_get_range @sequence_name = N' dbo.Sequence-20220824-113918',
    @range_size = @variableNumberOfIdsNeeded,
    @range_first_value = @FirstSeqNum OUTPUT,
    @range_last_value = @LastSeqNum OUTPUT,
    @sequence_increment = @SeqIncr OUTPUT;

-- The following statement returns the output values
SELECT @FirstSeqNum AS FirstVal, @LastSeqNum AS LastVal, @SeqIncr AS SeqIncrement;
```

十五、约束

SqlServer数据库为了保存的数据更具备准确性，一致性，在SqlServer中支持的有约束，有规则来限定，如果符合规则就可以保存，如果不符合，就不能保存。

1. 主键约束
2. 外键约束
3. Not Null约束
4. 唯一约束
5. 检查约束

1.主键约束(PRIMARY KEY)

主键 在每个表中对应的一行记录的唯一的标识，且主键同时要设置Not Null约束，如果在指定主键约束时没有指定Not Null约束，SQL Server会自动添加Not Null约束，下面使用图片演示创建表时指定主键约束

主键：唯一的标识----人的身份信息---身份证---身份证号码

汇集多个特点于一身：唯一标识，非空，聚集索引--排序

```
CREATE TABLE [dbo].[Company](
    [Id] [int] IDENTITY(1,1) NOT FOR REPLICATION NOT NULL,
    [Name] [varchar](50) NULL,
    [CreateTime] [datetime] NULL,
    [CreatorId] [int] NOT NULL,
    [LastModifierId] [int] NULL,
    [LastModifyTime] [datetime] NULL,
    [NewName] [dbo].[CompanyName] NULL,
    CONSTRAINT [PK_Company] PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
```

2.外键约束

外键是指在表中保存其他表的主键，同样的外键一样需要设置Not Null约束，下面使用图片演示创表时添加外键约束；

Company: 公司----SysUser 职员---主键（公司的唯一标识）

SysUser: 信息--包含了所属公司，（需要保存用户的公司信息，就保存Compnay的主键）

```
USE [ZhaoxiEdu]
GO

ALTER TABLE [dbo].[SysUser] WITH NOCHECK ADD CONSTRAINT [FK_SysUser_Company] FOREIGN KEY([CompanyId])
REFERENCES [dbo].[Company] ([Id])
GO

ALTER TABLE [dbo].[SysUser] CHECK CONSTRAINT [FK_SysUser_Company]
GO
```

3.Not Null约束

在表中，被指定Not Null约束的列的值不能为null，下面使用图片演示添加和删除Not Null约束，首先是添加Not Null约束

```
update [ZhaoxiEdu].[dbo].[Company] set NewName=null where
id=2
```

160 %

消息

消息 515, 级别 16, 状态 2, 第 11 行

不能将值 NULL 插入列 'NewName', 表 'ZhaoxiEdu.dbo.Company'; 列不允许有 Null 值。语句已终止。

4.唯一约束/唯一索引

唯一约束是保证该数据在表数据中是唯一的，下面是唯一约束的核心价值。

userInfo表---很多字段---No(用户的身份证号码)---不应该出现重复。

```
USE [ZhaoxiEdu]
GO

/***** Object:  Index [IX_SysUser]      Script Date: 2022/8/24 16:04:17 *****/
ALTER TABLE [dbo].[SysUser] ADD CONSTRAINT [IX_SysUser] UNIQUE NONCLUSTERED
(
    [Name] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, SORT_IN_TEMPDB = OFF, IGNORE_DUP_KEY = OFF,
ONLINE = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON
[PRIMARY]
GO
```

5.检查约束

检查约束用于指定列是否满足列的要求，比如年龄的数据不可以是负数下面使用图片演示添加检查约束

```
USE [ZhaoxiEdu]
GO

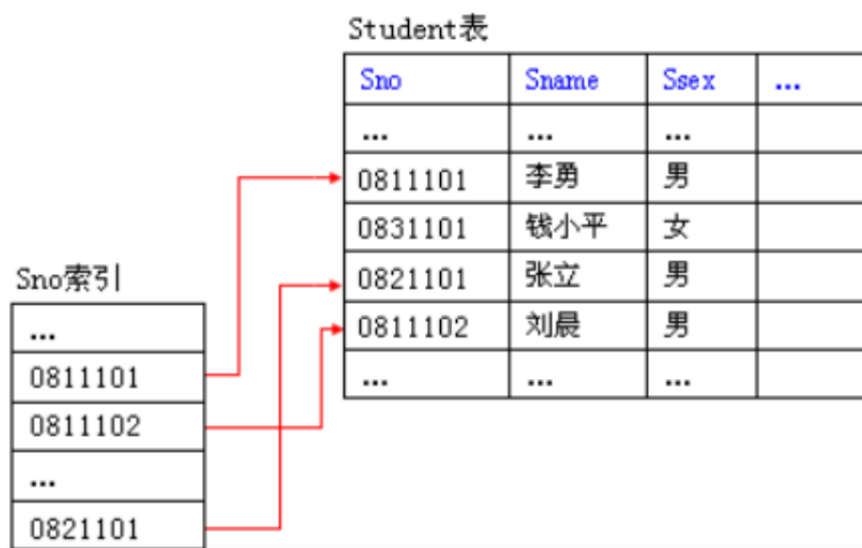
ALTER TABLE [dbo].[SysUser] WITH CHECK ADD CONSTRAINT [CK_SysUser] CHECK (([age]>(0)))
GO

ALTER TABLE [dbo].[SysUser] CHECK CONSTRAINT [CK_SysUser]
GO
```

十六、索引

索引基本概念

在数据库中建立索引是为了加快数据的查询速度。数据库中的索引与书籍中的目录或书后的术语表类似。在一本书中，利用目录或术语表可以快速查找所需信息，而无须翻阅整本书。在数据库中，索引使对数据的查找不需要对整个表进行扫描，就可以在其中找到所需数据。书籍的索引表是一个词语列表，其中注明了包含各个词的页码。而数据库中的索引是一个表中所包含的列值的列表，其中注明了表中包含各个值的行数据所在的存储位置。可以为表中的单个列建立索引，也可以为一组列建立索引。索引一般采用B树结构。索引由索引项组成，索引项由来自表中每一行的一个或多个列（称为搜索关键字或索引关键字）组成。B树按搜索关键字排序，可以对组成搜索关键字的任何子词条集合上进行高效搜索。例如，对于一个由A、B、C三个列组成的索引，可以在A以及A、B和A、B、C上对其进行高效搜索。例如，假设在Student表的Sno列上建立了一个索引（Sno为索引项或索引关键字），则在索引部分就有指向每个学号所对应的学生的存储位置的信息，如下图。



当数据库管理系统执行一个在Student表上根据指定的Sno查找该学生信息的语句时，它能够识别该表上的索引列（Sno），并首先在索引部分（按学号有序存储）查找该学号，然后根据找到的学号所指方向的数据的存储位置，直接检索出需要的信息。如果没有索引，则数据库管理系统需要从Student表的第一行开始，逐行检索指定的Sno值。从数据结构的算法知识我们知道有序数据的查找比无序数据的查找效率要高很多。

但索引为查找所带来的性能好处是有代价的，首先索引在数据库中会占用一定的存储空间来存储索引信息。其次，在对数据进行插入、更改和删除操作时，为了使索引与数据保持一致，还需要对索引进行相应维护。对索引的维护是需要花费时间的。

因此，利用索引提高查询效率是以占用空间和增加数据更改的时间为代价的。在设计和创建索引时，应确保对性能的提高程度大于在存储空间和处理资源方面的代价。

数据页

在数据库管理系统中，数据一般是按数据页存储的，数据页是一块固定大小的连续存储空间。不同的数据库管理系统数据页的大小不同，有的数据库管理系统数据页的大小是固定的，比如SQL Server的数据页就固定为8KB；

存放数据的数据页与存放索引项的数据页采用的都是通过指针链接在一起的方式连接各数据页，而且在页头包含指向下一页及前面页的指针，这样就可以将表中的全部数据或者索引链在一起。数据页的组织方式示意图如图6-2所示。

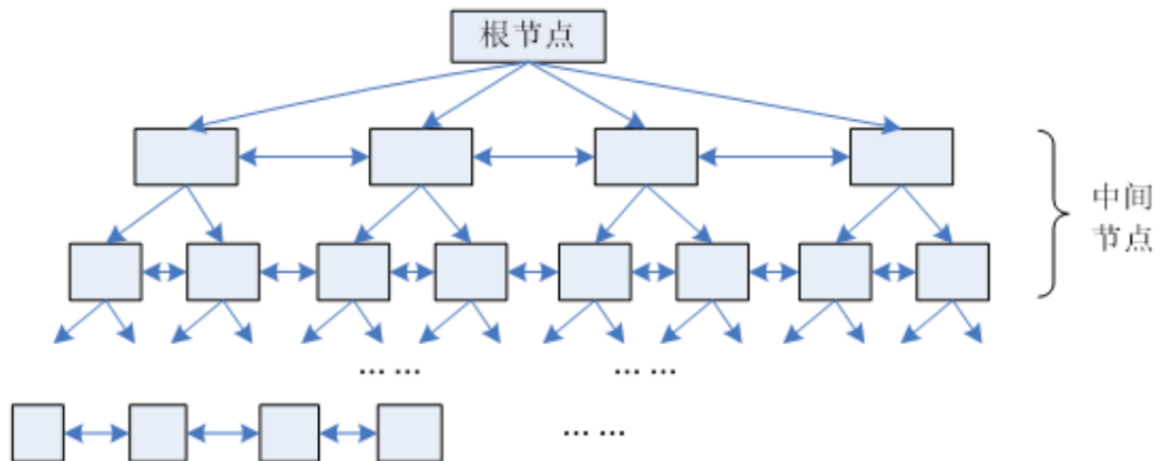


索引的存储结构及分类

索引分为两大类，一类是聚集索引（Clustered Index，也称为聚簇索引），另一类是非聚集索引（Non-Clustered Index，也称为非聚簇索引）。聚集索引对数据按索引关键字值进行物理排序，非聚集索引不对数据按索引关键字值进行物理排序，而只将索引关键字按值进行排序。图6-1所示的索引示意图即为非聚集索引。在SQL Server中聚集索引和非聚集索引都采用B树结构来存储索引项，而且都包含数据页和索引页，其中索引页用来存放索引项和指向下一层的指针，数据页用来存放数据。不同的数据库管理系统中索引的存储结构不尽相同，本章我们主要介绍SQL Server对索引采用的存储结构。

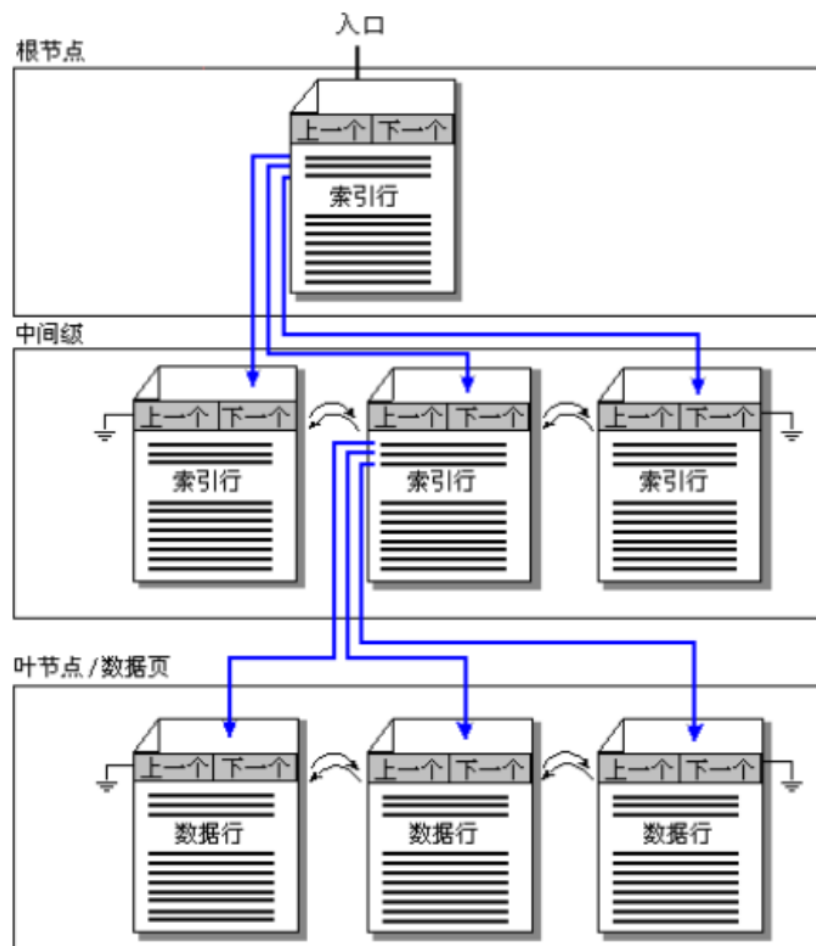
B树结构

B树（Balanced Tree，平衡树）的最上层节点称为根节点（Root Node），最下层节点称为叶节点（Leaf Node）。在根节点所在层和叶节点所在层之间的层上的节点称为中间节点（Intermediate Node）。B树结构从根节点开始，以左右平衡的方式存放数据，中间可根据需要分成许多层。



聚集索引

聚集索引的B树是自下而上建立的，最下层的叶级节点存放的是数据，因此它即是索引页，同时也是数据页。多个数据页生成一个中间层节点的索引页，然后再由数个中间层节点的索引页合成更上层的索引页，如此上推，直到生成顶层的根节点的索引页。生成高一层节点的方法是：从叶级节点开始，高一层节点中每一行由索引关键字值和该值所在的数据页编号组成，其索引关键字值选取的是其下层节点中的最大或最小索引关键字的值。



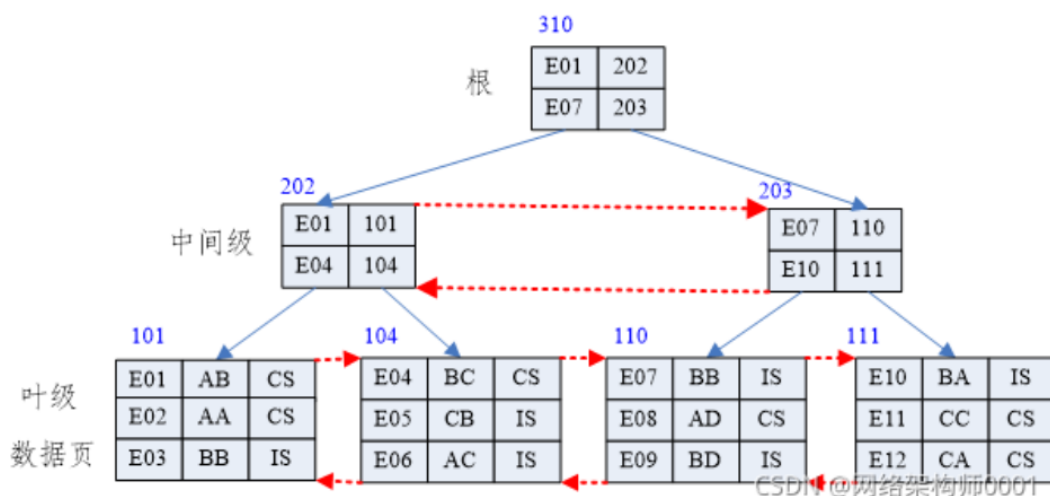
除叶级节点之外的其他层节点，每一个索引行由索引项的值以及这个索引项在下层节点的数据页编号组成。

例如，设有职工（Employee）表，其包含的列有：职工号（Eno）、职工名（Ename）和所在部门（Dept）。假设在Eno列上建有一个聚集索引（按升序排序）。（注：每个节点左上方位置的数字代表数据页编号），其中的虚线代表数据页间的链接。

案例分析

现有职工（Employee）表，其包含的列有：职工号（Eno）、职工名（Ename）和所在部门（Dept）。假设在Eno列上建有一个聚集索引（按升序排序）。（注：每个节点左上方位置的数字代表数据页编号），其中的虚线代表数据页间的链接。

Eno	Ename	Dept
E01	AB	CS
E02	AA	CS
E03	BB	IS
E04	BC	CS
E05	CB	IS
E06	AS	IS
E07	BB	IS
E08	AD	CS
E09	BD	IS
E10	BA	IS
E11	CC	CS
E12	CA	CS



如上图：在Eno列上建有聚集索引的B-树 在聚集索引的叶节点中，数据按聚集索引关键字的值进行物理排序。因此，聚集索引很类似于电话号码簿，在电话号码簿中数据是按姓氏排序的，这里姓氏就是聚集索引关键字。由于聚集索引关键字决定了数据在表中的物理存储顺序，因此一个表只能包含一个聚集索引。但该索引可以由多个列（组合索引）组成，就像电话号码簿按姓氏和名字进行组织一样。当在建

有聚集索引的列上查找数据时，系统首先从聚集索引树的入口（根节点）开始逐层向下查找，直到达到B树索引的叶级，也就是达到了要找的数据所在的数据页，最后只在这个数据页中查找所需数据即可。例如，若执行语句：SELECT * FROM Employee WHERE Eno = 'E08' 则首先从根（310数据页）开始查找，用“E08”逐项与310页上的每个索引关键字的值进行比较。由于“E08”大于此页的最后一个索引项“E07”的值，因此，选“E07”所在的数据页203，再进入到203数据页中继续与该页上的各索引关键字进行比较。由于“E08”大于203数据页上的“E07”而小于“E10”，因此，选“E07”所在的数据页110，再进入到110数据页中进行逐项比较，这时可找到Eno等于“E08”的项，而且这个项包含了此职工的全部数据信息。至此查找完毕。

索引的使用代价

当插入或删除数据时，除了会影响数据的排列顺序外，还会引起索引页中索引项的增加或减少，系统会对索引页进行分裂或合并，以保证B树的平衡性，因此B树的中间节点数量以及B树的高度都有可能发生变化，但这些调整都是数据库管理系统自动完成的，因此，在对有索引的表进行插入、删除和更改操作时，有可能会降低这些操作的执行性能。聚集索引对于那些经常要搜索列在连续范围内的值的查询特别有效。使用聚集索引找到包含第一个列值的行后，由于后续要查找的数据值在物理上相邻而且有序，因此只要将数据值直接与查找的终止值进行比较即可。在创建聚集索引之前，应先了解数据是如何被访问的，因为数据的访问方式直接影响了索引的使用。如果索引建立的不合适，则非但不能达到提高数据查询效率的目的，而且还会影响数据的插入、删除和更改操作的效率。因此，索引并不是建立的越多越好（建立索引需要占用空间，维护索引需要耗费时间），而是要有一些考虑因素。

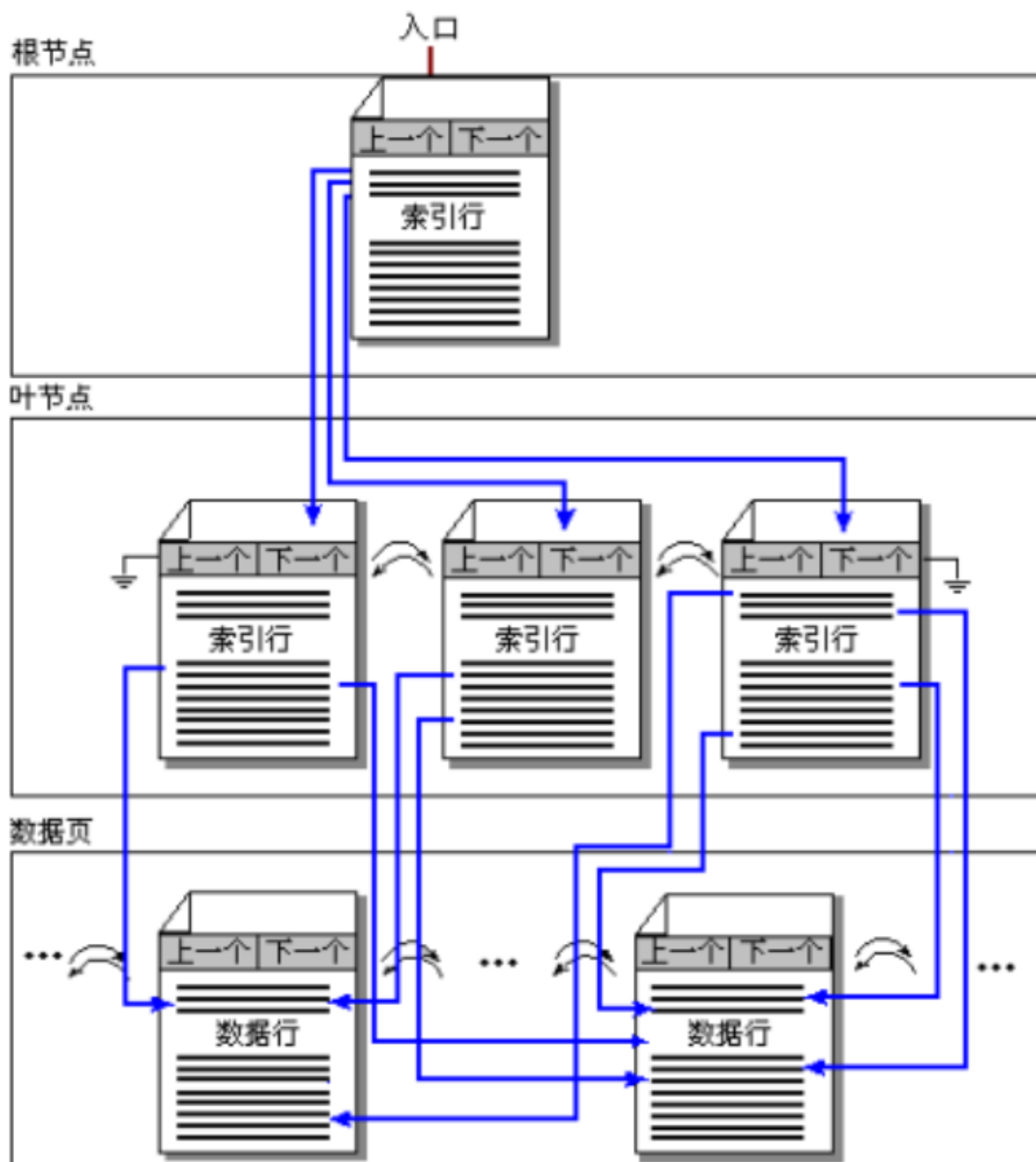
聚集索引使用建议

下列情况可考虑创建聚集索引：

1. 包含大量非重复值的列。
2. 使用下列运算符返回一个范围值的查询：BETWEEN AND、>、>=、< 和 <=。
3. 经常被用作连接的列，一般来说，这些列是外键列。
4. 对ORDER BY或GROUP BY子句中指定的列建立索引，可以使数据库管理系统在查询时不必对数据再进行排序，从而可以提高查询性能。对于频繁进行更改操作的列则不适合建立聚集索引。

非聚集索引

非聚集索引与新华字典的偏旁部首查字法类似。字典的内容（数据）存储在一个地方，（部首）存储在另一个地方。而且字典的内容（数据）并不按部首（索引）的顺序存放，但偏旁部首中的每个词在字典中都有确切的位置。非聚集索引就类似偏旁部首，而数据就类似于一本字典的文字。非聚集索引的存储示意图如下图：



非聚集索引与聚集索引一样用B树结构，但有两个重要差别：

1. 数据不按非聚集索引关键字值的顺序排序和存储。
2. 非聚集索引的叶级节点不是存放数据的数据页。非聚集索引B树的叶级节点是索引行。每个索引行包含非聚集索引关键字值以及一个或多个行定位器，这些行定位器指向该关键字值对应的数据行（如果索引不唯一，则可能是多行）。

例如，假设在Employee表的Eno列上建有一个非聚集索引，则数据和其索引B树的形式。在建有非聚集索引的表上查找数据的过程与聚集索引类似，也是从根节点开始逐层向下查找，直到找到叶级节点，在叶级节点中找到匹配的索引关键字值之后，其所对应的行定位器所指位置即是查找数据的存储位置。由于非聚集索引并不改变数据的物理存储顺序，因此，可以在一个表上建立多个非聚集索引。就象一本书可以有多个术语表一样，如一本介绍园艺的书可能会包含一个植物通俗名称的术语表和一个植物学名称的术语表，因为这是读者查找信息的两种最常用的方法

非聚集索引使用建议

在创建非聚集索引之前，应先了解数据是如何被访问的，以使建立的索引科学合理。对于下述情况可考虑创建非聚集索引：

1. 包含大量非重复值的列。如果某列只有很少的非重复值，比如只有1和0，则不对这些列建立非聚集索引。
2. 经常作为查询条件使用的列。
3. 经常作为连接和分组条件的列。

索引创建

初始化表

```
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[Commodity](
    [Id] [int] NULL,
    [Title] [varchar](50) NULL,
    [CreateTime] [datetime] NULL,
    [CreateId] [int] NULL,
    [Url] [varchar](200) NULL
) ON [PRIMARY]
GO
```

初始化数据

```
INSERT INTO [dbo].[Commodity] ([Id],[Title],[CreateTime],[CreateId],[Url])
VALUES (10,'商品1',getdate(),10,'zhoxiedu.com');

INSERT INTO [dbo].[Commodity] ([Id],[Title],[CreateTime],[CreateId],[Url])
VALUES (3,'商品1',getdate(),10,'zhoxiedu.com');

INSERT INTO [dbo].[Commodity] ([Id],[Title],[CreateTime],[CreateId],[Url])
VALUES (7,'商品1',getdate(),10,'zhoxiedu.com');

INSERT INTO [dbo].[Commodity] ([Id],[Title],[CreateTime],[CreateId],[Url])
VALUES (5,'商品1',getdate(),10,'zhoxiedu.com');

INSERT INTO [dbo].[Commodity] ([Id],[Title],[CreateTime],[CreateId],[Url])
VALUES (2,'商品1',getdate(),10,'zhoxiedu.com');
```

十七、游标

游标是一种能从包含多个**元组**的集合中每次读取一个元组的机制。游标总是和一段SELECT语句关联，SELECT语句查询出的结果集就作为集合，游标能每次从该集合中读取出一个元组进行不同操作。

游标的核心价值

1. 将游标定位在结果集特定元组。
2. 将游标指定结果集中的元组数据读出。
3. 利用循环读取结果集中的多个元组数据。
4. 对游标指定结果集的元组进行数据修改。
5. 为其它用户设置结果集数据的更新限制。

6. 提供脚本、存储过程和触发器中访问结果集中数据的TSQL语句

声明游标

```
DECLARE cursor_name CURSOR [INSENSITIVE] [SCROLL] CURSOR
    FOR <SELECT语句>
    [FOR READ ONLY|UPDATE[OF <列名>[,...n]]]
```

INSENSITIVE:定义游标所选出的结果集存放在一个临时表，对该游标的读取操作都有该临时表来应答。游标不会随着基本表内容的改变而改变，同时也无法通过游标来更新基本表。如果不使用该关键字，对基本表的更新、删除都会反应到游标中。

SCROLL:指定游标使用的读取选项，默认值为NEXT，如果不使用该关键字，那么读取游标只能进行NEXT操作，如果使用该关键字，那么游标向任何方向，或者任何位置移动，进行NEXT、LAST、FIRST、PRIOR、RELATIVE n、ABSOLUTE n操作。

FOR READ ONLY:表示定义游标为只读游标，不允许使用UPDATE、DELETE语句更新游标内的数据。
UPDATE[OF <类名>[,...n]]：指定游标内可以更新的列，如果有指定要更新的列，则表明所有列都允许更新。

打开游标

```
OPEN [LOCAL|GLOBAL] cursor_name;
```

读取游标

```
FETCH [NEXT|PRIOR|FIRST|LAST
      |ABSOLUTE n|@nvar]
      [RELATIVE n|@nvar]
FROM [GLOBAL]cursor_name
INTO[@nvar1,...n]
```

参数解读

名称	含义
FIRST	读取游标中第一行数据
LAST	读取游标最后一行数据
NEXT	读取游标当前行下一行数据
PRIOR	读取游标当前行上一行数据
RELATIVE n	读取游标当前行之前或之后第n行数据（n为正则向前，反之则向后）
ABSOLUTE n	读取游标第n行数据（n为负从最后一行开始，反之则从第一行开始）

关闭游标

在处理完结果集中数据之后，必须关闭游标来释放结果集

```
CLOSE cursor_name
```

释放游标

游标使用不再需要之后，需要释放游标，以获取与游标有关的一切资源。

```
DEALLOCATE cursor_name
```

游标系统白能量与函数

游标系统变量与函数返回有关游标的信息。

```
@@CURSOR_NUM
```

返回最后打开的游标中满足条件的元组数。

```
@@FETCH_STATUS
```

返回上次执行FETCH命令的状态

返回值	说明
0	FETCH语句成功
-1	FETCH语句失败或此元组不在结果集中
-2	被读取的元组不存在

- **CURSOR_STATUS('GLOBAL|LOCAL','cursor_name')**

返回值	意义
1	结果集至少一行
0	结果集为空
-1	游标被关闭
-2	游标不可用
-3	指定游标不存在

用于确定是否返回游标和结果集

利用游标修改和删除表数据

可以通过游标来UPDATE修改或删除DELETE表中的当前数据行。

```
--修改数据
UPDATE table_name
    SET column_name=value|expression
    WHERE CURRENT OF cursor_name
--删除数据
DELETE FROM table_name
    WHERE CURRENT OF cursor_name
```

十八、数据库备份

测试环境，发布到生产环境来。

数据库备份还原

备份Bak

还原数据库

可能会存在数据库的版本差异---测试环境使用的数据库的版本和生产环境不同

数据库脚本备份

十九、误操作恢复

执行了一条Delete 语句---后面没有跟where条件~~

误删除一条表

脚本执行错误~~

---数据库中的灾难---尤其是生产环境中，影响很大的~

准备工作

1、初始化数据库ZhaoxiEduRecovery

```
USE [master]
GO
/***** Object:  Database [ZhaoxiEduRecovery]      Script Date: 2022/9/13 9:55:27 *****/
CREATE DATABASE [ZhaoxiEduRecovery]
    CONTAINMENT = NONE
    ON PRIMARY
    ( NAME = N'ZhaoxiEduRecovery', FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL15.MSSQLSERVER\MSSQL\DATA\ZhaoxiEduRecovery.mdf' , SIZE = 8192KB , MAXSIZE = UNLIMITED,
FILEGROWTH = 65536KB )
    LOG ON
    ( NAME = N'ZhaoxiEduRecovery_log', FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL15.MSSQLSERVER\MSSQL\DATA\ZhaoxiEduRecovery_log.ldf' , SIZE = 8192KB , MAXSIZE = 2048GB ,
FILEGROWTH = 65536KB )
    WITH CATALOG_COLLATION = DATABASE_DEFAULT
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET COMPATIBILITY_LEVEL = 150
GO
IF (1 = FULLTEXTSERVICEPROPERTY('IsFullTextInstalled'))
begin
EXEC [ZhaoxiEduRecovery].[dbo].[sp_fulltext_database] @action = 'enable'
end
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET ANSI_NULL_DEFAULT OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET ANSI_NULLS OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET ANSI_PADDING OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET ANSI_WARNINGS OFF
GO
```



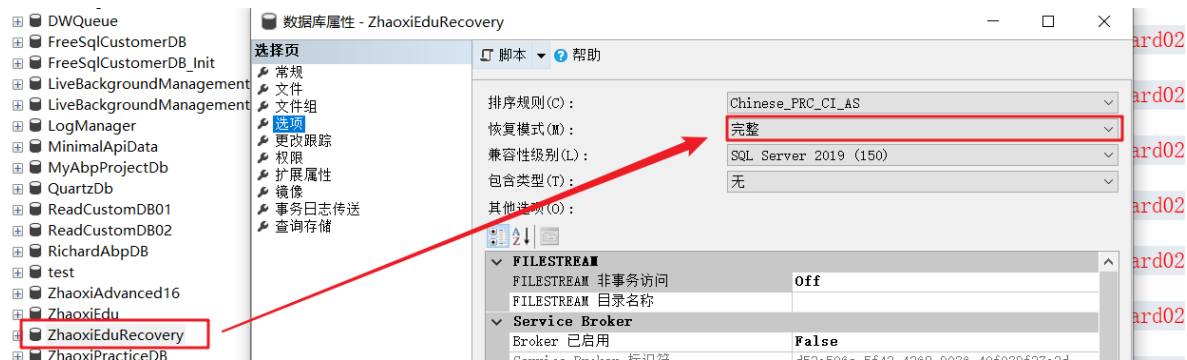
```
ALTER DATABASE [ZhaoxiEduRecovery] SET ARITHABORT OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET AUTO_CLOSE OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET AUTO_SHRINK OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET AUTO_UPDATE_STATISTICS ON
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET CURSOR_CLOSE_ON_COMMIT OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET CURSOR_DEFAULT GLOBAL
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET CONCAT_NULL_YIELDS_NULL OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET NUMERIC_ROUNDABORT OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET QUOTED_IDENTIFIER OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET RECURSIVE_TRIGGERS OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET DISABLE_BROKER
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET AUTO_UPDATE_STATISTICS_ASYNC OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET DATE_CORRELATION_OPTIMIZATION OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET TRUSTWORTHY OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET ALLOW_SNAPSHOT_ISOLATION OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET PARAMETERIZATION SIMPLE
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET READ_COMMITTED_SNAPSHOT OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET HONOR_BROKER_PRIORITY OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET RECOVERY FULL
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET MULTI_USER
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET PAGE_VERIFY CHECKSUM
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET DB_CHAINING OFF
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET FILESTREAM( NON_TRANSACTED_ACCESS = OFF )
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET TARGET_RECOVERY_TIME = 60 SECONDS
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET DELAYED_DURABILITY = DISABLED
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET ACCELERATED_DATABASE_RECOVERY = OFF
GO
EXEC sys.sp_db_vardecimal_storage_format N'ZhaoxiEduRecovery', N'ON'
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET QUERY_STORE = OFF
GO
USE [ZhaoxiEduRecovery]
GO
```

```

/***** Object: Table [dbo].[UserTest]      Script Date: 2022/9/13 9:55:27 *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[UserTest](
    [id] [int] NULL,
    [uName] [varchar](50) NULL,
    [age] [int] NULL
) ON [PRIMARY]
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard01', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
INSERT [dbo].[UserTest] ([id], [uName], [age]) VALUES (123, N'Richard02', 37)
GO
USE [master]
GO
ALTER DATABASE [ZhaoxiEduRecovery] SET READ_WRITE
GO

```

1、设置数据库恢复模式



设置数据库--属性---选项---恢复模式设置为：完整；如上图

或者通过脚本查询

```
SELECT recovery_model, recovery_model_desc
FROM sys.databases
WHERE name = 'ZhaoxiEduRecovery'
```

注意：确保数据库的恢复模式最起码不能为【简单】

建议：

对于任何重要环境，不仅仅是客户正式环境（俗称生产环境），都强烈建议使用【完整恢复模式】，虽然对于另外两种（大容量日志（BULK_LOGGED）、简单（SIMPLE））来说，完整恢复模式产生的日志会大，但是在出现问题的时候，就会觉得这些都不算什么了。并且我也想不到任何理由对于正式环境不使用完整恢复模式。只要管理得当，完整恢复模式的日志也不会太大。

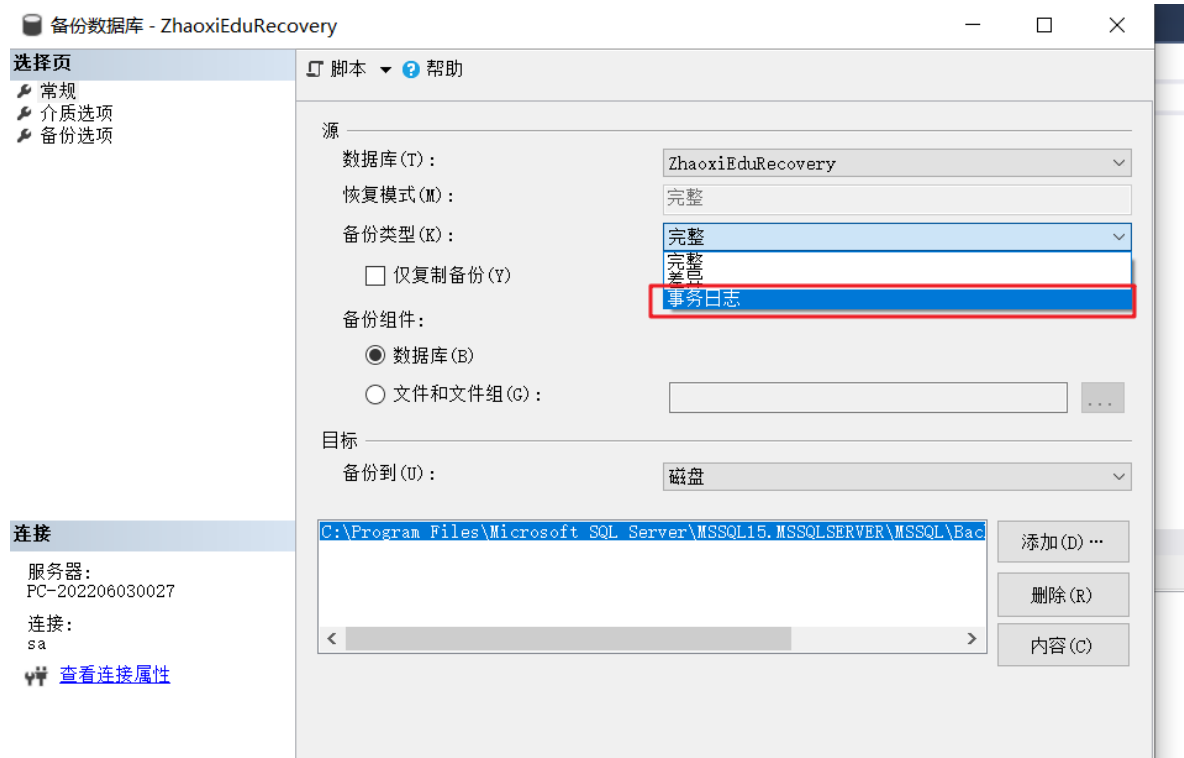
2、至少做一次完整备份

至少曾经做过最少一次的完整备份。因为所有类型的备份都基于完整备份，如果没有最少一次完整备份，其他类型的备份都是多余的，所以在这里强调一下，在创建完一个新数据库之后，强烈建议甚至强制做一次完整备份。

脚本查询备份情况：

```
SELECT database_name, recovery_model, name
FROM msdb.dbo.backupset
```

3、做一次日志尾部位份



发生误操作

基于之前的数据库新增个表，同时删除部分数据，这部分操作我们认为是误操作。

```
USE [ZhaoxiEduRecovery]
GO

select GETDATE() as 开始时间

-----创建了一个新表-----
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[UserTestNew] (
    [id] [int] NULL,
    [uName] [varchar](50) NULL,
    [age] [int] NULL
) ON [PRIMARY]
GO

-----删除了一些数据-----
delete UserTest where uName='Richard02'
Go

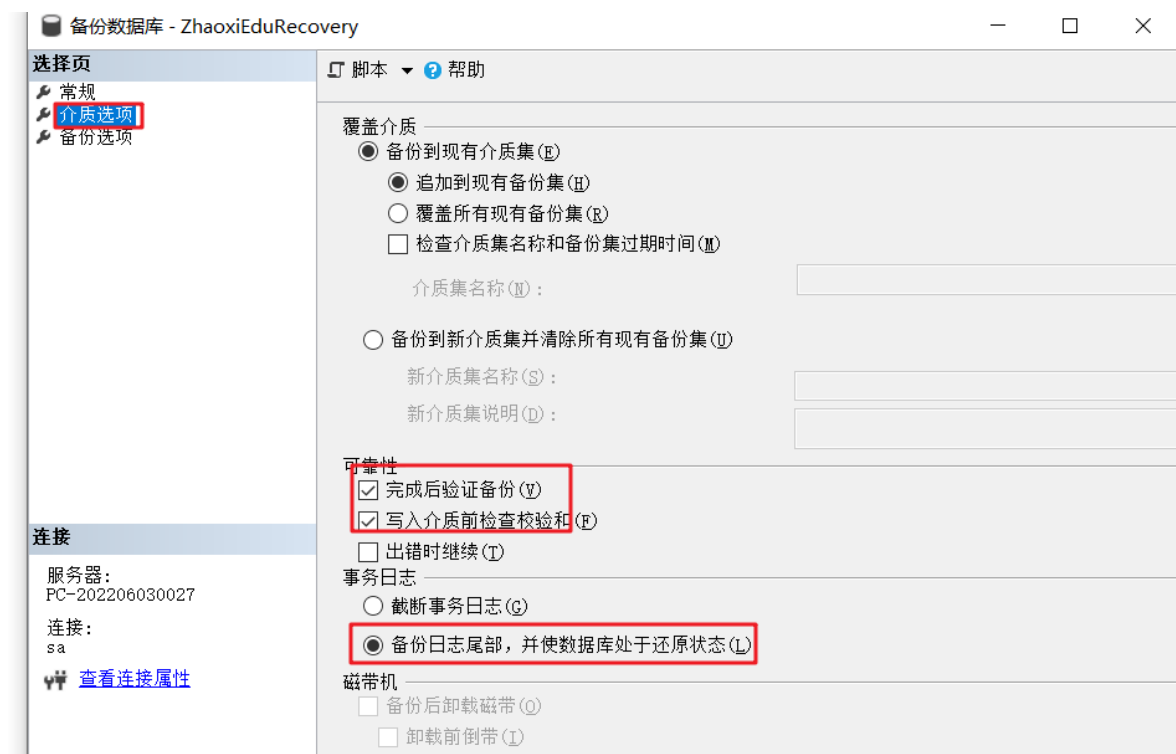
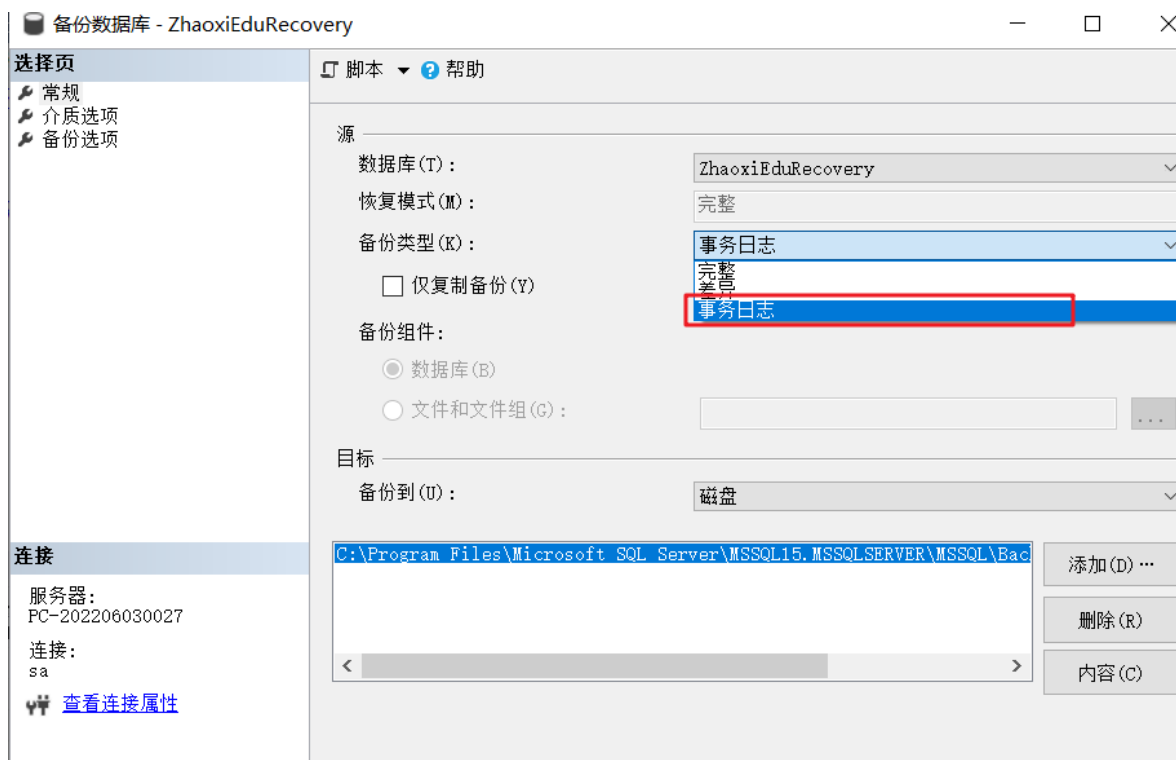
select GETDATE() as 结束时间
```

到这一步，灾难发生了~~ 接下来开始做恢复~~

开始恢复

数据误操作一旦发生，第一时间就要保证数据库不允许操作。不允许其他的用户来操作数据库了。

1、做一次日志备份，最重要是选择【备份日志尾部】



此时, 数据库会处于【正在还原】的状态

注意:

为了不让其他的用户使用当前数据库, 可以设置为单用户模式

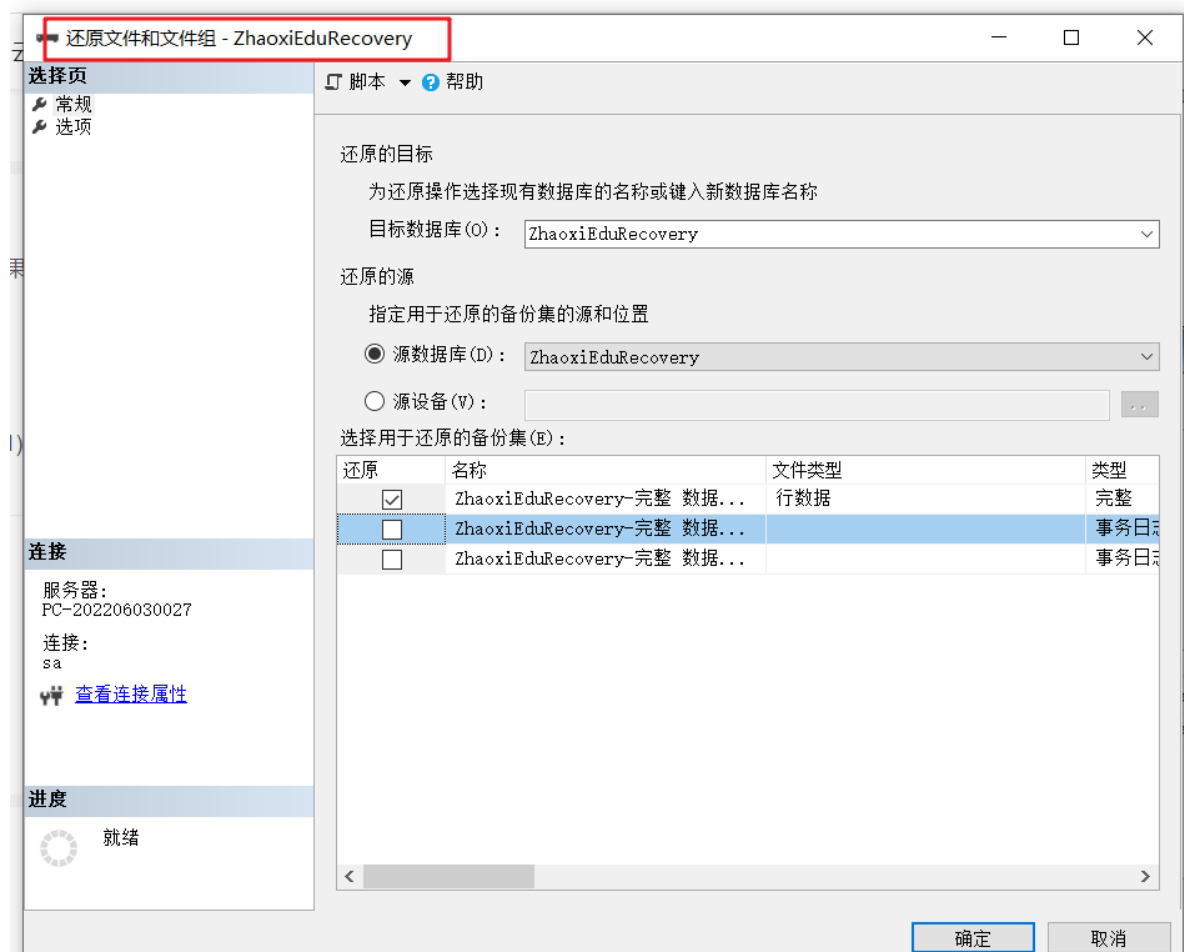
如果发现备份不了可以用下面语句查看, 并把spid杀掉:

```
SELECT * FROM sys.sysprocesses WHERE dbid=DB_ID('ZhaoxiEduRecovery')
```

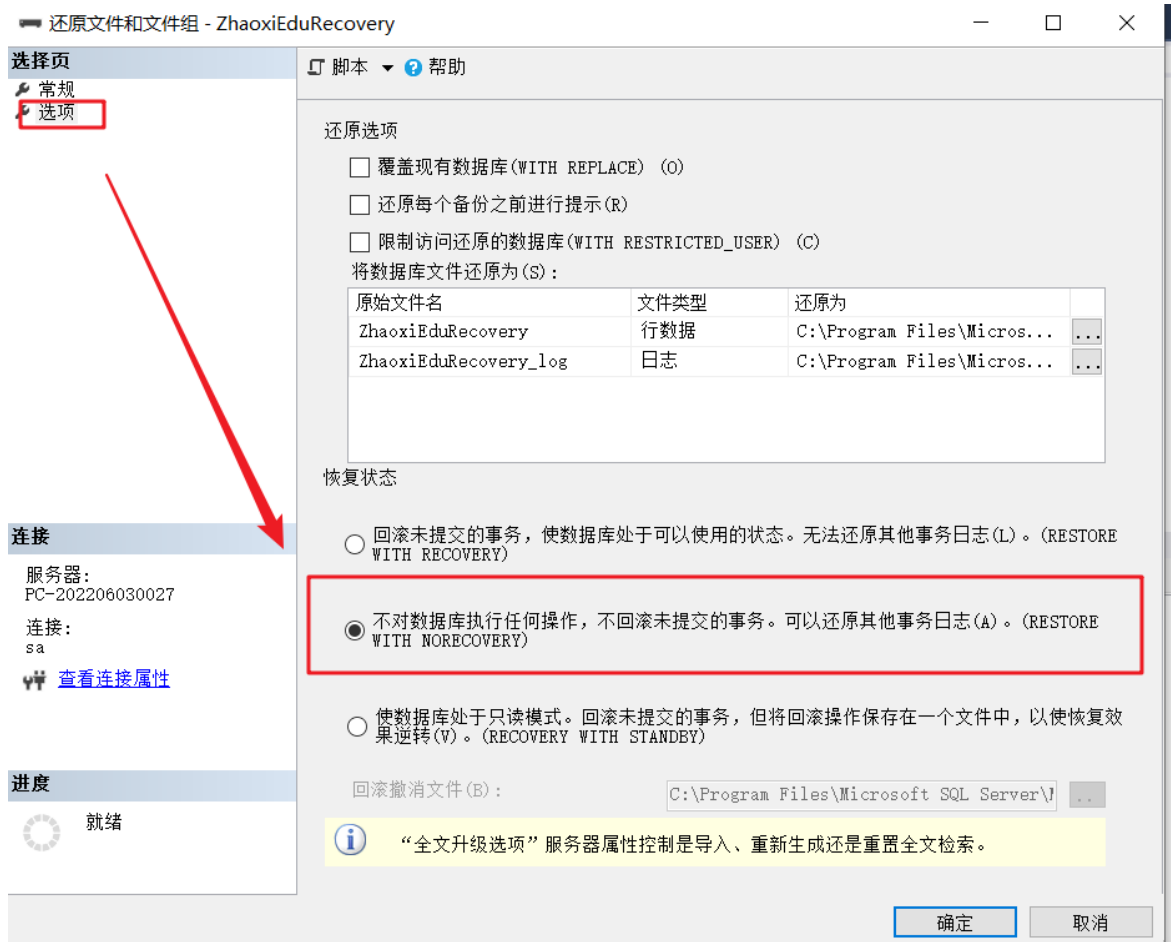
杀掉spid脚本

```
KILL spid
```

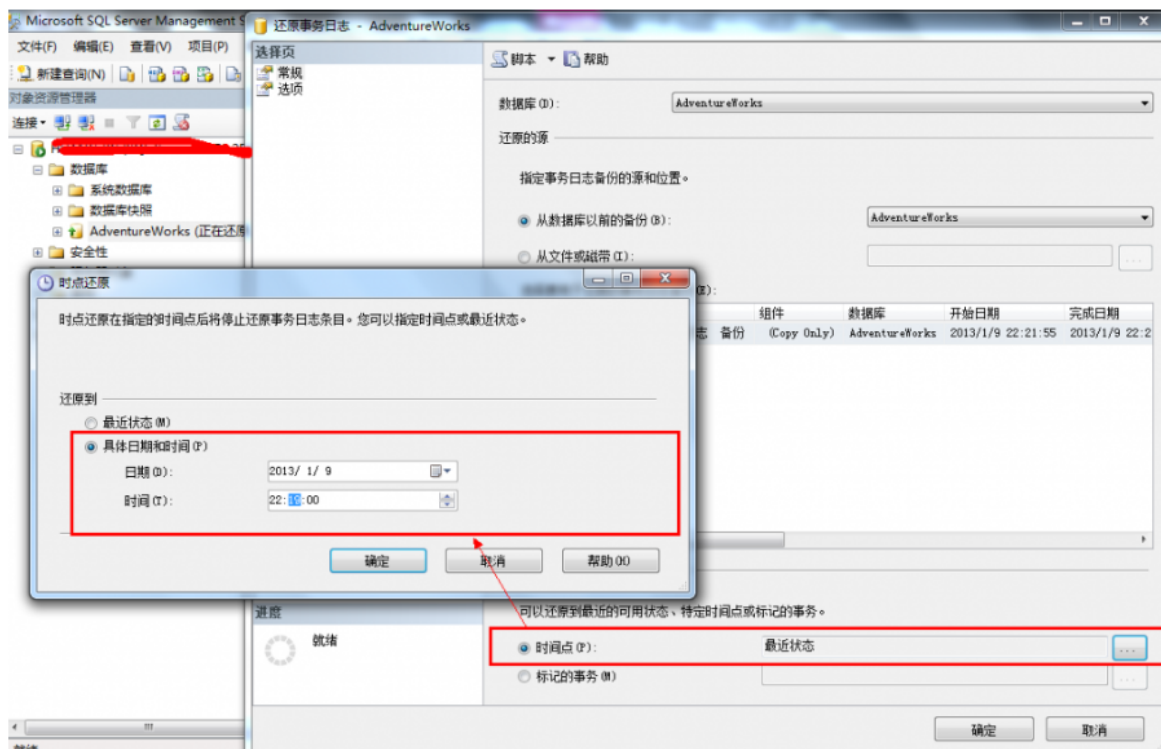
2、进行还原**文件和文件组**，先要还原完整备份，选择最近的那次。也就是最后备份的那次。

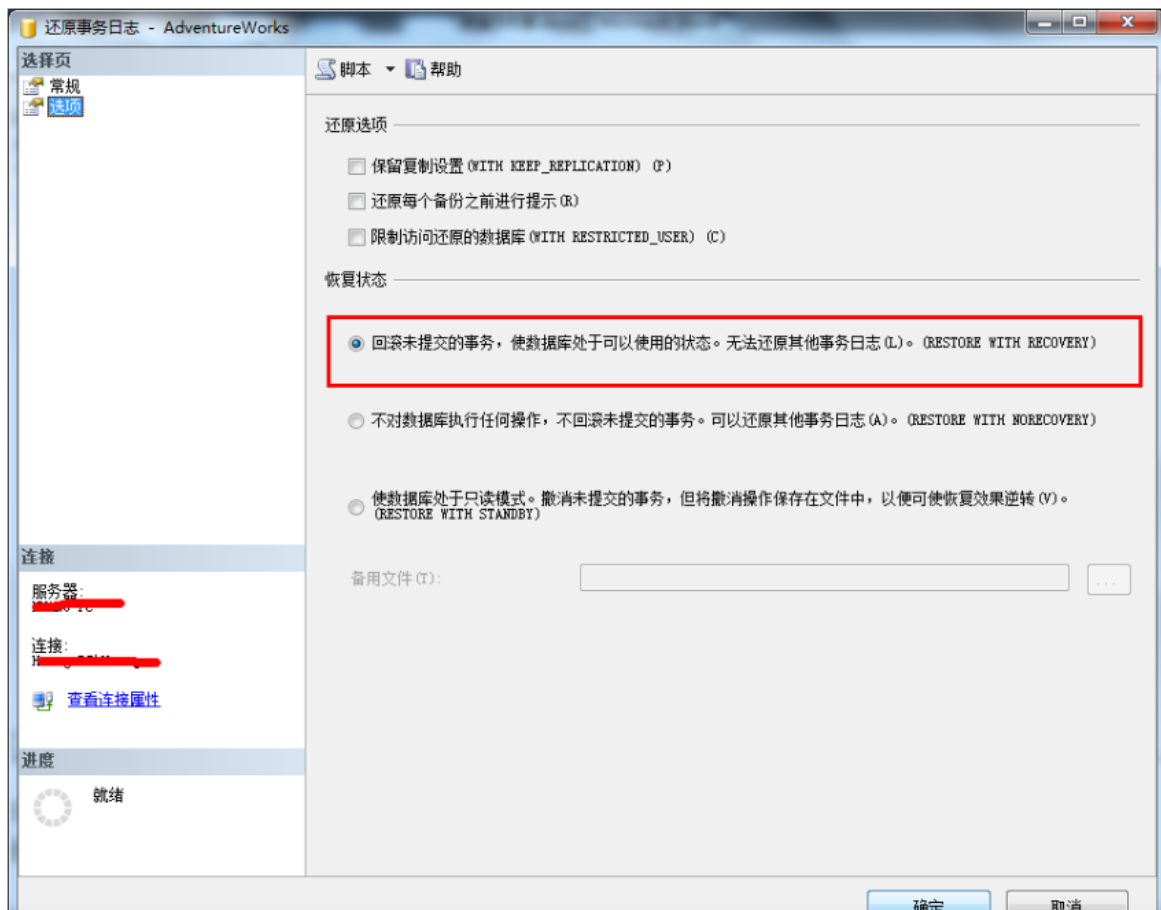


这里又有一个注意事项，记得选择：



3、接着还原日志文件，这是最重要的一步：





4、测试数据

```
SELECT TOP (1000) [id]
      , [uName]
      , [age]
FROM [ZhaoxiEduRecovery].[dbo].[UserTest]
```

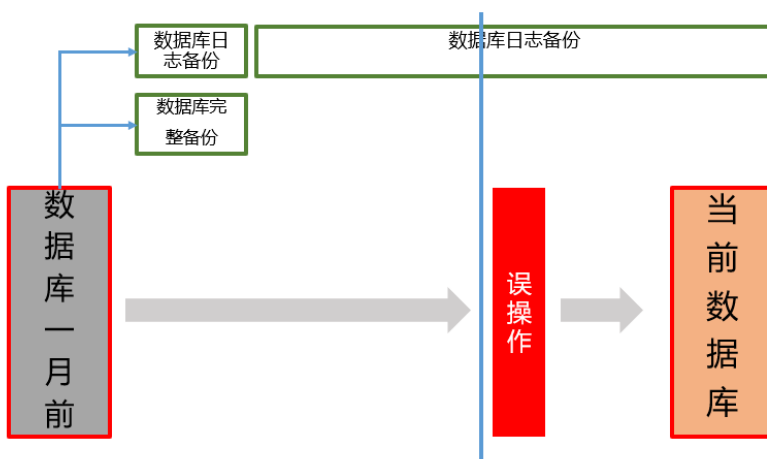
误操作恢复实现原理

数据库包含数据库文件和日志文件：

mdf 文件：数据库文件

ndf文件： 数据库文件

ldf文件： 日志文件，记录了数据库自存在以来，做的所有的相关操作；都会记录在ldf中。



二十、SqlServer事务

1、什么是事务？

事务是在数据库上按照一定的逻辑顺序执行的任务序列，是恢复和控制并发的基本单位，既可以由用户手动执行，也可以由某种数据库程序自动执行。

事务究竟有什么价值呢？

转账：

张三像李四转账一万块钱;

业务操作： ---存在两个操作： 业务中，必须要保证两个操作都成功才行！

- 1、张三的账户----存款减去10000元
- 2、同时李四的账户----存款数量要加上10000元。

张三账户：50000元 转出了10000元 ===40000

李四账户: 40000元 接受了10000元 =====500000

2.事务分类

2.1、显式事务

用 `begin transaction` 明确指定事务的开始，由 `commit transaction` 提交事务、`rollback transaction` 回滚事务到事务结束。

2.2、隱式事務

通过设置 `set implicit_transactions on` 语句，将隐式事务模式设置为打开。当以隐式事务模式操作时，不必使用 `begin transaction` 开启事务，当一个事务结束后，这个模式会自动启用下一个事务。只需使用 `commit transaction` 提交事务或 `rollback transaction` 回滚事务即可。

2.3、自动提交事务

SQL Server的默认模式，它将每条单独的T-SQL语句视为一个事务。如果成功执行，则自动提交，否则回滚。

3、事务的特性

3.1、原子性

保证任务中的所有操作都执行完毕；否则，事务会在出现错误时终止，并回滚之前所有操作到原始状态。

3.2、一致性

事务必须使数据库从一个一致性状态变换到另一个一致性状态。。

3.3、持久性

保证任务中的所有操作都执行完毕；否则，事务会在出现错误时终止，并回滚之前所有操作到原始状态。

持久性是指一个事务一旦被提交了，那么对数据库中的数据的改变就是永久性的，即便是在数据库系统遇到故障的情况下也不会丢失提交事务的操作。

3.4、隔离性

保证不同的事务相互独立、透明地执行。

4、事务不隔离导致的问题

以上介绍完事务的四大特性(简称ACID)，现在重点来说明下事务的隔离性，当多个线程都开启事务操作数据库中的数据时，数据库系统要能进行隔离操作，以保证各个线程获取数据的准确性，在介绍数据库提供的各种隔离级别之前，我们先看看如果不考虑事务的隔离性，会发生的几种问题：

更新丢失（Lost update）、脏读（Dirty Reads）、不可重复读（Non-repeatable Reads）

4.1、更新丢失

两个事务都同时更新一行数据，但是第二个事务却中途失败退出，导致对数据的两个修改都失效了。这是因为系统没有执行任何的锁操作，因此并发事务并没有被隔离开来。

4.2、脏读

一个事务开始读取了某行数据，但是另外一个事务已经更新了此数据但没有能够及时提交。这是相当危险的，因为很可能所有的操作都被回滚。

当一个事务正在多次修改某个数据，而在这个事务中这多次的修改都还未提交，这时一个并发的事务来访问该数据，就会造成两个事务得到的数据不一致。

```
--用户A向用户B转账1000元，B的钱增加
update account set money=money+1000 where name='B';

--此时A通知B我给你转钱了，A的钱减少
update account set money=money - 1000 where name='A';
```

当只执行第一条SQL时，A通知B查看账户，B发现确实钱已到账（此时即发生了脏读）而之后无论第二条SQL是否执行，只要该事务不提交，则所有操作都将回滚，那么当B以后再次查看账户时就会发现钱其实并没有转。

4.3、不可重复读

不可重复读是指在对于数据库中的某个数据，一个事务范围内多次查询却返回了不同的数据值，这是由于在查询间隔，被另一个事务修改并提交了

(1) **虚读**：事务T1读取某一数据后，事务T2对其做了修改，当事务T1再次读该数据时得到与前一次不同的值。

(2) **幻读**：事务在操作过程中进行两次查询，第二次查询的结果包含了第一次查询中未出现的数据或者缺少了第一次查询中出现的数据（这里并不要求两次查询的SQL语句相同）。

这是因为在两次查询过程中有另外一个事务插入数据造成的。

5、事务的隔离级别

5.1、未提交读取（相当于with(nolock)）：第一级别

也称未授权读取：允许脏读取，但不允许更新丢失。

如果一个事务已经开始写数据，则另外一个事务则不允许同时进行写操作，但允许其他事务读此行数据。

该隔离级别可以通过“排他写锁”实现。

缺点：会产生脏读、不可重复读、幻读。

案例解读：以前交易所炒股的时候，股民老王购买了5000股，操作员操作录入（此时开启事务），操作时手误，多输入了一个0，数据保存但是未提交。

此时老王查询自己的持股信息，发现自己居然有50000股，瞬间血压升高，昏倒在地。

然后操作员发现自己录入错误，修改成正确的信息，录入完成（事务结束）。

老王被救醒后，哆嗦这查询自己的持股，发现只有5000，没有增减，他之前看到的就是脏读数据。

解决方案：采用更高级的隔离机制，如提交读。

案例实操

初始化表

```
SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO

CREATE TABLE [dbo].[StudentsScore](
    [Id] [int] IDENTITY(1,1) NOT NULL,
    [Number] [int] NULL,
    [C#] [decimal](2, 0) NULL,
    [SqlDB] [decimal](2, 0) NULL,
    [Java] [decimal](2, 0) NULL,
    [Python] [decimal](2, 0) NULL,
    CONSTRAINT [PK_StudentsScore] PRIMARY KEY CLUSTERED
(
    [Id] ASC
```

```

)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON,
ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]
GO

USE [ZhaoxiEdu]
GO
INSERT INTO [dbo].[StudentsScore] ([Number] ,[C#] ,[SqlDB] ,[Java] ,[Python])
VALUES (10000 ,80 ,90 ,85 ,70)
GO

```

5.1.1、未提交读取 (READ UNCOMMITTED)

READ UNCOMMITTED 事务隔离级别根本就没有提供事务间的隔离，它允许违反并发性原则的最基本形式之一 -- 脏读。当一个事务能够读取另一个事务中已经Update但尚未Commit的数据时，“脏读”就发生了。READ UNCOMMITTED 读操作不申请锁，运行读取未提交的修改，也就是允许读脏数据，读操作不会影响写操作请求排他锁。

Session-1

SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED
BEGIN TRANSACTION
SELECT * FROM StudentsScore WHERE
Number='10000'

结果

消息

	Id	Number	C#	SqlDB	Java	Python
	1	10000	81	90	85	70

Session-2

BEGIN TRANSACTION
UPDATE StudentsScore SET C#=C#+1 WHERE
Number='10000'
SELECT * FROM StudentsScore WHERE
Number='10000'

结果

消息

	Id	Number	C#	SqlDB	Java	Python
	1	10000	81	90	85	70

SELECT * FROM StudentsScore WHERE
Number='10000'

ROLLBACK TRANSACTION

结果

消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	81	90	85	70

ROLLBACK TRANSACTION

READ UNCOMMITTED 常应用于：单用户系统；系统中两个事务同时访问同一资源的可能性为零或几乎为零；

5.2、提交读取（Oracle和SQLServer默认的）：第二级别

这是大多数数据库系统的默认隔离级别（Oracle和SQLServer默认的）。

也称为授权读取：允许 **不可重复读取**，但不允许脏读取。

这可以通过“瞬间共享读锁”和“排他写锁”实现。读取数据的事务允许其他事务继续访问该行数据，但是未提交的写事务将会禁止其他事务访问该行。

缺点：会产生不可重复读、幻读。

案例解读：股市升高后，老王查看自己持有5000股，就想卖掉4000股，在老王卖股票的时候，老王的老婆看股市太高，就登录老王的账号，卖掉3000股。

当老王想卖股票时，发现自己只有2000股，不是之前看到的5000股，这就是不可重复读问题。

解决方案：采用更高级的隔离机制，如可重复读。

案例实操

通过仅允许一个事务读取另一个事务中已经提交的数据，READ COMMITTED 事务隔离级别防止了“脏读”问题。这是SQL Server中默认的事务隔离级别。

它是SQL SERVER默认的隔离级别，可以避免读取未提交的数据，隔离级别比**READ UNCOMMITTED**未提交读的级别更高；该隔离级别读操作之前首先申请并获得共享锁，允许其他读操作读取该锁定的数据，但是写操作必须等待锁释放，一般读操作读取完就会立刻释放共享锁。

Session-1

SET TRANSACTION ISOLATION LEVEL
READ COMMITTED;BEGIN
TRANSACTION;SELECT * FROM
StudentsScore WHERE Number='10000'

结果 消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	80	90	85	70

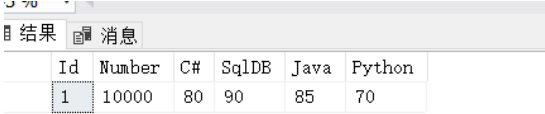
正在执行查询...

Session-2

BEGIN TRANSACTION;UPDATE
StudentsScore SET C#=C#+1 WHERE
Number='10000';SELECT * FROM
StudentsScore WHERE Number='10000'

结果 消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	81	90	85	70

Session-1	Session-2
	ROLLBACK TRANSACTION
SELECT * FROM StudentsScore WHERE Number='10000' 	
ROLLBACK TRANSACTION	

注意：但是由于READ COMMITTED读操作一完成就立即释放共享锁,读操作不会在一个事务过程中保持共享锁,也就是说在一个事务的两个查询过程之间有另一个会话对数据资源进行了更改,会导致一个事务的两次查询得到的结果不一致,这种现象称之为不可重复读。

5.3、可重复读取（相当于(HOLDLOCK))：第三级别

MySQL的默认事务隔离级别。

可重复读取（Repeatable Read）：禁止 **不可重复读取** 和脏读取，但是有时可能出现幻读数据。

这可以通过“共享读锁”和“排他写锁”实现。读取数据的事务将会禁止写事务（但允许读事务），写事务则禁止任何其他事务。

缺点：会产生幻读。

问题解读：股市忽涨忽跌，老王焦虑不安，按捺不住，想把持有的多种股票全部抛掉。与此同时，老王老婆听信专家所言，使用老王的账号买了某只神股。

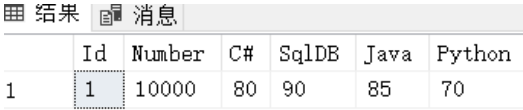
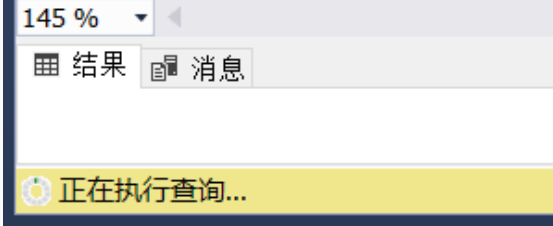
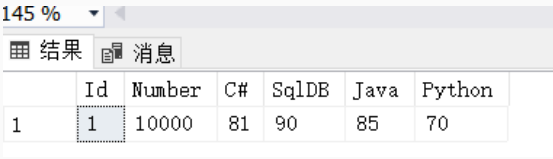
老王抛掉所有股票后，查看自己的持股，猛然发现自己居然还持有一只股票，瞬间觉得一脸懵逼，这就是幻读导致。

解决方案：采用更高级的隔离机制，序列化。

案例实操

正如你在前一个事务隔离级别的步骤2所看到的，Session 2中的事务能够修改已经被Session 1中的事务读取的数据。正像真实场景中所描述的，这可能导致“LOST UPDATE”。REPEATABLE READ 事务隔离级别不允许这种情况发生，因为它违背了REPEATABLE READ原则。换句话说，Session 1中的事务读取同一数据可能会产生不同的结果。

该级别保证在一个事务中的两个读操作之间，其他的事务不能修改当前事务读取的数据，该级别事务获取数据前必须先获得共享锁同时获得的共享锁不立即释放一直保持共享锁至事务完成，所以此隔离级别查询完并提交事务很重要。

Session-1	Session-2
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; BEGIN TRANSACTION; SELECT * FROM StudentsScore WHERE Number='10000' 	
BEGIN TRANSACTION; UPDATE StudentsScore SET C#=C#+1 WHERE Number='10000'; 	SELECT * FROM StudentsScore WHERE Number='10000' 
ROLLBACK TRANSACTION	

5.4序列化（这是最高的隔离级别）：第四级别

序列化（Serializable）：提供严格的事务隔离。

它要求事务 **序列化** 执行，事务只能一个接着一个地执行，不能并发执行。

仅仅通过“行级锁”是无法实现事务序列化的，必须通过其他机制保证新插入的数据不会被刚执行查询操作的事务访问到。

缺点：可以解决并发事务的所有问题。但是效率底下，消耗数据库性能，一般不使用。

隔离级别越高，越能保证数据的完整性和一致性，但是对并发性能的影响也越大。

对于多数应用程序，可以优先考虑把 **数据库系统** 的隔离级别设为Read Committed。它能够避免脏读取，而且具有较好的并发性能。

尽管它会导致 **不可重复读**、幻读和第二类丢失更新这些并发问题，在可能出现这类问题的个别场合，可以由应用程序采用 **悲观锁** 或 **乐观锁** 来控制。

案例实操

对于前面的REPEATABLE READ能保证事务可重复读，但是事务只锁定查询第一次运行时获取的数据资源（数据行），而不能锁定查询结果之外的行，就是原本不存在于数据表中的数据。因此在一个事务中当第一个查询和第二个查询过程之间，有其他事务执行插入操作且插入数据满足第一次查询读取过滤的条件时，那么在第二次查询的结果中就会存在这些新插入的数据，使两次查询结果不一致，这种读操作称之为幻读。

为了避免幻读需要将隔离级别设置为 SERIALIZABLE 。为了向你展示SERIALIZABLE 事务隔离级别防止的并发性问题，本例我们从REPEATABLE READ 事务隔离级别开始。

Session-1	Session-2																					
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; BEGIN TRANSACTION; SELECT * FROM StudentsScore WHERE Number='10000'; 结果 消息 <table><tr><th></th><th>Id</th><th>Number</th><th>C#</th><th>SqlDB</th><th>Java</th><th>Python</th></tr><tr><td>1</td><td>1</td><td>10000</td><td>80</td><td>90</td><td>85</td><td>70</td></tr></table>		Id	Number	C#	SqlDB	Java	Python	1	1	10000	80	90	85	70	BEGIN TRANSACTION; INSERT INTO [dbo].[StudentsScore] ([Number] ,[C#] ,[SqlDB] ,[Java] ,[Python]) VALUES (10000 ,90 ,90 ,90 ,90); Commit TRANSACTION 145 % 消息 (1 行受影响) 完成时间: 2022-09-15T17:26:06.3238989+08:00							
	Id	Number	C#	SqlDB	Java	Python																
1	1	10000	80	90	85	70																
SELECT * FROM StudentsScore WHERE Number='10000'; 结果 消息 <table><tr><th></th><th>Id</th><th>Number</th><th>C#</th><th>SqlDB</th><th>Java</th><th>Python</th></tr><tr><td></td><td>1</td><td>10000</td><td>80</td><td>90</td><td>85</td><td>70</td></tr><tr><td></td><td>18</td><td>10000</td><td>90</td><td>90</td><td>90</td><td>90</td></tr></table>		Id	Number	C#	SqlDB	Java	Python		1	10000	80	90	85	70		18	10000	90	90	90	90	
	Id	Number	C#	SqlDB	Java	Python																
	1	10000	80	90	85	70																
	18	10000	90	90	90	90																
ROLLBACK TRANSACTION SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; BEGIN TRANSACTION; SELECT * FROM StudentsScore WHERE Number='10000'; 145 % 结果 消息 <table><tr><th></th><th>Id</th><th>Number</th><th>C#</th><th>SqlDB</th><th>Java</th><th>Python</th></tr><tr><td>1</td><td>1</td><td>10000</td><td>80</td><td>90</td><td>85</td><td>70</td></tr><tr><td>2</td><td>18</td><td>10000</td><td>90</td><td>90</td><td>90</td><td>90</td></tr></table>		Id	Number	C#	SqlDB	Java	Python	1	1	10000	80	90	85	70	2	18	10000	90	90	90	90	
	Id	Number	C#	SqlDB	Java	Python																
1	1	10000	80	90	85	70																
2	18	10000	90	90	90	90																
	BEGIN TRANSACTION; INSERT INTO [dbo].[StudentsScore] ([Number] ,[C#] ,[SqlDB] ,[Java] ,[Python]) VALUES (10000 ,90 ,90 ,90 ,90); Commit TRANSACTION; 145 % 结果 消息 正在执行查询...																					
ROIIBACK TRANSACTION																						

Session-1	Session-2
	45 % ▾ 消息 命令已成功完成。 完成时间: 2022-09-15T17:37:30.9841825+08:00

SNAPSHOT 快照分为 SNAPSHOT和READ COMMITTED SNAPSHOT两种隔离（可以把事务已经提交的行的上一版本保存在TEMPDB数据库中）：

[1] SNAPSHOT隔离级别在逻辑上与SERIALIZABLE类似；

[2] READ COMMITTED SNAPSHOT隔离级别在逻辑上与 READ COMMITTED类似；

不过在快照隔离级别下读操作不需要申请获得共享锁，所以即便是数据已经存在排他锁也不影响读操作。而且仍然可以得到和SERIALIZABLE与READ COMMITTED隔离级别类似的一致性；如果目前版本与预期的版本不一致，读操作可以从TEMPDB中获取预期的版本。

如果启用任何一种基于快照的隔离级别，DELETE和UPDATE语句在做出修改前都会把行的当前版本复制到TEMPDB中，而INSERT语句不需要在TEMPDB中进行版本控制，因为此时还没有行的旧数据。

无论启用哪种基于快照的隔离级别都会对更新和删除操作产生性能的负面影响，但是有利于提高读操作的性能因为读操作不需要获取共享锁。

5.5、快照

(1)SNAPSHOT 在SNAPSHOT隔离级别下,当读取数据时可以保证操作读取的行是事务开始时可用的最后提交版本。

(2)同时SNAPSHOT隔离级别也满足前面的已提交读,可重复读,不幻读;该隔离级别实用的不是共享锁,而是行版本控制。

案例实操

在上述例1到例4中，防止并发性问题的同时也降低了数据的可访问性。先是不允许Read，然后是不允许Update，不允许Insert。

SNAPSHOT事务隔离级别防止了之前那些隔离级别所能防止的许多并发性问题，同时降低了与之相关的成本。它允许更高的数据可用性。

通过在事务开始前在

中使用row versions创建一份数据库的虚拟快照，SNAPSHOT事务隔离级别完成了此壮举。此后它只允许事务访问该数据库虚拟快照。这种方法被称做“基于版本控制的隔离” (versioning-based isolation) 。

使用versioning-based isolation，事务仅能看到虚拟快照中的数据。因此，其他事务仍然能够访问同一数据，只要它们不去修改已经被第一个事务修改过的数据就好。如果那样做了（企图修改数据），那么，那些事务将会被回滚并以错误消息终止。

只有当数据库中启用SNAPSHOT事务隔离级别的开关打开后，才能使用它。打开此开关将告知数据库去设置版本化环境。理解这一点很重要，因为，一旦版本化开启，数据库会有维护版本化的开销，无论是否有事务正在使用SNAPSHOT事务隔离级别。

在SNAPSHOT隔离级别下,当读取数据时可以保证操作读取的行是事务开始时可用的最后提交版本。同时SNAPSHOT隔离级别也满足前面的已提交读,可重复读,不幻读;该隔离级别实用的不是共享锁,而是行版本控制,使用SNAPSHOT隔离级别首先需要在数据库级别上设置相关选项。

Session01

USE ZhaoxiEdu;
GO
ALTER DATABASE ZhaoxiEdu SET
ALLOW_SNAPSHOT_ISOLATION ON;
GO
SELECT * FROM [ZhaoxiEdu].[dbo].[StudentsScore] where
ID=1;
SET TRANSACTION ISOLATION LEVEL SNAPSHOT
GO
BEGIN TRANSACTION;

145 %

结果

消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	70	90	85	70

Session02

USE ZhaoxiEdu;
GO
BEGIN TRANSACTION;
UPDATE [ZhaoxiEdu].
[dbo].[StudentsScore] set
[C#]=[C#]-5 where ID=1;
SELECT * FROM
[ZhaoxiEdu].[dbo].
[StudentsScore] where
ID=1;

45 %

结果

消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	65	90	85	70

SELECT * FROM [ZhaoxiEdu].[dbo].[StudentsScore] where
ID=1;

结果

消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	70	90	85	70

COMMIT TRANSACTION
GO

消息

命令已成功完成。

完成时间: 2022-09-19T11:03:26.5648026+08:00

Session01	Session02
<pre>SELECT * FROM [ZhaoxiEdu].[dbo].[StudentsScore] where ID=1; UPDATE [ZhaoxiEdu].[dbo].[StudentsScore] set [C#]=[C#]+5 where ID=1; SELECT * FROM [ZhaoxiEdu].[dbo].[StudentsScore] where ID=1; UPDATE [ZhaoxiEdu].[dbo].[StudentsScore] set [C#]=[C#]+5 where ID=1;</pre> 结果 消息 (1 行受影响) 消息 3960, 级别 16, 状态 2, 第 15 行 快照隔离事务由于更新冲突而中止。您无法在数据库'ZhaoxiEdu'中使用快照隔离来直接或间接访问表 'dbo.StudentsScore'。 完成时间: 2022-09-19T11:17:11.7752454+08:00	

5.6、已提交读快照 (READ COMMITTED SNAPSHOT)

READ COMMITTED SNAPSHOT也是基于行版本控制，但是READ COMMITTED SNAPSHOT的隔离级别是读操作之前的最后已提交版本，

而不是事务前的已提交版本，有点类似前面的READ COMMITTED能保证已提交读，但是不能保证可重复读，不能避免幻读，

但是又比 READ COMMITTED隔离级别多出了不需要获取共享锁就可以读取数据。

案例实操

到目前为止，所有的隔离级别都是将事务相互间隔离开来。一旦初始事务完成了，对其他事务变得不可用的资源才又变得可用。READ COMMITTED SNAPSHOT 隔离级别在这点上有所不同，它能够读取其已经被他事务提交的数据。它也是通过数据库开关来打开的。然后，任何使用READ COMMITTED SNAPSHOT 隔离级别的事务将通过版本化起作用。

READ COMMITTED SNAPSHOT也是基于行版本控制，但是READ COMMITTED SNAPSHOT的隔离级别是 “ 读操作之前的最后已提交版本,而不是事务前的已提交版本 ” ，有点类似前面的READ COMMITTED能保证已提交读，但是不能保证可重复读，不能避免幻读，但是又比 READ COMMITTED隔离级别多出了不需要获取共享锁就可以读取数据。

要启用READ COMMITTED SNAPSHOT隔离级别同样需要修改数据库选项。

Session01

```
USE ZhaoxiEdu;
GO
ALTER DATABASE [ZhaoxiEdu] SET SINGLE_USER
WITH ROLLBACK IMMEDIATE ;
ALTER DATABASE [ZhaoxiEdu] SET
READ_COMMITTED_SNAPSHOT ON;
ALTER DATABASE [ZhaoxiEdu] SET MULTI_USER;
GO
SELECT * FROM StudentsScore WHERE ID=1;
SET TRANSACTION ISOLATION LEVEL READ
COMMITTED;
GO
BEGIN TRANSACTION;
```

145 %

结果 消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	60	90	85	70

Session02

```
USE ZhaoxiEdu;
GO
BEGIN TRANSACTION;
UPDATE [ZhaoxiEdu].[dbo].
[StudentsScore] set [C#]=[C#]-5
where ID=1;
SELECT * FROM [ZhaoxiEdu].[dbo].
[StudentsScore] where ID=1;
```

.45 %

结果 消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	55	90	85	70

```
SELECT * FROM [ZhaoxiEdu].[dbo].
[StudentsScore] where ID=1;
```

145 %

结果 消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	60	90	85	70

Session01

SELECT * FROM [ZhaoxiEdu].[dbo].[StudentsScore] where ID=1;

145 %

结果

消息

	Id	Number	C#	SqlDB	Java	Python
1	1	10000	55	90	85	70

UPDATE [ZhaoxiEdu].[dbo].[StudentsScore] set [C#]=[C#]+5 where ID=1;

145 %

消息

(1 行受影响)

完成时间：2022-09-19T12:03:56.6458692+08:00

COMMIT TRANSACTION

Session02

COMMIT TRANSACTION

GO

消息

命令已成功完成。

完成时间：2022-09-19T11:03:26.5648026+08:00

获取事务隔离级别(isolation level)

DBCC USEROPTIONS

7、事务隔离总结

隔离级别	解决的并发性问题	存在的并发性问题
READ UNCOMMITTED (未提交读取)	不适用于并发场合, 适合单用户系统	Dirty Reads, Non-repeatable Reads, Phantom Reads (脏读、不可重复读、虚读)
READ COMMITTED (提交读取)	Dirty Reads (脏读)	Lost Update , Non-repeatable Reads, Phantom Reads (丢失更新、不可重复读取、虚拟读取)
REPEATABLE READ (可重复读取)	Non-repeatable Reads (不可重复读取)	Phantom Reads, potentially Deadlocking (幻像读取, 可能死锁)
SERIALIZABLE (序列化)	Phantom Reads (幻读)	Less Data Availability, potentially Deadlocking (数据可用性降低, 可能出现死锁)
SNAPSHOT (快照)	上述所有并发性问题	事务访问的是虚拟快照, 其他事务Committed的数据对当前事务仍然不可见, 也不允许Update被

隔离级别	解决的问题 解决的并发性	其他事务Updated的数据。 存在的并发性问题
READ COMMITTED	问题	
SNAPSHOT（已提交快照）	上述所有并发 性问题	无

二十一、SqlServer锁

1、SqlServer概述

SQL Server 数据库支持多个用户同时访问数据库，但当用户同时访问数据库时，就会造成并发问题，锁的机制能很好地解决这个问题，保证数据的完整性和一致性；

SQL Server 自带锁机制，若是简单的数据库访问机制，完全能满足用户的需求；但对于数据完全与数据完整性有特殊要求，就必须自动控制锁机制解决；

2、SQL Server 锁机制

锁是处理 SQL Server 中并发问题的最有效手段，当多个事务访问同一数据时，能很好地保证数据的完整性和一致性；

在很多数据库系统中（如DB2、MySQL、Oracle）都有锁机制，其规则也大同小异；

在 SQL Server 中采用系统来管理锁，SQL Server 中采用的是动态加锁的机制；
SQL Server 中有一套默认的锁机制，若用户在使用数据库的过程中不设置任何锁，系统将自动对锁管理；

3、锁模式

在 SQL Server 中有不同的锁，在各种锁的类型中有些是能相互兼容的，锁的类型决定了并发发生时数据资源的范文模式，在 SQL Server 中常用的锁以下 5 种；

3.1、共享锁 (Shared lock)

又称读锁（S锁），共享锁不阻塞其他事务的读操作，但阻塞写操作，同一数据对象A可以共存多个共享锁，这被称为共享锁兼容。。

案例分析

当Session1为数据对象A加上共享锁后，可以对A进行读操作，但不能进行写操作，并且Session2可以再次对A加共享锁，大家都可以正常地读A，但是在A上的共享锁释放之前，任何事务不可以对A进行写操作。

例 1：

```
Session1: select * from [ZhaoxiEdu].[dbo].[StudentsScore]
```



```
Session2:  update [ZhaoxiEdu].[dbo].[StudentsScore] set Python=80
```

理解:

假设Session1先执行, 则Session2必须等待Session1执行完才可以执行。

因为Session2为写操作, 需要为table加一个排他锁, 而数据库规定相同资源不可以同时存在共享锁和排他锁, 所以Session2必须等待Session1执行完, 释放掉共享锁, 才可以加排他锁, 然后执行update。

例 2: (可能发生死锁)

Session1:

```
begin Transaction t1
select * from [ZhaoxiEdu].[dbo].[StudentsScore] with (holdlock)    -- (holdlock)共享锁
update [ZhaoxiEdu].[dbo].[StudentsScore] set Python=80
```

Session2:

```
begin Transaction t2
select * from [ZhaoxiEdu].[dbo].[StudentsScore] with (holdlock)    -- (holdlock)共享锁
update [ZhaoxiEdu].[dbo].[StudentsScore] set Python=67
```

理解:

分析:

假设Session1和Session2同时到达select语句, 都为table加上了共享锁, 那么当Session1、Session2要执行update时, 根据锁机制, 共享锁需要升级为排他锁, 但是排他锁与共享锁不能共存, 要给table加排他锁, 必须等待table上的共享锁全部释放才可以, 可是holdlock的共享锁必须等待事务结束才能释放, 因此Session1和Session2都在等待对方释放共享锁, 形成循环等待, 造成死锁。

例3:

```
Session1:  update [ZhaoxiEdu].[dbo].[StudentsScore] set Python=60 where Id=4
```

```
Session2:  update [ZhaoxiEdu].[dbo].[StudentsScore] set Python=65 where Id=4
```

理解:

此种情况有可能造成等待, 分为id列有索引与无索引两种情况。

(1) id列有索引, 则Session1直接定位到Id=4行, 加排他锁, 更新; Session2直接定位到Id=4行, 加排他锁, 更新。互不影响。

(2) id列无索引, Session1扫描全表, 找到Id=4行, 加排他锁后, Session2为了找到Id=4行, 需要全表扫描, 那么就会为table加共享锁或更新锁或排他锁, 但不管加什么锁, 都需要等待Session1释放Id=4行的排他锁, 不然无法为全表加锁。

死锁可以通过直接对表加排他锁来解决, 即将事务的隔离级别提高至最高级——串行读, 各个事务串行执行, 可是这样虽然避免了死锁, 但是效率太低了。

3.2、更新锁 (Update lock)

更新锁 (U锁)。当Session1给资源A加上更新锁后，代表该资源将在稍后更新，更新锁与共享锁兼容，更新锁可以防止例2里那种一般情况的死锁发生，更新锁会阻塞其他的更新锁和排他锁。因此相同资源上不能存在多个更新锁。

更新锁允许其他事务在更新之前读取资源。

但不可以修改。因为其他事务想获取资源的排他锁时，发现该资源已存在U锁，则等待U锁释放。

在Session1找到需要更新的数据时，更新锁直接转为排他锁，开始更新数据，不需要等待其他事务释放共享锁啥的。

那么就问了，共享锁为什么不可以直接升级为排他锁，而必须等待其他共享锁都释放掉才可以转为排他锁呢？

这就是共享锁和更新锁的一个区别了，共享锁之间是兼容的，但是更新锁之间互不兼容，因此仅有一个更新锁直接转为排他锁是安全的，而多个共享锁问也不问直接转为排他锁，那怎么行呢，排他锁只能有一个的，这就是为什么共享锁需要等待其他共享锁释放才可以升级为排他锁的原因了。

案例分析

例 4:

Session1:

```
begin
select * from [ZhaoxiEdu].[dbo].[StudentsScore] with (updlock)    --- (加更新锁)
update [ZhaoxiEdu].[dbo].[StudentsScore] set C#=70    -- (重点: 这里Session1做update时, 不需要等
Session2释放什么, 而是直接把更新锁升级为排他锁, 然后执行update)
```

Session2:

```
begin
select * from [ZhaoxiEdu].[dbo].[StudentsScore]    -- (T1的更新锁不影响T2的select)
update [ZhaoxiEdu].[dbo].[StudentsScore] set C#=75 -- (T2的update需要等待T1的update执行完)
```

理解:

(1) Session1先到达，Session1的select句对table加更新锁，此时Session2紧接着到达，Session2的select句对table加共享锁，假设Session2的select先执行完，要开始Session2的update，发现table已有更新锁，则Session2等，Session1此时执行完select，然后将更新锁升级为排他锁，开始更新数据，执行完成，事务结束，释放排他锁，此时Session2才开始对table加排他锁并更新。

(2) Session2先到，Session1紧接着，Session2加共享锁 => Session1加更新锁 => 假设Session2先结束select => 试图将共享锁升级为排他锁 => 发现已有更新锁 => 之后的情况相同；

3.3、排他锁

又叫独占锁，写锁，X锁，很容易理解，排他锁阻塞任何锁，假设Session1为资源A加上排他锁，则其他事务不允许对资源A进行任何的读写操作。

案例分析

例 5：(假设id都是自增长且连续的)

```
Session1: update [ZhaoxiEdu].[dbo].[StudentsScore] set C#=70 where id<5
```

```
Session2: update [ZhaoxiEdu].[dbo].[StudentsScore] set C#=75 where id>5
```

理解：

假设Session1先达，Session2随后至，这个过程中Session1会对id<5的记录施加排他锁.但不会阻塞Session2的update。

例6

```
Session1: update [ZhaoxiEdu].[dbo].[StudentsScore] set C#=60 where id<5
```

```
Session2: update [ZhaoxiEdu].[dbo].[StudentsScore] set C#=60 where id>2
```

理解：

假设Session1先达，Session2立刻也到，Session1加的排他锁会阻塞Session2的update。

3.4、意向锁

意向锁，就是说当你给数据加锁时，必须先给他的上级加锁，用来向其他事务表明这段数据中的某些数据正在被加某某锁，你看着办吧。其实是一个节省开销的做法。

案例分析：

例7

```
Session1:

begin tran
select * from [ZhaoxiEdu].[dbo].[StudentsScore] with (xlock) where id=5
--意思是对id=5这一行强加排他锁
```

```
Session2:

begin tran
select * from [ZhaoxiEdu].[dbo].[StudentsScore] with (tablock) --意思是要加表级锁
```

理解：

假设Session1先执行，Session2后执行，Session2执行时，欲加表锁，为判断是否可以加表锁，数据库系统要逐条判断table表每行记录是否已有排他锁，如果发现其中一行已经有排他锁了，就不允许再加表锁了。只是这样逐条判断效率太低了。

实际上，数据库系统不是这样工作的。当Session1的select执行时，系统对表table的id=5的这一行加了排他锁，还同时悄悄的对整个表加了意向排他锁(IX)，当Session2执行表锁时，只需要看到这个表已经有意向排他锁存在，就直接等待，而不需要逐条检查资源了。

常用的意向锁有三种：意向共享锁（Intent Share Lock，简称IS锁）；意向排他锁（Intent Exclusive Lock，简称IX锁）；共享意向排它锁（Share Intent Exclusive Lock，简称SIX锁），共享意向排它锁的意思是，某事务要读取整个表，并更新其中的某些数据。

4、锁的粒度

Microsoft SQL Server 数据库引擎具有多粒度锁定，允许一个事务锁定不同类型的资源；为了减少锁定的开销，数据库引擎自动将资源锁定在适合任务的级别；锁定在较小的粒度（如行）能提高并发度，但开销较高，因为若锁定了许多行，就需要持有更多的锁；锁定在加大的粒度（如表）会降低并发，因为锁定整个表限制了其他事务对表中任意部分的访问；但其开销较低，因为需要维护的锁较少；数据库引擎通常必须获取多粒度级别上才能完整地保护资源，多粒度级别上的所称为层次结构；

资源	说明
RID	用户锁定堆中的单个行的行标识符
KEY	索引中用于可序列化事务中的键范围的行锁
PAGE	数据库中的8kb页，如数据页或索引页
EXTENT	一组连续的八页，如数据页或索引页
HoBT	堆或者B树，用户保护没有聚集索引的表中的树（索引）或堆数据页的锁
TABLE	包括所有数据和索引的整个表
FILE	数据库文件
APPLICATION	应用程序专用的资源
METADATA	元数据锁
ALLOCATION_UNIT	分配单元
DATABASE	整个数据库

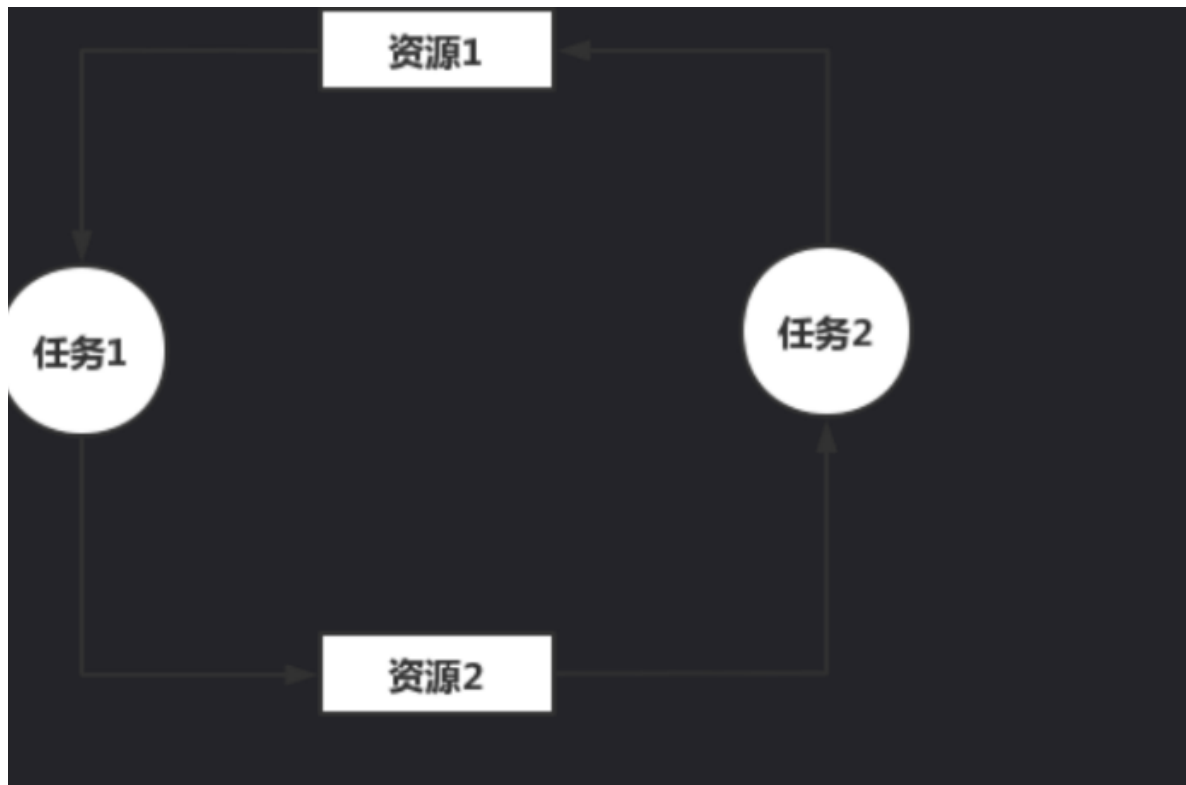
5、查看锁

在 SQL Server数据库中，能通过查看 sys.dm_tran_locks 返回 SQL Server 数据库中有关当前活动的锁的管理的信息；向锁管理器发出的已授予锁或正等待授予锁的每个当前活动请求分别对应一行；结果集中的列大体分为两组：资源组和请求组；

```
select * from sys.dm_tran_locks
```

6、死锁

在两个或多个任务中，若每一个任务都锁定了其他的资源，就会造成永久的阻塞，这种情况就是死锁；



6.1 形成死锁有以下 4 个必要条件：

1. **互斥条件**：资源不能被共享，只能被一个进程使用；
2. **请求与保持条件**：已获得资源的进程能同时申请其他资源；
3. **非剥夺条件**：已分配的资源不能从该进程中被剥夺；
4. **循环等待条件**：多个进程构成环路，且每个进程都在等待相邻进程正在使用的资源；

在一个复杂的数据库系统中很难百分之百地避免死锁，但能按照以下的访问策略减少死锁的发生；

1. 所有事务中以相同的次序使用资源；避免出现循环；
2. 减少事务持有资源的时间，避免事务中的用户交互；
3. 让事务保持在一个批处理中；
4. 由于锁的隔离级别越高共享锁的时间就越长，因此能降低隔离级别来达到减少竞争的目的；

注意： SQL Server 数据库引擎自动检测 SQL Server 中的死锁循环；数据库引擎选择一个会话作为死锁的牺牲品，然后终止当前事务（出现错误）来打断死锁；

二十二、数据库设计

1、数据库设计的重要性

在系统研发中，数据库作为数据的保存介质，那么数据库如何保存业务数据。这就需要开发者来设计了。当数据库比较复杂（如数据量大，表较多，业务关系复杂）时：

1、良好的数据库设计可以：

- 节省数据的存储空间
- 能够保证数据的完整性
- 方便进行数据库应用系统的开发

2、糟糕的数据库设计

- 数据冗余、存储空间浪费
- 数据更新和插入的异常

数据库设计对于开发者来说，是很重要的一块。

2、数据库的生命周期

分为四个阶段

2.1 需求分析阶段

此阶段在软件项目开发周期中的需求分析阶段进行）这个阶段主要是收集并分析用户的业务和数据处理需求

- 收集信息（我们需要与数据的创造者和使用者进行访谈。对访谈获得的信息进行整理、分析，并撰写正式的需求文档。需求文档中需包含：需要处理的数据；数据的自然关系；数据库实现的硬件环境、软件平台等；）
- 标识对象
- 标识每个对象的属性
- 标识对象之间的关系

2.2 逻辑设计阶段

此阶段在软件项目开发周期中的概要设计和详细设计阶段进行，使用ER或UML建模技术，创建概念数据模型图，展示所有数据以及数据间关系。最终概念数据模型必须被转化为范式化的表。

- a. 绘制E-R图
- b. 将E-R图转换为表格
- c. 应用三大范式规范化表格

2.3 物理设计阶段

根据特定数据库管理系统所提供的多种存储结构和存取方法等依赖于具体计算机结构的各项物理设计措施，对具体的应用任务选定最合适的物理存储结构(包括文件类型、索引结构和数据的存放次序与位逻辑等)、存取方法和存取路径等。这一步设计的结果就是所谓“物理数据库”。物理设计阶段遵循以下要求

- a. 数据库物理设计的目标是尽可能优化性能。
- b. 物理设计阶段，表结构可能需要进行重构来满足性能上的需求，这被称为反范式化。
- c. 反范式化。
- d. 实现和维护阶段。

反范式化的步骤包括

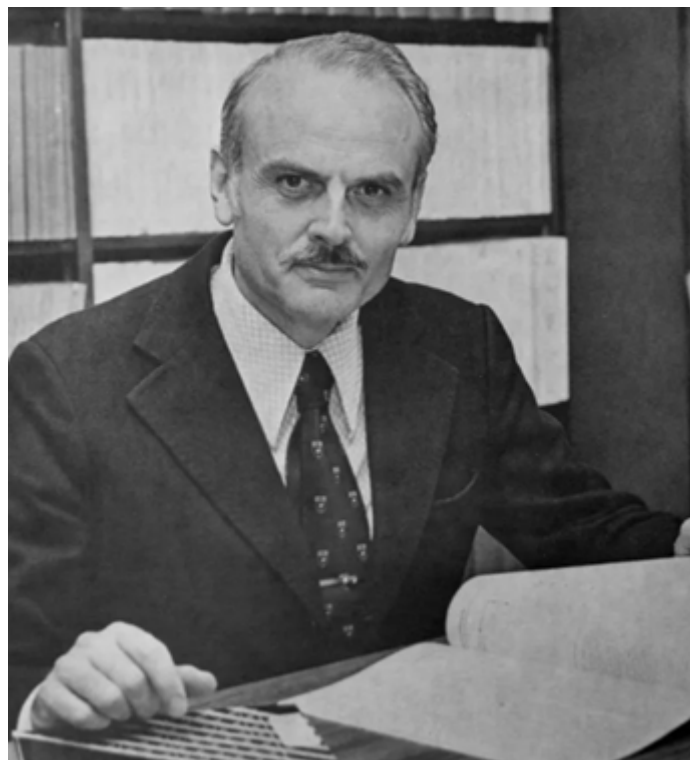
1. 辨别关键性流程，如频繁运行、大容量、高优先级的处理操作
2. 通过增加冗余来提高关键性流程的性能
3. 评估所造成的代价（对查询、修改、存储的影响）和可能损失的数据一致性

实现和维护阶段

1. 当设计完成之后，使用数据库管理系统（DBMS）中的数据定义语言（DDL）来创建数据结构。
2. 数据库创建完成后，应用程序或用户可以使用数据操作语言（DML）来使用（查询、修改等）该数据库。
3. 一旦数据库开始运行，就需要对其性能进行监视。当数据库性能无法满足要求或用户提出新的功能需求时，就需要对该数据库进行再设计与修改。这形成了一个循环：监视 -> 再设计 -> 修改 -> 监视...

3 设计范式

范式：英文名称是 Normal Form，它是英国人 E.F.Codd（**关系数据库**的鼻祖）在上个世纪70年代提出关系数据库模型后总结出来的，范式是关系数据库理论的基础，也是我们在设计数据库结构过程中所要参考的一个建议。



3.1 第一范式(1NF)

第一范式强调的是列的原子性，**信息存储列不可再分**，原子：就是不可再分；即列不能够再分成其他几列。避免字段中信息冗余。

考虑这样一个表：【联系人】（姓名，性别，电话）

如果在实际场景中，一个联系人有家庭电话和公司电话，那么这种表结构设计就没有达到 1NF。要符合 1NF 我们只需把列（电话）拆分，即：【联系人】（姓名，性别，家庭电话，公司电话）。

姓名	身高/体重
小明	170/60
小红	175/60

第一列具备原子性 第二列不具备原子性

3.2 第二范式(2NF)

所谓第二范式，第一，表必须符合第一范式的规则。第二，每列必须依赖主键。

在实际中，一个实体还需要分割成两个表，这样就有两个实体。表头是两个有关系的表中父表的一部分。表头的信息只需要存储一次，而细节表存储可能有多个实例信息。表头通常保持原表的名称，细节表通常以表头名称开头，添加一些细节表的信息。

表1:

订单ID (主键)	公司名称	货物
1	一公司	甲牌笔记本/甲手机/甲牌缝纫机
2	二公司	乙牌笔记本/乙牌开水壶
3	三公司	丁牌电话

不满足第一范式，需要调整

表2:

订单ID (主键)	公司名称	货物
1	一公司	甲牌笔记本
1	一公司	甲手机
1	一公司	甲牌缝纫机
2	二公司	乙牌笔记本
2	二公司	乙牌开水壶
3	三公司	丁牌电话

表3:

订单ID (主键)	订单时间	公司名称
1	2021/6/6	一公司
2	2021/7/7	二公司
3	2021/9/9	三公司

表4:

订单ID (主键)	行项号 (主键)	货物
1	1	甲牌笔记本
1	2	甲手机
1	3	甲牌缝纫机
2	1	乙牌笔记本
2	2	乙牌开水壶
3	1	丁牌电话

3.3 第三范式(3NF)

第三范式使得表中所有的列不仅仅依赖于某个事物，而是依赖于正确的事物。

有以下三个规则：

- 1、表必须符合2NF。
- 2、任何列都不能依赖于非键列。
- 3、不可以有派生的数据（不建议冗余）。

表1：

订单ID (主键)	行项号 (主键)	型号	描述	单价	数量	总价
1	1	a123	甲公司最新款笔记本	5000	2	10000
1	2	a34	全新5G手机	4000	3	12000
1	3	a67	收藏价值很高的缝纫机	8888	1	8888
2	1	b57	乙公司旗舰机	7000	3	21000
2	2	b89	咕噜咕噜的开水壶	40	20	800
3	1	c444	叮叮叮的电话	200	4	800

可以看出描述只跟型号有关，而与键无关。

（描述依赖于型号而不是键）。所以我们可以把表拆分成下面两个表。

表2：

订单ID (主键)	行项号 (主键)	型号	单价	数量	总价
1	1	a123	5000	2	10000
1	2	a34	4000	3	12000
1	3	a67	8888	1	8888
2	1	b57	7000	3	21000
2	2	b89	40	20	800
3	1	c444	200	4	800

表3：

型号	描述
a123	甲公司最新款笔记本
a34	全新5G手机
a67	收藏价值很高的缝纫机
b57	乙公司旗舰机
b89	咕噜咕噜的开水壶
c444	叮叮叮的电话

此处有个总价的列。总价可以根据单价和数量计算出（即为派生数据。）。这在规范中使用不允许的，删除该列即可，在需要时进行计算，避免占用多的空间。

4、绘制E-R图

E-R图也称实体-联系图(Entity Relationship Diagram)，提供了表示实体类型、属性和联系的方法，用来描述现实世界的概念模型图。

4.1、工具准备

PowerDesigner（推荐使用）

ApexSQLLog

Tuning Advisor

SSMSTools

DBDiff

PAL Tool

RML

SqlNexus

SQLIO和SQLIOSim

PowerDesigner 模型设计工具

链接: https://pan.baidu.com/s/1A8KTCMLaSbR0j_vOsPFvwg?pwd=6666

提取码: 6666

4.2、生成数据库

- a. PowerDesigner生成脚本----通过脚本执行初始化数据库
- b. PowerDesigner直接连接数据库服务器, 初始化数据库

4.3、ODBC & 如何配置ODBC

开放数据库连接 (Open Database Connectivity, **ODBC**) 是为解决 **异构数据库** 间的数据共享而产生的, 现已成为WOSA(The Windows Open System Architecture(Windows开放 **系统体系结构**))的主要部分和基于Windows环境的一种数据库访问接口标准。ODBC 为异构数据库访问提供统一接口, 允许应用程序以SQL 为数据存取标准, 存取不同DBMS管理的数据; 使应用程序直接操纵DB中的数据, 免除随DB的改变而改变。用ODBC 可以访问各类计算机上的DB文件, 甚至访问如Excel 表和ASCII数据文件这类非数据库对象。

开始配置:

控制面板----> 系统和安全----->管理工具----> ODBC数据源--->配置用户DSN

4.4、E-R图生成数据库 && 通过数据库生成E-R图

二十三、执行计划

1、什么是执行计划

用户提交的 **sql** 语句, 数据库查询优化器, 经过分析生成多个数据库可以识别的高效执行查询方式。然后优化器会在众多执行计划中找出一个资源使用最少, 而不是最快的执行方案, 给你展示出来, 可以是文本格式, 也可以是图形化的执行方案。

2、为什么要读懂执行计划?

首先执行计划让你知道你复杂的 **sql** 到底是怎么执行的, 有没有按照你想的方案执行, 有没有按照最高效的方式执行, 使用了众多索引的哪一个, 怎么排序, 怎么合并数据的, 有没有造成不必要资源浪费等等。官方数据显示, 执行 **t-sql** 存在问题, 80%都可以在执行计划中找到答案。

3、这对于图形化的执行计划分析

执行计划，可以以文本，图形化展示出来。我们主要以图形化执行计划主导进行分析，然而执行计划中包含78个可用的操作符。

图片介绍：[https://msdn.microsoft.com/zh-cn/library/ms175913\(v=sql.90\).aspx](https://msdn.microsoft.com/zh-cn/library/ms175913(v=sql.90).aspx)

图标	运算符
	Arithmetic Expression 【算术表达式】
	Assert 【明确肯定】
	Bitmap 【位图】
	Bookmark Lookup 【书签查找】
	Clustered Index Delete 【聚集索引删除】
	Clustered Index Insert 【聚集索引插入】
	Clustered Index Scan 【聚集索引扫描】
	Clustered Index Seek 【聚集索引查找】
	Clustered Index Update 【聚集索引更新】
	Collapse 【崩溃】
	Compute Scalar 【标量计算】
	Concatenation 【串联】
	Constant Scan 【恒定扫描】
	Delete 【删除】
	Deleted Scan 【删除扫描】
	Eager Spool
	Filter 【过滤】
	Hash Match 【哈希匹配】
	Hash Match Root 【哈希匹配根】
	Hash Match Team 【哈希匹配团队】
	Insert 【插入】
	Inserted Scan 【插入的扫描】

图标	运算符
	Iterator Catchall 【迭代器捕获】
	Lazy Spool
	Log Row Scan 【日志行扫描】
	Merge Interval 【合并间隔】
	Merge Join 【合并连接】
	Nested Loops 【嵌套循环】
	Nonclustered Index Delete 【非聚集索引删除】
	Nonclustered Index Insert 【非聚集索引插入】
	Nonclustered Index Scan 【非聚集索引扫描】
	Nonclustered Index Seek 【非聚集索引查找】
	Nonclustered Index Spool 【非聚集索引假脱机】
	Nonclustered Index Update 【非聚集索引更新】
	Online Index Insert
	Parameter Table Scan 【参数表扫描】
	Remote Delete 【远程删除】
	Remote Insert 【远程插入】
	Remote Query 【远程插入】
	Remote Scan 【远程扫描】
	Remote Update 【远程修改】
	RID Lookup 【RID查找】
	Row Count Spool
	Segment

图标	运算符
	Sequence 【序列】
	SequenceProject 【序列项目】
	Sort 【分类】
	Split 【拆分】
	Spool
	Stream Aggregate
	Switch 【分支】
	Table Delete
	Table Insert
	Table Scan
	Table Spool
	Table Update
	Table-valued Function
	Top
	UDX
	Update

图形执行计划中显示的下列图标表示的是 **SQL Server** 用于执行语句的 **游标逻辑运算符和物理 Showplan 运算符**。

图标	游标物理运算符
	Cursor Catchall
	Dynamic
	Fetch Query
	Keyset
	Population Query
	Refresh Query
	Snapshot

图形执行计划中显示的下列图标表示的是 **SQL Server** 用于执行语句的 **Parallelism Showplan 运算符** 物理运算符。

图标	并行物理运算符
	Distribute Streams
	Repartition Streams
	Gather Streams

图形执行计划中显示的下列图标表示的是 **SQL Server** 使用的 **Transact-SQL** 语言元素。

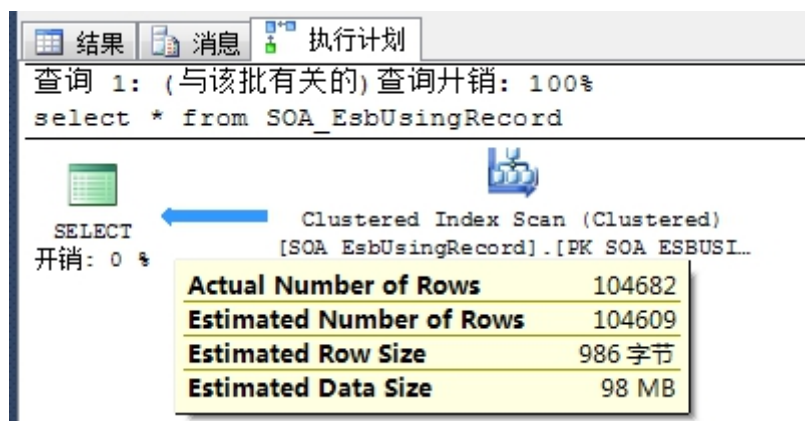
图标	语言元素
	Assign 【分配】
	Convert 【转换】
	Declare 【声明】
	If 【判断】
	Intrinsic
	Language Element Catchall
	Result
	While

4、清除执行计划

```
DBCC FREEPROCCACHE  
DBCC DROPCLEANBUFFERS
```

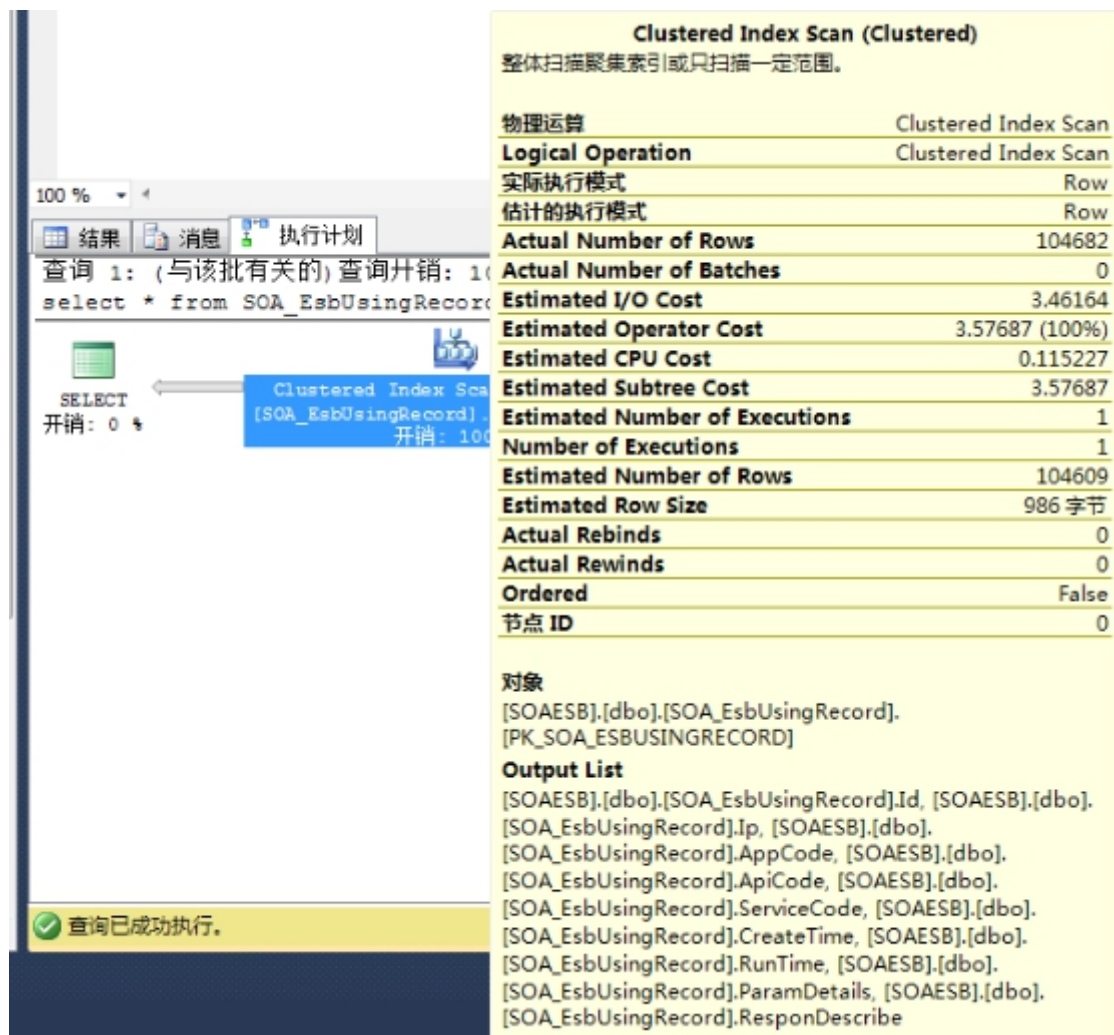
5、如何查看执行计划

5.1、连线



- 越粗表示扫描影响的行数愈多。
- Actual Number of Rows 扫描中实际影响的行数。
- Estimated Number of Rows 预估扫描影响的行数。
- Estimated row size 操作符生成的行的估计大小（字节）。
- Estimated Data Size 预估影响的数据的大小。

5.2、Tooltips ,当前步骤执行信息



Clustered Index Scan (Clustered)
整体扫描聚集索引或只扫描一定范围。

物理运算	Clustered Index Scan
Logical Operation	Clustered Index Scan
实际执行模式	Row
估计的执行模式	Row
Actual Number of Rows	104682
Actual Number of Batches	0
Estimated I/O Cost	3.46164
Estimated Operator Cost	3.57687 (100%)
Estimated CPU Cost	0.115227
Estimated Subtree Cost	3.57687
Estimated Number of Executions	1
Number of Executions	1
Estimated Number of Rows	104609
Estimated Row Size	986 字节
Actual Rebinds	0
Actual Rewinds	0
Ordered	False
节点 ID	0

对象
[SOAESB].[dbo].[SOA_EsbUsingRecord].
[PK_SOA_ESBUSINGRECORD]

Output List
[SOAESB].[dbo].[SOA_EsbUsingRecord].Id, [SOAESB].[dbo].[SOA_EsbUsingRecord].Ip, [SOAESB].[dbo].[SOA_EsbUsingRecord].AppCode, [SOAESB].[dbo].[SOA_EsbUsingRecord].ApiCode, [SOAESB].[dbo].[SOA_EsbUsingRecord].ServiceCode, [SOAESB].[dbo].[SOA_EsbUsingRecord].CreateTime, [SOAESB].[dbo].[SOA_EsbUsingRecord].RunTime, [SOAESB].[dbo].[SOA_EsbUsingRecord].ParamDetails, [SOAESB].[dbo].[SOA_EsbUsingRecord].ResponDescribe

查询 1: (与该批有关的) 查询开销: 100%
select * from SOA_EsbUsingRecord

SELECT
开销: 0 %

Clustered Index Scan
[SOA_EsbUsingRecord].
开销: 100%

查询已成功执行。

Note: 这个tips的信息告诉我们执行的对象是什么, 采用的操作操作是什么, 查找的数据是什么, 使用的索引是什么, 排序与否, 预估 cpu、I/O 影响行数, 实际行数等信息。

5.3、Table Scan (表扫描)



当表中没有聚集索引, 又没有合适索引的情况下, 会出现这个操作。这个操作是很耗性能的, 他的出现也意味着优化器要遍历整张表去查找你所需要的数据。



5.4、Clustered Index Scan(聚集索引扫描)、Index Scan (非聚集索引扫描)

这个图标两个操作都可以使用, 一个聚集索引扫描, 一个是非聚集索引扫描。

聚集索引扫描: 聚集索引的数据体积实际是就是表本身, 也就是说表有多少行多少列, 聚集所有就有多少行多少列, 那么聚集索引扫描就跟表扫描差不多, 也要进行全表扫描, 遍历所有表数据, 查找出你想要的。

非聚集索引扫描: 非聚集索引的体积是根据你的索引创建情况而定的, 可以只包含你要查询的列。那么进行非聚集索引扫描, 便是你非聚集中包含的列的所有行进行遍历, 查找出你想要的。

5.5、Key Lookup(键值查找)



首先需要说的是查找，查找与扫描在性能上完全不是一个级别的，扫描需要遍历整张表，而查找只需要通过键值直接提取数据，返回结果，性能要好。

当你查找的列没有完全被非聚集索引包含，就需要使用键值查找在聚集索引上查找非聚集索引不包含的列。

5.6、RID Lookup (RID查找)



5.7、Clustered Index Seek (聚集索引查找)、Index Seek (非聚集索引查找)

聚集索引查找和非聚集索引查找都是使用该图标。

聚集索引查找：聚集索引包含整个表的数据，也就是在聚集索引的数据上根据键值取数据。

非聚集索引查找：非聚集索引包含创建索引时所包含列的数据，在这些非聚集索引的数据上根据键值取数据。

5.8、Hash Match



这个图标有两种地方用到，一种是表关联，一种是数据聚合运算时。

再分别说这两种运算的前面，我先说说Hashing（编码技术）和Hash Table（数据结构）。

Hashing：在数据库中根据每一行的数据内容，转换成唯一符号格式，存放到临时哈希表中，当需要原始数据时，可以给还原回来。类似加密解密技术，但是他能更有效的支持数据查询。

Hash Table：通过hashing处理，把数据以key/value的形式存储在表格中，在数据库中他被放在tempdb中。

接下来，来说说Hash Math的表关联跟行数据聚合是怎么操作运算的。

表关联：



如上图，关联两个数据集时，Hash Match会把其中较小的数据集，通过Hashing运算放入 HashTable 中，然后一行一行的遍历较大的数据集与 HashTable 进行相应的匹配拉取数据。

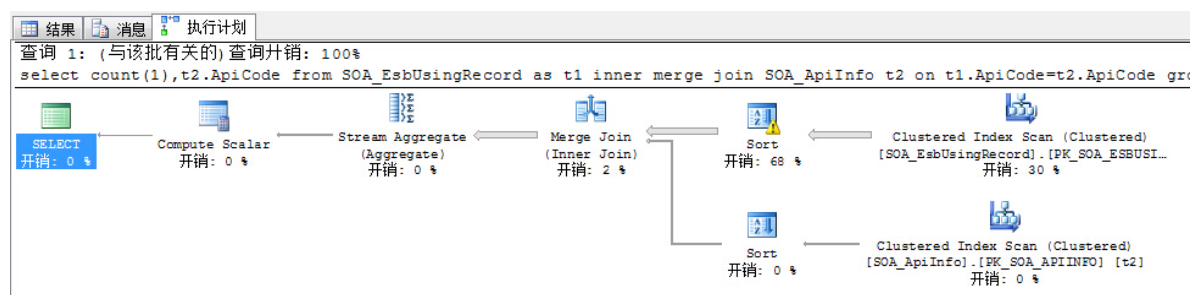
数据聚合：当查询中需要进行Count/Sum/Avg/Max/Min时，数据可能会采用把数据先放在内存中的 HashTable 中然后进行运算。

5.9、Nested Loops



这个操作符号，把两个不同列的数据集汇总到一张表中。提示信息中的Output List中有两个数据集，下面的数据集 (inner set) 会——扫描与上面的数据集 (out set)，扫描完为止，这个操作才算是完成。

5.10、Merge Join



这种关联算法是对两个已经排过序的集合进行合并。如果两个聚合是无序的则将先给集合排序再进行一一合并，由于是排过序的集合，左右两个集合自上而下合并效率是相当快的

5.11、Sort (排序)



对数据集进行排序，需要注意的是，有些数据集在索引扫描后是自带排序的。

5.12、Filter (筛选)



根据出现在having之后的操作运算符，进行筛选

5.13、Computer Scalar



在需要查询的列中需要自定义列，比如 `count (*) as cnt` , `select name+''+age` 等会出现此符号。

6、根据执行计划做优化

1、对查询进行优化，应尽量避免全表扫描（避免Table Scan），首先应考虑在 `where` 及 `order by` 涉及的列上建立索引。

2、应尽量避免在 `where` 子句中使用 `!=` 或 `<>` 操作符，否则将引擎放弃使用索引而进行全表扫描。对于非聚集索引，如果使用 `!=` 或 `<>` 操作符，非聚集索引不生效；

3、应尽量避免在 `where` 子句中对字段进行 `null` 值判断，否则将导致引擎放弃使用索引而进行全表扫描，如：可以在数据库设计时 `num` 上设置默认值0，确保表中 `num` 列没有 `null` 值，然后这样查询：

4、应尽量避免在 `where` 子句中使用 `or` 来连接条件，否则将导致引擎放弃使用索引而进行全表扫描，如：可以这样查询：

5、下面的查询也将导致全表扫描： 如果要走非聚集索引，`like` 的时候，只能把 `%` 放到后面；会走索引若要提高效率，可以考虑全文检索。

6、`in` 和 `not in` 也要慎用，否则会导致全表扫描，如：如果使用 `in` ,有聚集索引，会根据索引查找，`not in` 聚集索引扫描（不推荐）

```
select id from t where num is null select id from t where num=0 select id from t where num=10 or num=20 select id from t where num=10 union all select id from t where num=20 select id from t where name like '%abc%' select id from t where num in(1,2,3)
```

对于连续的数值，能用 `between` 就不要用 `in` 了： `between` 或者是 `not between` 都会索引查找

7、如果在 `where` 子句中使用参数（变量），也会导致全表扫描。因为SQL只有在运行时才会解析局部变量，但优化程序不能将访问计划的选择推迟到运行时；它必须在编译时进行选择。然而，如果在编译时建立访问计划，变量的值还是未知的，因而无法作为索引选择的输入项。如下面语句将进行全表扫描：可以改为强制查询使用索引：

8、应尽量避免在 `where` 子句中对字段进行表达式操作，这将导致引擎放弃使用索引而进行全表扫描。如：应改为：9、应尽量避免在 `where` 子句中对字段进行函数操作，这将导致引擎放弃使用索引而进行全表扫描。如：应改为：10、不要在 `where` 子句中的 `"=`" 左边进行函数、算术运算或其他表达式运算，否则系统将可能无法正确使用索引。

```
select id from t where num between 1 and 3 select id from t where num=@num select id from t with(index(索引名)) where num=@num select id from t where num/2=100 select id from t where num=100*2 select id from t where substring(name,1,3)='abc' --name以abc开头的id select id from t where datediff(day,createdate,'2005-11-30')=0 --'2005-11-30'生成的id select id from t where name like 'abc%' select id from t where createdate>='2005-11-30' and createdate<'2005-12-1'
```

11、在使用索引字段作为条件时，如果该索引是复合索引，那么必须使用到该索引中的第一个字段作为条件时才能保证系统使用该索引，否则该索引将不会被使用，并且应尽可能的让字段顺序与索引顺序相一致。

12、不要写一些没有意义的查询，如需要生成一个空表结构：

这类代码不会返回任何结果集，但是会消耗系统资源的，应改成这样：

13、很多时候用 exists 代替 in 是一个好的选择：用下面的语句替换：

14、并不是所有索引对查询都有效，SQL是根据表中数据来进行查询优化的，当索引列有大量数据重复时，SQL查询可能不会去利用索引，如一表中有字段sex，male、female几乎各一半，那么即使在sex

上建了索引也对查询效率起不了作用。

15、索引并不是越多越好，索引固然可以提高相应的 select 的效率，但同时也降低了 insert 及 update

的效率，因为 insert 或 update 时有可能会重建索引，所以怎样建索引需要慎重考虑，视具体情况而定。一个表的索引数最好不要超过6个，若太多则应考虑一些不常使用到的列上建的索引是否有必要。

16、应尽可能的避免更新 clustered 索引数据列，因为 clustered 索引数据列的顺序就是表记录的物理存储顺序，一旦该列值改变将导致整个表记录的顺序的调整，会耗费相当大的资源。若应用系统需要频繁更新 clustered 索引数据列，那么需要考虑是否应将该索引建为 clustered 索引。

17、尽量使用数字型字段，若只含数值信息的字段尽量不要设计为字符型，这会降低查询和连接的性能，并会增加存储开销。这是因为引擎在处理查询和连接时会逐个比较字符串中每一个字符，而对于数字型而言只需要比较一次就够了。

18、尽可能的使用 varchar/nvarchar 代替 char/nchar，因为首先变长字段存储空间小，可以节省存储空间，其次对于查询来说，在一个相对较小的字段内搜索效率显然要高些。

19、任何地方都不要使用 select * from t，用具体的字段列表代替“*”，不要返回用不到的任何字段。

```
select col1,col2 into #t from t where 1=0 create table #t(...) select num from a where num in(select num from b) select num from a where exists(select 1 from b where num=a.num)
```

20、尽量使用表变量来代替临时表。如果表变量包含大量数据，请注意索引非常有限（只有主键索引）。

21、避免频繁创建和删除临时表，以减少系统表资源的消耗。

22、临时表并不是不可使用，适当地使用它们可以使某些例程更有效，例如，当需要重复引用大型表或常用表中的某个数据集时。但是，对于一次性事件，最好使用导出表。

23、在新建临时表时，如果一次性插入数据量很大，那么可以使用 select into 代替 create table，避免造成大量 log，以提高速度；如果数据量不大，为了缓和系统表的资源，应先create table，然后 insert。

24、如果使用到了临时表，在存储过程的最后务必将所有的临时表显式删除，先 truncate table，然后 drop table，这样可以避免系统表的较长时间锁定。

25、尽量避免使用游标，因为游标的效率较差，如果游标操作的数据超过1万行，那么就应该考虑改写。

26、使用基于游标的方法或临时表方法之前，应先寻找基于集的解决方案来解决问题，基于集的方法通常更有效。

27、与临时表一样，游标并不是不可使用。对小型数据集使用 FAST_FORWARD 游标通常要优于其他逐行处理方法，尤其是在必须引用几个表才能获得所需的数据时。在结果集中包括“合计”的例程通常要比使用游标执行的速度快。如果开发时间允许，基于游标的方法和基于集的方法都可以尝试一下，看哪一种方法的效果更好。

28、尽量避免向客户端返回大数据量，若数据量过大，应该考虑相应需求是否合理。

29、尽量避免大事务操作，提高系统并发能力。

二十四、性能优化

数据库性能瓶颈-IO瓶颈

IO瓶颈：

第一种：磁盘读IO瓶颈，热点数据太多，数据库缓存放不下，每次查询会产生大量的IO，降低查询速度-

第二种：网络IO瓶颈，请求的数据太多，网络带宽不够

数据库性能瓶颈-CPU瓶颈

第一种：SQL问题：如SQL中包含join,group by, order by, 非索引字段条件查询等，增加CPU运算的操作->SQL优化，建立合适的索引，在业务Service层进行业务计算（不建议在数据库中对数据进行操作）。

第二种：单表数据量太大，查询时扫描的行太多，SQL效率低，增加CPU运算的操作。

选择性处理，没有一劳永逸的方案。需要结合业务的诉求来支持。某一个业务支持的查询快，侧重于去做查询额。某一块业务要求增删改要快，侧重于做增伤改的提高性能。

增删改查四项；

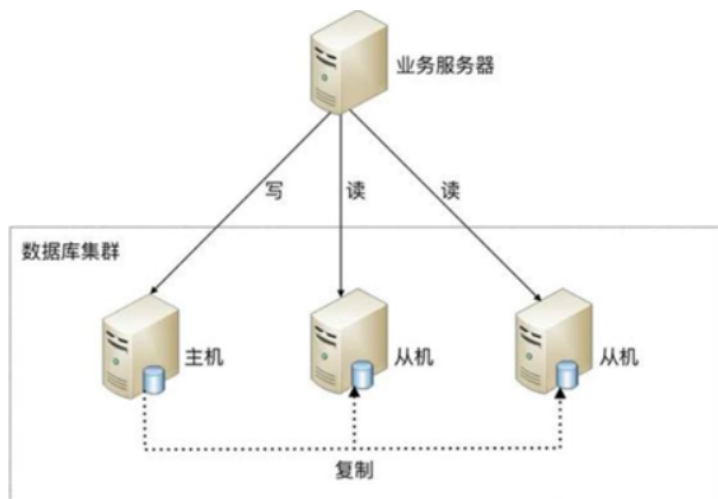
两大项：读取【查询】和写入【增删改】。

数据库操作二八原则

数据库二八原则：**80%的数据库操作是查询，20%的数据库操作是增删改；**

结合业务，有限解决查询的问题，让跟多的服务器来承担查询功能。

读写分离：把数据库的写入【增删改】，读取【查询】分开处理。查询操作占大部分。独立出来让更多的服务器参与查询。



- 1、主从库之间，参与做主从复制的数据表的结构必然一致
- 2、主从库，保存的数据量也算是一致
- 3、通过数据库的【日志】、【快照】来恢复
- 4、必然不是通过Sql语句~
- 5、数据同步——必然有延迟~~【有解决方案~】

读写分离的四种方式

- 1、快照发布：发布服务器按预定的时间间隔向订阅服务器发送已发布数据的快照。
- 2、事务发布【比较常见的，实时性会好些，也会延迟】：在订阅服务器收到已发布数据的初始快照后，发布服务器将事务流式传输到订阅服务器。
- 3、对等发布：对等发布支持多主复制。发布服务器将事务流式传输到拓扑中的所有对等方。所有对等节点可以读取和写入更改，且所有更改将传播到拓扑中的所有节点。
- 4、合并发布：在订阅服务器收到已发布数据的初始快照后，发布服务器和订阅服务器可以独立更新已发布数据。更改会定期合并。Microsoft SQL Server Compact Edition 只能订阅合并发布

配置实操和注意事项

- 1、参与发布订阅的数据库表**必须包含主键**
- 2、只能在局域网内做发布订阅
- 3、如果使用SqlServer代理，则必须设置代理为自动启动
- 4、发布快照的文件夹必须要共享、权限放开

快照发布配置

事务发布配置

对等发布配置

要求：

- 1、数据表必须有主键--有主键才能参与数据同步
- 2、需要开启Sql Server Browser 和 Sql Server 代理

